ChancaDEM: A Crusher-Comminution Studio and a Held-Out Evaluation of Cone-Crusher Models on Ten Industrial Surveys

Felipe Santibáñez-Leal
CAOS open-research programme, Santiago, Chile (fsantibanez@gmail.com)

Technical report · v0.12.001, 4 November 2026 · Licence CC BY 4.0
CAOS open-research programme · CAOS_ChancaDEM, crusher comminution studio
Version DOI `10.5281/zenodo.21518356` · concept DOI (always latest) `10.5281/zenodo.21518356`
Not peer reviewed. Source code, data and computational records: `github.com/fsantibanezleal/CAOS_ChancaDEM`

Abstract

A cone crusher turns a feed into a product whose size, throughput and power are set by four levers: the closed-side setting, the eccentric throw, the eccentric speed and the feed size distribution. In Chilean copper mining the concentrator, where crushing and grinding sit, took 55.7% of the sector's electricity in 2024, so the setting of a crusher is an energy decision. ChancaDEM is an open studio that couples a Whiten population balance with an Andersen and Napier-Munn classification, a JKMRC t_{10} breakage function, a reduced Evertsson capacity model, Bond's power law and Morrell's size-specific energy in one engine written in Python and in TypeScript, runs it in the browser, and holds it against ten measured surveys of a Metso HP500 secondary cone crusher on itabirite iron ore (Rocha et al., 2024, CC BY). Every fitted quantity is refit without the held-out survey (strict leave-one-survey-out), with a constant-mean baseline, a label-shuffle control and a nested 80% predictive band. A one-feature line on the setting reaches a held-out mean absolute percentage error of 13.67% on throughput and 12.78% on power; the calibrated engine reaches 12.09% on throughput and 25.94% on power, because its Bond draw is nearly flat (196 to 237 kW) against measured values of 187 to 355 kW. CrusherBayes, the Bayesian calibration with a Kennedy and O'Hagan structural discrepancy that the product proposes, reaches 12.00% and 14.31% with 80% intervals of coverage 0.80 and 0.90 that are 7% and 44% narrower than the band of the calibrated engine; it does not beat the line on power. The surveys identify the capacity base (posterior over prior standard deviation 0.45), the noise and the discrepancy slopes on the setting, and leave the classification and breakage scalars at their prior. A learned surrogate emulates the engine at R^2 from 0.979 to 0.998 on an independent draw of 2196 points.

Keywords. Comminution, cone crusher, population balance, Whiten model, Bond, Morrell, Bayesian calibration, structural discrepancy, leave-one-out validation, neural surrogate, mineral processing.

1 Introduction

1.1 Crushing as an energy and capacity decision

Crushing is the first stage of comminution, and its setting decides how much size reduction is left to the grinding mill downstream. In Chilean copper mining the concentrator, the plant that holds the crushing and grinding circuits, took 56,970 TJ of electricity in 2024, 55.7% of the 102,307 TJ the sector used, and the national copper commission states that this electricity goes especially to the crushing and grinding operations [1]. The electricity per tonne of fine copper in concentrates rose from 8,946 to 13,079 MJ between 2010 and 2024, an increase of 46.2%, while the national average ore grade fell from 0.75% to 0.62%; the same report ties the rise to deeper deposits and harder rock that directly affect comminution [1]. The projection of the same agency for 2025 to 2034 has the electricity of the copper sector growing by 20.2%, from 27.6 to 33.2 TWh, against 8.3% for fine copper, with concentration alone at 18.1 TWh in 2034 [2]. Outside Chile, an analysis of global metallic mining reports a decline of average copper ore grades of about a quarter over ten years together with a rise in energy use larger than the rise in production [3], and the International Energy Agency names declining ore grades among the reasons for an implied supply deficit of copper in 2035 under its stated-policies scenario [4].

The energy is not spent efficiently. The United States Department of Energy bandwidth study of mining (data year 2000) attributes 494 of 1,246 trillion Btu per year to grinding and 52 to crushing, and records that one percent is the most common estimate of the efficiency of grinding against about fifty percent for crushing on a mechanical basis, with the warning that the estimates depend on the metric [5]. An exergy analysis of the comminution of a central-Chile copper ore in a laboratory ball mill finds an efficiency of about three percent against the Hukki and Morrell benchmark [6]. A coarser crusher product passes work to grinding, the more energy-intensive step, so the setting of a crusher is a decision about the energy of the whole circuit and not only about the crusher.

1.2 Who decides, and how it is decided today

Three roles carry the decision. The comminution or process engineer chooses the flowsheet, the machines and their settings at design and at each review; the plant operator and the control-room supervisor move the setting and the speed day to day; the maintenance and reliability team decides when the liners are changed. Four families of tools serve them. Empirical energy laws, of which the Bond work index is the best known, size machines and estimate energy and have been used for about sixty years in mill design, but they cannot give a full circuit mass balance or a size distribution [7], and design test data and benchmarks are often modified, worn or misapplied in practice [8]. Population-balance crusher models in the Whiten and Andersen tradition fit industrial surveys closely when calibrated: a review classifies 61 works on cone-crusher models and control [9], a calibration with industrial data from two iron-ore mines treats every entry of the breakage matrix as a fitted parameter [10], and a fit-for-purpose model of a Metso HP500 secondary cone was calibrated on ten surveys over several settings and feeds [11]. Discrete-element simulation reproduces the coarse part of the product and the control signals of a crusher with relatively good correspondence [12, 13], and particle-replacement models carry breakage into industrial compression crushers [14, 15], at a cost that one bonded-particle study of a quarter section of an industrial cone put at 172 and 404 CPU hours for about four seconds of operation [16]. Automatic setting regulation and real-time control raised the saleable yield of a crushing stage by 3.5% (setting algorithm) and 4.2 to 6.9% (speed algorithm) at full-scale aggregate plants [17], and the common opinion quoted by the same group is a yield gain of 20 to 25% when liner wear is compensated [18]; the optimised chamber geometry changes as the liners wear [19].

The practical gap that this report addresses lies between the first two families. A calibrated population balance is judged, in the work cited above, on the surveys it was fitted to; ten surveys of one machine leave little room for a free-form correction; and the uncertainty of a prediction for a survey the model has not seen is not reported. A decision maker who reads a calibrated model at a new setting needs the error the model makes out of sample, a band that covers the measured value at the stated rate, and a baseline that says how much of the skill is physics and how much is the setting alone.

1.3 What the product does

ChancaDEM is a static web studio with an offline pipeline behind it. A reader picks one of 27 crushing cases (seventeen circuit cases, fourteen across five machine classes and three controls, and the ten measured HP500 surveys of Rocha et al. [11]), changes the closed-side setting, the throw, the speed, the feed and the ore, and reads the product gradation, the throughput and the power from one engine: a Whiten population balance [20, 21] with an Andersen and Napier-Munn classification [22], a JKMRC t_{10} energy-fineness relation with an Austin appearance function [21, 23], a reduced form of the Evertsson capacity model [26], the Bond power law [27], and, held out beside it, the Morrell size-specific crusher energy [28, 29]. The engine runs in the browser in a TypeScript port held to the canonical Python engine by a shared reference of 124 operating points. Two learned models, a multilayer perceptron that emulates the engine in the manner of Koh et al. [30] and a denoising autoencoder that scores how far a result lies from the training manifold, run in the browser through ONNX Runtime. A projection of the hours until the product leaves a size target as the setting drifts open, with the drift rate supplied by the user, completes the operating view.

1.4 Contributions

The report documents every method of the product and the evaluation that holds them to measured data. Its contributions are:

1. A complete account of a crusher engine (feed, breakage, classification, the Whiten solve, capacity,

power, regimes and an envelope check) with its equations, constants and numerical scheme, implemented twice and held equal to a largest relative difference of 2.3×10^{-12} over 124 reference points.

2. A strict leave-one-survey-out evaluation of nine predictors on the ten HP500 surveys, the same folds for every predictor, with a constant-mean baseline, a label-shuffle control, the optimism of a leaky evaluation measured beside the strict one, and a nested 80% predictive band whose coverage is computed out of sample.

3. The finding that a line on the closed-side setting alone is the strongest power predictor on these data (12.78% held-out error), that the Bond draw of the calibrated engine carries little of the measured power variation, and that Morrell's energy with a fitted crushing work index does not beat the line.

4. CrusherBayes, a Bayesian calibration of the engine's eight fitted scalars with a structural discrepancy per target and a differential-evolution sampler, whose held-out intervals are calibrated and narrower than the residual band, and whose posterior shows which scalars ten surveys identify; the method is reported in full in its own manuscript [36] and summarised here.

5. An inventory of thirteen data contracts with their schemas, producers, consumers and missing-data behaviour, which lets a reader replace any component and keep the rest.

1.5 Structure of the report

Section 2 states the problem as an estimation problem with its inputs, outputs and assumptions, and Section 3 collects the notation. Section 4 describes the data sources and every data contract. Section 5 describes the pipeline, the engine, the web and the dormant interface. Section 6 gives one subsection per method, from the feed distribution to CrusherBayes. Section 7 states the cases, the metrics, the baselines and the acceptance criterion of each method; Section 8 reports every result the studio shows; Section 9 explains how the results are known to be correct. Sections 10 and 11 discuss what the results mean and what bounds them, Section 12 lists the commands that rebuild every table and figure, and Section 13 concludes. The appendices hold the parameter tables, the derivation of the integrated discrepancy of CrusherBayes, and the field tables of every contract schema. A reader interested only in the evaluation on the measured surveys can read Sections 6.9 to 6.11, 6.16 and 8.2 to 8.6; a reader who wants to reimplement the engine needs Sections 6.1 to 6.7 and the appendices.

2 Problem formulation

2.1 The forward problem: an operating point to a product

An operating point of a crusher is the vector

$$\mathbf{u} = (\mathrm{m},\ s,\ T,\ N,\ x_{63},\ n,\ A{\times}b,\ W_i)\,, \tag{1}$$

with m the machine class (gyratory, jaw, secondary cone, tertiary cone or short-head cone), s the closed-side setting (CSS) in millimetres, T the eccentric throw in millimetres, N the eccentric speed in revolutions per minute, x_{63} and n the characteristic size (mm) and the uniformity exponent of a Rosin-Rammler feed, $A \times b$ the JKMRC drop-weight ore competence (dimensionless, lower is harder) and W_i the Bond work index in kWh/t. The engine is a deterministic map

$$\mathcal{F}:\ \mathbf{u} \mapsto (\mathbf{p},\ P_{80},\ Q,\ P,\ \mathbf{r})\,, \tag{2}$$

where $\mathbf{p} \in [0,1]^K$ is the product mass per size class on a fixed grid of $K = 28$ classes, P_{80} the size in millimetres that 80% of the product mass passes, Q the throughput in t/h, P the power in kW, and $\mathbf{r}$ a set of qualifiers: the regime, a validity flag, the nip angle against its grip limit and the envelope codes. The forward problem is to evaluate $\mathcal{F}$ fast enough to be re-run at every move of a control in a browser, with every output in the unit of its observable, and with the trends a process engineer expects: a tighter setting gives a finer product and less capacity, the capacity rises with speed to an optimum and falls past it, and a wider throw coarsens the product at a fixed setting.

The constraints that every evaluation satisfies are physical. Mass is conserved, $\sum_k p_k = \sum_k f_k = 1$ within the round-off of the solve; the classification probability lies in $[0,1]$ and is monotone non-decreasing in size; broken mass reports only to finer classes; the throughput and the power are finite and non-negative

at every valid point. A point at which the setting is at or above the 80 percent passing size of the feed, or at or above its top size, is not answered with a confident number: it is flagged as a pass-through or an invalid point.

2.2 The inverse problem: calibration on ten surveys

The measured data are ten surveys $\mathcal{D} = \{(s_i, f_{80,i}, Q_i, P_i)\}_{i=1}^{10}$ of one Metso HP500 secondary cone crusher on itabirite iron ore [11]: the closed-side setting, the feed f_{80}, the measured feed rate (t/h) and the motor power (kW). On this machine the engine depends on a vector of fitted scalars $\boldsymbol{\theta}$ (the capacity base, the work index of the blend, the two constants of the power form, two multipliers and the exponent of the classification, and a shift of the t_{10} relation), so the model of a survey is

$$y_{t,i} = f_t(\boldsymbol{\theta};\ s_i, f_{80,i}) + e_{t,i}, \qquad t \in \{Q, P\}, \tag{3}$$

where f_t is the engine output for target t at the survey's setting and feed with the machine's nominal throw and speed, and $e_{t,i}$ the error. The quantity estimated is the predictive distribution of (Q_j, P_j) for a survey j that the fit has not seen, and the objective is twofold: a small held-out error of its centre, measured by the mean absolute percentage error, and an 80% interval that contains the measured value in about 80% of the held-out surveys while being as narrow as possible. The product size is not a target, because the size distributions of the surveys are printed only as figures in the source [11]; the engine's P_{80} at a survey is a model output.

2.3 Assumptions and where each comes from

Each assumption below bounds the results; Section 11 returns to them.

1. The chamber is one breakage zone with a classification function and a breakage function, in steady state and choke fed: the Whiten crusher model [20, 21]. No spatial resolution of the chamber, no liner wear and no feed segregation enter the engine.
2. A feed is a Rosin-Rammler distribution [24]. For a survey the feed is rebuilt from the measured f_{80} with a uniformity exponent $n = 0.9$, an assumption of the product, and $x_{63} = f_{80}/(\ln 5)^{1/n}$.
3. On the survey lane the throw (34 mm) and the speed (360 rpm) are machine constants of the product, because the surveys vary only the setting and the feed; the constants of the classification, the t_{10} relation, the capacity base and the power form are those of the calibration of Rocha et al. [11]. The correspondence of each constant to a numbered table of that paper is carried from the product's code and could not be re-read at the publisher (Section 11).
4. The seventeen circuit cases use per-machine constants that are typical of their class and reproduce trends, not a vendor's numbers; only the HP500 lane is calibrated.
5. The two learned models are trained on, and judged against, the engine itself; they emulate the engine, not a plant.
6. The ten surveys are exchangeable for the purpose of leave-one-out evaluation: no order in time is used.

3 Notation

Table 1 collects every symbol of the report with its unit. Four conventions hold throughout and are worth stating before the methods, because the engine depends on them and a reimplementation that changes one of them gives different numbers.

Size classes. The sieve grid runs from coarse to fine. Edge e_k is $256/(\sqrt{2})^k$ mm for $k = 0, \ldots, 28$, so there are $K = 28$ classes and the finest edge is $256/2^{14} = 0.015625$ mm. Class k lies between the upper edge e_k and the lower edge e_{k+1} and has the geometric mid-size $\bar{d}_k = \sqrt{e_k e_{k+1}}$. Because the classes run coarse to fine, a matrix whose entry (i, j) moves mass from class j to class i is strictly lower triangular when mass only moves to finer classes, and the breakage matrix of Section 6.2 has this form. Vectors indexed by class are bold ($\mathbf{f}$, $\mathbf{p}$, $\mathbf{c}$); a diagonal matrix built from a vector is the upright capital ($\mathrm{C} = \mathrm{diag}(\mathbf{c})$).

Table 1: Notation. Every symbol used in the equations of the report, its meaning and its unit; a dash marks a dimensionless quantity. Symbols local to one derivation are defined where they appear.

Symbol	Meaning	Unit
K, k	number of size classes (28) and class index, coarse to fine	–
e_k, $\bar{d}_k$	upper edge and geometric mid-size of class k	mm
$\mathbf{f}$, $\mathbf{p}$	feed and product mass per class (each sums to one)	–
$R(x)$	cumulative fraction passing size x	–
x_{63}, n	Rosin-Rammler characteristic size and uniformity exponent	mm, –
F_{80}, f_{80}	feed 80% passing size, on the engine grid and as measured	mm
P_{80}, P_{50}, P_{20}	product sizes passing 80, 50 and 20%	mm
s (CSS), T, OSS	closed-side setting, eccentric throw, open-side setting $s+T$	mm
T_{ref}	design throw of a machine class	mm
N, N_{opt}	eccentric speed and the speed of maximum capacity	rpm
A, b, $A\times b$	JKMRC ore competence parameters and their product	%, t/kWh, –
E_{cs}	specific comminution energy per nip	kWh/t
t_{10}	fraction of progeny passing one tenth of the parent size	– or %
$B(u)$, φ, γ, β	Austin cumulative appearance function of relative size u, its weight and slopes	–
B, B_{ij}	breakage matrix, fraction of broken class-j mass reporting to class i	–
$C(d)$, $\mathbf{c}$, C	classification probability of size d, its vector and diagonal matrix	–
K_1, K_2, K_3	classification sizes and exponent	mm, mm, –
$\mathbf{x}$	internal mass flow of the breakage zone per unit feed	–
κ	conditioning proxy of I − BC	–
Q, Q_{ref}, Q_T	throughput, reference capacity, calibrated capacity base	t/h
g_{ref}	reference discharge gap of a machine class	mm
W_i, W	Bond work index and Bond specific energy	kWh/t
P_d, P_c, P	Bond net draw, current-based calibrated power, power	kW
ζ, P_n	slope and no-load term of the current-based power form	–, kW
M_{ic}, W_c, S_c	Morrell crushing work index, crusher specific energy, hardness correction	kWh/t, kWh/t, –
x_1, x_2	feed and product 80% passing sizes in the Morrell equation	μm
α, μ_f	nip angle and friction coefficient of the grip limit	deg, –
$\mathcal{D}$, n	set of surveys and their number (10)	–
$\mathcal{T}_{-i}$	training surveys of the fold that holds out survey i	–
$y_{t,i}$, $\hat{y}_{t,i}$	measured value and held-out prediction of target t at survey i	t/h or kW
σ, σ_{-i}	residual standard deviation, and that of the nine other surveys	t/h or kW
z_{CSS}, $z_{f_{80}}$	setting and feed size standardised on the training surveys	–
$\boldsymbol{\theta}$	CrusherBayes engine scalars and log noise scales	mixed
δ_t, $\mathbf{d}_t$, τ_t	discrepancy of target t, its coefficients and their prior scale	t/h or kW
σ_t	observation noise of target t	t/h or kW
$\hat{\tau}$	integrated autocorrelation time of a chain	steps
Δs, r, t_{out}	setting opening to a target, drift rate, hours to the target	mm, mm/h, h

Gradations. A gradation is stored either as mass per class (a vector that sums to one) or as cumulative fraction passing at each edge (a vector that starts at one and decreases). The size at a passing fraction (P_{80}, P_{50}, P_{20}, F_{80}) is read by linear interpolation in the logarithm of the size between the two edges that bracket the fraction. Capital F_{80} is the feed 80% passing size as the engine reads it on its grid; lower-case f_{80} is the measured value of a survey. They differ when the feed is coarser than the 256 mm top of the grid.

Units. Sizes are in millimetres in the engine and in micrometres inside the two energy laws (Bond and Morrell), as those laws are printed; throughput is in tonnes per hour; power in kilowatts; specific energy in kWh per tonne; speed in revolutions per minute; angles in degrees. Percentages are written with the percent sign; a difference between two percentage errors is written in points.

Survey indices and folds. Survey i runs from 1 to $n = 10$ in the order of the published table. A fold of the leave-one-survey-out evaluation is indexed by its held-out survey; $\mathcal{T}_{-i}$ is the set of the other nine surveys, and a quantity fitted on them carries the subscript $-i$. A standardised variable is written z, with the mean and standard deviation of the training surveys of the fold. In CrusherBayes, $\boldsymbol{\theta}$ is the vector of the eight engine scalars and the two noise scales, and $\mathbf{d}_t$ the three discrepancy coefficients of target t.

4 Data and data contracts

4.1 Sources

Four data sources enter the product; a fifth, the sector statistics of the introduction, is context and enters no computation.

The ten HP500 surveys. Table A1 of Rocha, Campos, Silva and Tavares [11] (Minerals, MDPI, open

Table 2: The ten HP500 surveys of Rocha et al. [11] as committed, with the calibrated engine evaluated at each measured setting and feed f_{80} (throw 34 mm, speed 360 rpm): its grid F_{80}, product P_{80}, throughput Q, Bond draw and the published current-based power $1.30\,P_d + 110$. Generated from `backbone.json`.

	Measured (Table A1)				Calibrated engine				
Survey	CSS (mm)	f_{80} (mm)	Feed (t/h)	Power (kW)	F_{80} (mm)	P_{80} (mm)	Q (t/h)	Bond (kW)	P_c (kW)
1	54	88	1639	209	84.86	32.56	1208.9	229.5	408.3
2	54	84	1267	212	81.70	31.36	1208.9	233.7	413.9
3	48	47	1240	262	47.15	21.23	1106.7	224.9	402.3
4	48	88	1034	349	84.86	30.80	1106.7	225.7	403.4
5	48	62	1124	325	61.66	24.54	1106.7	234.8	415.2
6	40	79	813	355	77.55	26.54	970.5	222.5	399.3
7	38	140	1050	306	118.94	36.60	936.5	196.1	365.0
8	44	123	1138	325	109.85	35.69	1038.6	212.7	386.6
9	40	81	820	350	79.24	26.97	970.5	221.6	398.0
10	55	81	1334	187	79.24	30.72	1225.9	237.5	418.7

access under CC BY 4.0, `https://doi.org/10.3390/min14090919`) gives, for ten surveys of a Metso HP500 secondary cone crusher at an itabirite iron-ore operation, the closed-side setting, the feed 80% passing size f_{80}, the solid feed rate and the motor power estimated from the measured current. The abstract of the paper, read through its OpenAlex record because the publisher page refused automated access, describes surveys collected over several years, an Andersen-Whiten model implemented in an integrated extraction simulator and a fit-for-purpose throughput model; the exact dates of the surveys are not printed in the material available to the product. The table is transcribed into `data/examples/hp500-surveys.csv` (ten rows, five columns, 210 bytes) and validated by the `hp500-survey` contract below. Table 2 lists the ten rows beside the calibrated engine's outputs at each of them. Across the surveys the setting spans 38 to 55 mm, the feed f_{80} 47 to 140 mm, the feed rate 813 to 1639 t/h and the power 187 to 355 kW. The measured power is correlated with the setting ($r = -0.82$) and with the feed rate ($r = -0.85$), and the feed rate with the setting ($r = +0.81$), so a tighter setting draws more power at a lower feed rate (Figure 1). The paper prints the product size distributions only as figures, so no product size is measured in the product's data.

The circuit cases. The seventeen circuit cases are the product's own table `data/examples/params.csv` (MIT licence, seventeen rows and nine columns, one operating point each): two gyratory, two jaw, six secondary cone, two tertiary cone and two short-head cases, and three controls, a pass-through case (CP1, a setting of 160 mm against a feed F_{80} of 126 mm), an invalid case (CI1, a setting of 280 mm, above the top size of the grid) and a synthetic anchor (CK1). The case CK1 is synthetic because the only other open industrial cone calibration found, Duarte et al. [10], reports seven operating points of an HP400 and a CH660 with the setting, the throughput and the feed F_{80} only, without throw, speed, ore properties or power, and with its size data as plots. Table 3 lists the 27 cases as baked.

The engine sweeps of the learned models. The training data of the surrogate and the autoencoder are generated, not measured: a Latin-hypercube sweep of the canonical engine, 1400 points per machine class over that class's own ranges (Table 20), labelled by the engine, with degenerate draws dropped (a setting at or above the feed top size, an ill-conditioned matrix, a reduction ratio below 1.02 or a non-positive P_{80}), which leaves 6430 training points. An independent second draw with another seed (67890, 480 points per class) leaves 2196 held-out points.

Vendor bounds. Two sourced warnings of the engine read published vendor documents: the minimum closed-side setting of the HP Series standard (16 mm) and short-head (6 mm) chambers and the typical reduction ratios of 3 to 5 and 2 to 4 from the Metso HP wear-parts application guide [40]; the smallest minimum setting of the Nordberg C Series jaw crushers (40 mm, the C80) [41]; and the reduction ratio of 3 to 4 for jaw and gyratory crushers from the Metso handbook [42]. No vendor minimum setting was found for a primary gyratory, so that check is skipped.

The splits and why. The circuit cases are not fitted and need no split. The ten surveys are too few for a held-out block, so every fitted quantity is evaluated by leave-one-survey-out: ten folds of nine training surveys and one held-out survey, the same folds for every predictor (Section 6.11). The learned models are judged on an independent draw rather than a split of the training draw, so that a point near a training point does not flatter the error.

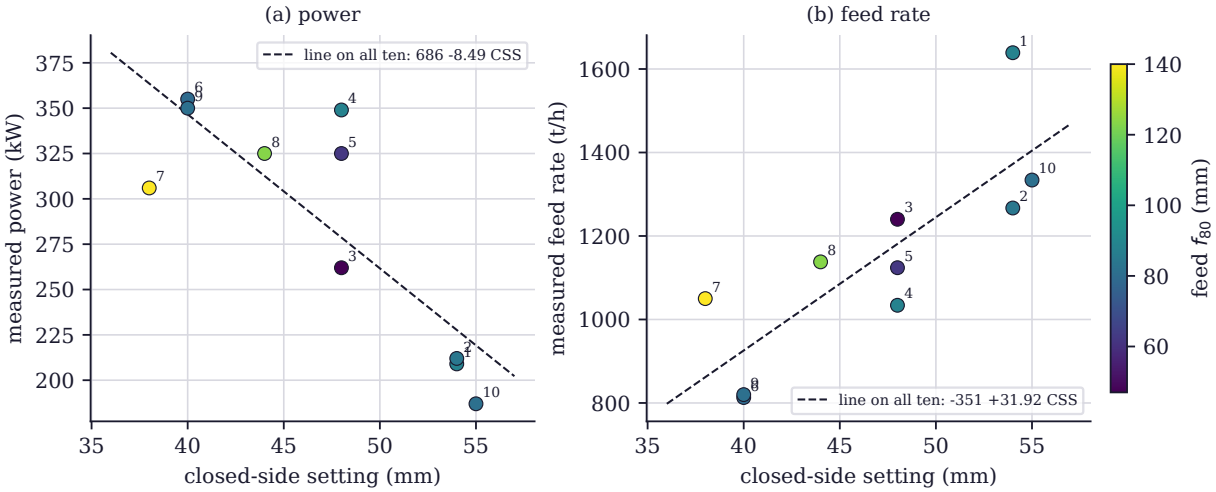


Figure 1: The ten HP500 surveys: (a) measured motor power and (b) measured feed rate against the closed-side setting, coloured by the measured feed f_{80} and numbered as in the published table, with the least-squares line on the setting fitted to all ten surveys.

Table 3: The 27 baked cases, from their manifests and traces: machine, setting, throw and speed, the engine's feed F_{80}, product P_{80}, reduction ratio, throughput, power, regime and validity. The ten HP500 cases run on the calibrated lane.

Case	Machine	CSS (mm)	Throw (mm)	Speed (rpm)	F_{80} (mm)	P_{80} (mm)	F_{80}/P_{80}	Q (t/h)	Power (kW)	Regime	Valid
CI1	secondary cone	280	30	360	126.0	126.0	1.00	4037	0	invalid	no
CK1	secondary cone	25	30	350	116.2	21.7	5.35	547	316	choke	yes
CP1	secondary cone	160	30	360	126.0	112.3	1.12	2395	56	pass-through	no
G01	gyratory	165	30	150	180.3	153.6	1.17	4629	145	choke	yes
G02	gyratory	195	32	160	185.8	170.1	1.09	5414	68	pass-through	no
H01	short-head cone	6	14	500	29.8	4.6	6.55	216	273	choke	yes
H02	short-head cone	10	16	470	41.6	8.6	4.86	300	319	choke	yes
HP500-01	secondary cone	54	34	360	84.9	32.6	2.61	1209	408	choke	yes
HP500-02	secondary cone	54	34	360	81.7	31.4	2.61	1209	414	choke	yes
HP500-03	secondary cone	48	34	360	47.1	21.2	2.22	1107	402	pass-through	no
HP500-04	secondary cone	48	34	360	84.9	30.8	2.76	1107	403	choke	yes
HP500-05	secondary cone	48	34	360	61.7	24.5	2.51	1107	415	choke	yes
HP500-06	secondary cone	40	34	360	77.6	26.5	2.92	970	399	choke	yes
HP500-07	secondary cone	38	34	360	118.9	36.6	3.25	936	365	choke	yes
HP500-08	secondary cone	44	34	360	109.9	35.7	3.08	1039	387	choke	yes
HP500-09	secondary cone	40	34	360	79.2	27.0	2.94	970	398	choke	yes
HP500-10	secondary cone	55	34	360	79.2	30.7	2.58	1226	419	choke	yes
J01	jaw	125	40	300	182.9	126.4	1.45	923	70	choke	yes
J02	jaw	80	36	280	184.2	82.4	2.24	622	100	choke	yes
S01	secondary cone	32	30	360	126.0	27.7	4.55	643	287	choke	yes
S02	secondary cone	40	32	300	147.1	33.8	4.35	754	235	choke	yes
S03	secondary cone	50	30	470	152.0	42.3	3.59	856	334	choke	yes
S04	secondary cone	80	36	360	161.8	70.5	2.30	1341	275	choke	yes
S05	secondary cone	16	26	360	75.2	13.6	5.51	397	273	choke	yes
S06	secondary cone	12	22	360	88.7	9.1	9.80	315	315	choke	yes
T01	tertiary cone	8	16	400	39.0	7.1	5.48	240	228	choke	yes
T02	tertiary cone	10	18	410	45.0	9.6	4.67	285	281	choke	yes

4.2 The contract catalog

Every boundary between two components of the product is a data contract: one JSON Schema file (or, for the interface, one OpenAPI document), an example, a test on the producer side and a test on the consumer side, a stated behaviour when the data are missing, and a version that moves by semantic versioning with the schema. The catalog `docs/contracts/catalog.md` lists thirteen contracts (Table 4); the TypeScript types of the web are generated from the same schema files, so a change to a contract fails the web build until both sides follow it. Missing data are described by one vocabulary of states: *available*, *missing* (the document is absent), *partial* (some files a document names are absent), *stale* (the document was baked by an older version of the product), *training* (a model bundle is absent or fails its checksum, so the slot answers with its baseline) and *provider* (a stand-in serves the contract). The readiness document records the state of every contract for one build, and every view of the web shows it. The subsections below describe each contract; the field tables generated from the schema files are in Appendix C.

Table 4: The thirteen data contracts of the product, generated from the catalog: identifier, version, producing module, schema file and status in the committed build. Their fields are listed in Appendix C.

Contract	Version	Producer (owner)	Schema file	Status
`params-record`	1.0.0	`pipeline/io/contract.py`	`params-record.schema.json`	real
`hp500-survey`	1.0.0	`pipeline/hp500/surveys.py`	`hp500-survey.schema.json`	real
`case-index`	1.0.0	`pipeline/core/manifest.py`	`case-index.schema.json`	real
`case-manifest`	1.1.0	`pipeline/core/manifest.py`	`case-manifest.schema.json`	real
`trace`	1.0.0	`pipeline/core/trace.py`	`trace.schema.json`	real
`hp500-backbone`	1.0.0	`pipeline/hp500/backbone.py`	`hp500-backbone.schema.json`	real
`hp500-loo`	1.2.0	`pipeline/hp500/loo.py`	`hp500-loo.schema.json`	real
`hp500-crusherbayes`	1.0.0	`pipeline/hp500/crusherbayes.py`	`hp500-crusherbayes.schema.json`	real
`surrogate-model`	1.0.0	`pipeline/stages/train.py`	`surrogate-model.schema.json`	real
`anomaly-model`	1.0.0	`pipeline/stages/train.py`	`anomaly-model.schema.json`	real
`model-metrics`	1.0.0	`pipeline/stages/evaluate.py`	`model-metrics.schema.json`	real
`readiness`	1.0.0	`pipeline/core/readiness.py`	`readiness.schema.json`	real
`api`	1.0.1	`app/`	`api.openapi.json`	real

4.3 params-record: one operating point

The `params-record` contract (version 1.0.0, schema `params-record.schema.json`) is the ingestion contract of the circuit cases: one row of `params.csv` as `pipeline/io/contract.py` accepts it. Its nine required fields are the case identifier (string, pattern of one capital letter followed by one to fifteen capitals, digits or hyphens), the machine class (one of five values), the closed-side setting `css_mm` (number, above 0 and at most 600 mm), the throw `throw_mm` (1 to 100 mm), the speed `speed_rpm` (50 to 1000 rpm), the feed characteristic size `feed_x63_mm` (above 0 and at most 2000 mm), the uniformity exponent `feed_m` (0.3 to 4), the ore competence `ore_axb` (10 to 1000) and the Bond work index `ore_wi` (2 to 40 kWh/t). A value is parsed, never coerced: a row the schema refuses is rejected with its reason, and a physically unusual but valid row (a setting at or above the feed's F_{80} or its top size) is accepted and flagged with the engine's own codes, so the controls are baked and shown. The consumers are the bake (`pipeline/pipeline.py`) and the tests of both sides. Missing-data behaviour: a rejected row is dropped with its reason and the bake goes on with the accepted rows, while readiness lists the rejected case identifiers as *partial*; with no source table at all, readiness says *missing* and the bake uses the registry's cases.

4.4 hp500-survey: one measured survey

The `hp500-survey` contract (version 1.0.0) is the ingestion contract of the real data: one row of `hp500-surveys.csv` as `pipeline/hp500/surveys.py` accepts it. Its five required fields are the survey number (integer, 1 to 10, consecutive without a gap), the measured feed f_{80} (number, above 0 and at most 400 mm), the measured closed-side setting (above 0 and at most 200 mm), the measured solid feed rate `feed_tph` (above 0 and at most 5000 t/h) and the crusher power estimated from the measured current `power_kw` (above 0 and at most 1000 kW). The bounds are wide on purpose: they refuse a unit error, such as a power in watts, and a typing error, not a plausible survey. The producer is the transcription of Table A1, its consumers the HP500 cases, the backbone, CrusherCal and CrusherBayes. A kill criterion of the product applies to this contract alone: if a re-transcription of the published table disagrees with the committed file, the real lane is removed from the build until the table is corrected. Missing-data behaviour: with no survey table, readiness says *missing*, the bake skips the ten real cases and the three HP500 documents, the case selector lists no real survey, and the survey views of the web say that the data are not available with the reason; a rejected row is dropped with its reason, its case is not baked and readiness lists the survey number as *partial*.

4.5 case-index: the inventory of a bake

The `case-index` contract (version 1.0.0, `data/derived/manifests/index.json`, schema `chancadem.index/v1`) is the flat inventory the web reads first. It carries the product version that baked it (`engine_version`, pattern `X.XX.XXX`), the number of cases (integer, at least one; 27 in the committed build), the default case (`S01`), one entry per case with its identifier, its title and category in English and Spanish, whether it is real or synthetic and the relative path of its manifest, and the list of the further files the site serves: the five model files, the three HP500 documents and the model metrics. Rules the schema cannot state are checked by the tests: the count equals the number of entries, every path

exists, and the default case is listed. It is produced by `pipeline/core/manifest.py` (`build_index`) and read by the web's case selector, the copy step of the site build (which copies exactly the files the index declares) and the dormant interface. Missing-data behaviour: without an index the site build stops at its copy step, because a site without an inventory would show nothing reliably; the interface answers 503 with the state *missing*. When `engine_version` is older than the product version, readiness says *stale* with both versions and the views show it.

4.6 case-manifest: one baked case

The `case-manifest` contract (version 1.1.0, one file `data/derived/manifests/<case>.json` per case, schema `chancadem.manifest/v1`) is the authoritative record of one case and a pure function of its parameters and seed: it holds no wall-clock time, so a re-bake does not change it. Its twenty required fields include the case identifier, the bilingual title, category and expected band in words, the circuit stage (primary, secondary, tertiary or control), the kind of control (none, negative, invalid or calibration), the case context (the question, the variables and how to read the views), the engine package and version, the operating point exactly as the ingestion contract accepted it with the `calibrated` switch, the seed, the artifact (relative path, format, trace schema, size in bytes and SHA-256), the lane decision of the lane check (live or precompute, with the measured trace size and the budgets of 1500 ms and 256 KiB), the flags as stable codes, the result metrics, which model answered per model slot (the released model or its baseline), the expected range of each metric, and the source. A real case adds the survey row as measured; its metrics add the signed percentage error of the throughput and the power against the measured values (-10.75% and $+53.55\%$ for survey 3). The producer is `pipeline/core/manifest.py`; the consumers are the web workbench and the interface. Missing-data behaviour: a manifest the index lists but that is absent is reported by readiness as *partial* with its path, the selector marks that case as not available and the other cases work, and the interface answers 404 for it; staleness follows the index.

4.7 trace: the replay artifact

The `trace` contract (version 1.0.0, `data/derived/<case>/trace.json`, schema `chancadem.trace/v1`) holds what the engine computed at the operating point of a case: the 29 sieve edges and 28 mid-sizes in millimetres, the feed and product cumulative passing at each edge and mass per class (fractions in $[0, 1]$), the classification $C(d)$ at each mid-size, the t_{10} of the appearance function in percent (0 to 100), a summary of scalars (F_{80}, P_{80}, P_{50} and P_{20} in mm, the reduction ratio, the passing at 1, 4, 8, 16 and 32 mm, the throughput in t/h, the power in kW, the specific energy in kWh/t, the open-side setting, the nip angle and its limit in degrees, the regime, the validity and envelope flags, the mass closure and the conditioning proxy, which may be the string `Infinity` for a singular solve), a speed sweep of 31 points from 0.5 to 1.5 times the case speed (throughput and power) and a setting sweep (product P_{80}, throughput and power). The largest trace of the committed build is 4,127 bytes. It is produced by `pipeline/core/trace.py` and read by the web's replay curves and by the parity test of the TypeScript engine, which re-runs each case and compares with the trace to the six significant digits it is written with. Missing-data behaviour: an artifact a manifest names but that is absent is listed by readiness as *partial*; the replay views of that case say that the data are not available, while the live lane still runs from the manifest's parameters, and the interface answers 404.

4.8 hp500-backbone: the calibrated engine at each survey

The `hp500-backbone` contract (version 1.0.0, `data/derived/hp500/backbone.json`, schema `chancadem.hp500.backbone/v1`) records the calibrated engine evaluated at each of the ten surveys. Its fields are the source (paper, DOI, licence and tables), the engine version, a note, the number of surveys and, per survey, its identifier (pattern `HP500-NN`), its number, the measured block (setting, f_{80}, feed rate and power, in mm, mm, t/h and kW) and the engine block (product P_{80} in mm, throughput in t/h, Bond power and calibrated power in kW, and the engine's own F_{80} on its grid in mm). Its note states the two facts a reader must not miss: the product size is a model output, graded only by a physical relation of 0.35 to 1.1 times the setting, and the power constants 1.30 and 110 kW of the source apply to the paper's size-specific net power and not to the Bond draw of the engine. It is produced by `pipeline/hp500/backbone.py`; CrusherCal, CrusherBayes and the web's validation views read it, and a test checks that the calibrated engine at each survey equals the committed document. Missing-data behaviour: when it is absent, readiness says *missing*, the survey views of the web say that the validation is not available with the reason, and the ten real cases still replay their own manifests.

4.9 hp500-loo: CrusherCal

The `hp500-loo` contract (version 1.2.0, `data/derived/hp500/loo.json`, schema `chancadem.hp500.loo/v1`) is the leave-one-survey-out evaluation of Section 6.11. It holds the source, the engine version, the number of surveys, the seed (42), the configuration (the residual clip fraction 0.20, the ridge penalty 1.0, the two features, the two primary targets and the secondary one, and the Morrell settings $K_s = 55$, $K_2 = 1.0$, closed circuit), and per target (throughput, power and the self-consistency target P_{80}) the measured values and two blocks, strict and leaky, each with the MAPE in percent, RMSE in the target unit, R^2 and largest deviation in percent of every predictor, the held-out predictions per survey, and, on the strict block, the band: σ of the held-out residuals, the half-width, the nominal and in-sample coverage, the relative half-width in percent, the nested coverage with the per-survey σ_{-i} and whether each survey is covered. The power block adds the per-fold Morrell fits. Two further blocks give the in-sample power and throughput parity. The version moved to 1.2.0 when the setting-only line and the two Morrell variants were added. The producer is `pipeline/hp500/loo.py`; the consumers are CrusherBayes, the web's Benchmark and Experiments views, the band shown on the real-survey gauges and the lower-bound band of the synthetic secondary-cone cases. Missing-data behaviour: readiness says *missing*, the gauges show the measured values without a band and say why, and the validation views say that the data are not available.

4.10 hp500-crusherbayes: the posterior and its held-out scores

The `hp500-crusherbayes` contract (version 1.0.0, `data/derived/hp500/crusherbayes.json`, schema `chancadem.hp500.crusherbayes/v1`) is the output of the offline sampler of Section 6.16. It holds the source, the engine version, the number of surveys, the seed, a SHA-256 over the survey table, the sampler size, the seed and the priors (so a reader can tell which inputs produced it), the method (sampler name and version, moves, 20 walkers, 7000 steps, 1000 discarded, thinning 10, 20 predictive draws per sample, the likelihood, the discrepancy and noise priors and the definition of the interval), the eight priors with their nominal value, scale, centre, standard deviation, truncation and source, the convergence block (criterion, factor 50, and per posterior the largest autocorrelation time, the post-burn steps, their ratio, the acceptance fraction, the retained samples and whether it converged), the held-out block per target (measured values, predictive means, the 10th and 90th percentiles, and the MAPE, RMSE, mean log predictive density, coverage and mean width), the comparators scored the same way, the contraction table, and every term of the adoption rule with its outcome. The producer is `pipeline/hp500/crusherbayes.py`; the consumers are the web's validation views and the notebooks. Missing-data behaviour: readiness says *missing* and the views that would show CrusherBayes say that it is not available, never a number from an older bake; every other document is unaffected.

4.11 surrogate-model: the emulator slot

The `surrogate-model` contract (version 1.0.0, `data/derived/models/surrogate.json` beside the graphs `surrogate.onnx`, 32,449 bytes, and `surrogate.int8.onnx`, 15,880 bytes) carries everything the meaning of the graph depends on: the order of the five machine classes of the one-hot block, the order of the six continuous inputs (setting, throw, speed, feed x_{63}, feed exponent and $A \times b$) with their means and standard deviations, the order of the ten outputs (three sizes, five passing fractions, throughput and power), which outputs are fitted in $\log_{10}$, their means and standard deviations, the path, size in bytes, SHA-256, opset, input name `x` (float32, $[1, 11]$) and output name `y` (float32, $[1, 10]$) of each graph, the architecture, and the training block (seed, sizes, sampler, the input ranges per machine, the torch version and the export parity). The producer is `pipeline/stages/train.py`; the consumers are the browser through onnxruntime-web and the evaluation stage. The slot answers only an operating point that is not calibrated and lies inside the training ranges; otherwise the engine answers and the answer is labelled as such. Missing-data behaviour: when the contract or a graph is absent, fails its schema or its checksum, readiness says *training* with the baseline as provenance, the what-if view answers with the engine itself, labelled, and the Benchmark labels the metrics with the model that answered.

4.12 anomaly-model: the extrapolation-score slot

The `anomaly-model` contract (version 1.0.0, `data/derived/models/psd-ae.json` beside `psd-ae.onnx`, 5,859 bytes) describes the denoising autoencoder: the order of its fourteen features (the logarithms of P_{80}, P_{50} and P_{20}, the passing at five sizes, throughput, power, reduction ratio, the logarithm of F_{80}, specific

energy and the feed exponent), their means and standard deviations, the alarm threshold (the 99th percentile of the training scores, 0.24454), the mean training reconstruction score (0.05181), the graph (path, bytes up to 5 MiB, SHA-256, opset at least 13, input `x` and output `xr`, float32 $[1, 14]$), the architecture with the input noise, and the training block. The score is the mean squared error between the standardised features and their reconstruction; a score above the threshold, or a non-finite score, is an alarm. The producer is the training stage; the consumers are the browser and the evaluation stage. The slot scores only an operating point that is not calibrated and lies inside the training ranges; otherwise the envelope check answers, labelled. Missing-data behaviour: readiness says *training* with the baseline as provenance and the anomaly view answers with the envelope check, labelled as the baseline.

4.13 model-metrics: the held-out evaluation of the two slots

The `model-metrics` contract (version 1.0.0, `data/derived/model-metrics.json`, schema `chancadem.model-metrics/v1`) holds the held-out evaluation of Sections 6.13 and 6.14. For the surrogate: the training and test sizes (6430 and 2196), the seed of the test draw (67890), per output the R^2 and the MAPE of the float32 graph and the MAPE of the int8 graph, whether P_{80} is monotone in the setting on the fifteen test curves, the export parity (the largest absolute difference of the standardised outputs, torch against float32 and float32 against int8), and a note on how each number was computed. For the anomaly slot: the number of held-out points, the false-alarm rate (a fraction in $[0, 1]$), the number of planted off-manifold points, the detection rate, the same two rates for the envelope check, a description of how the points were planted, and a note. Every number is computed by `pipeline/stages/evaluate.py` through onnxruntime on the shipped graphs, never in torch, so the metrics describe what the browser runs. The consumer is the Benchmark of the web. Missing-data behaviour: readiness says *missing* and the model views of the Benchmark say that the evaluation is not available, never a number from an older bake; an older `engine_version` makes it *stale*.

4.14 readiness: what is real in a build

The `readiness` contract (version 1.0.0, `data/derived/readiness.json`, schema `chancadem.readiness/v1`) answers, for one build, which contracts are served by their real producer and in which state. It carries the product version, the SHA-256 of the catalog it was computed from (with line endings normalised), and one row per contract other than itself with the identifier, version, catalog status, provenance (real, baseline, a provider kind, or null), state (one of the six of Section 4.2), a bilingual reason when the state is not *available*, and the list of missing paths. It is computed by `pipeline/core/readiness.py` from the catalog and the repository, so editing the catalog makes it stale until it is recomputed, and a check fails meanwhile. In the committed build twelve rows are *available* and the interface row is *missing*, with the reason that the interface is dormant and the site reads the committed documents. Every view of the web reads it, and so does the interface. Missing-data behaviour: without it the site build stops at its copy step; if it cannot be loaded at run time the views keep working and each says that readiness is unknown; the interface answers 503 with the state *missing*.

4.15 api: the dormant interface

The `api` contract (version 1.0.1, `docs/contracts/api.openapi.json`, OpenAPI 3.1) describes the read-only interface of `app/`, dormant until a deployment needs a server. It has six routes, all `GET`: two health routes, the case list, the manifest of a case, an artifact by relative path and the readiness document (Appendix C). It serves the committed documents unchanged, and its components reference the same schema files as the catalog, so the TypeScript types generated from it are the types of the static site. Its rule for missing data is that a document that does not exist yet is a declared state and never an empty body that looks like data: when a document a route serves is absent, the route answers 503 with a body that names the state *missing* and the reason, and an unknown case or artifact is 404. The producer is the FastAPI application of `app/`; the consumers are its contract tests and, when the interface is activated, the web. The static readiness reports this row as *missing* while the interface is dormant, and the interface's own readiness route reports it as available.

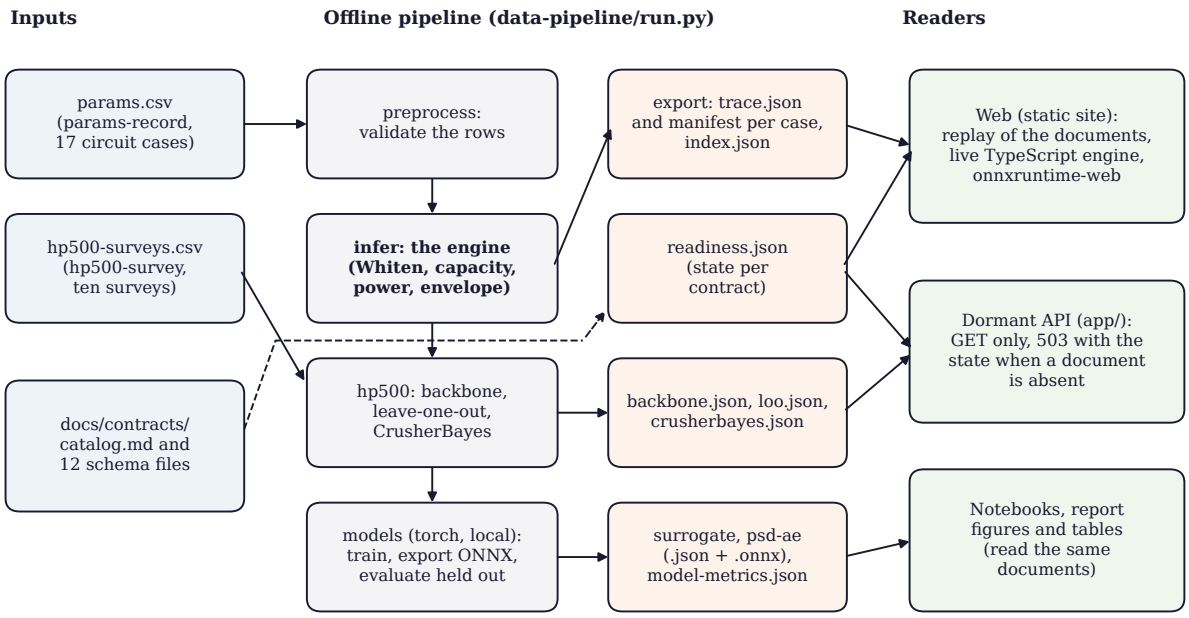


Figure 2: Data flow of the product: the two source tables and the contract catalog enter the offline pipeline, whose stages write the case traces, manifests and index, the three HP500 documents, the model bundles and their metrics, and the readiness document, which the static web, the dormant interface and the notebooks read.

5 System architecture

5.1 The offline pipeline, stage by stage

The pipeline is a module folder run by path (`data-pipeline/run.py`), not an installed package, and it runs on a workstation only; continuous integration validates the committed documents and never bakes or trains. Figure 2 shows its stages and the documents each writes. The release bake (`run.py all`) runs, in order: *preprocess*, which reads `params.csv` and the survey table and applies the two ingestion contracts; *infer*, which evaluates the canonical engine at the operating point of each of the 27 cases; *export*, which writes the trace and the manifest of each case, measures the lane of the case, checks the expected ranges and the units of every metric, and writes the index; the *HP500* step, which writes the backbone, the leave-one-survey-out document and the CrusherBayes document; and the readiness step, which computes the state of every contract from the catalog and the repository. A failing expected range or a value outside its unit's range stops the bake. The model step (`run.py models`) is separate because it needs torch: it draws the two Latin-hypercube sweeps, labels them with the engine, trains the surrogate and the autoencoder, exports them to ONNX, quantises the surrogate to int8, evaluates both through onnxruntime on the held-out draw, and refreshes the index and readiness. The bake never trains; it reads the released model bundles as they stand. Every random draw comes from one seeded generator factory, so a run is a pure function of its inputs and seeds: a re-bake into a sandbox on 27 October 2026, before the CrusherBayes document existed, reproduced every committed trace and both validation documents of that build byte for byte.

5.2 The engine in two languages

The engine is one vertical implementation written twice. The Python package `data-pipeline/pipeline/model/crusher/` is canonical: it labels the cases, the sweeps and the surveys. The TypeScript port `frontend/src/engine/` re-runs a case in the browser whenever the reader moves one of the six controls. Both are plain arithmetic on IEEE doubles. The Python engine deliberately avoids library linear algebra: the LU solve with partial pivoting is ported line for line, so the order of the floating-point operations is the TypeScript order, and a small module reproduces the JavaScript semantics where Python differs (a negative base with a fractional exponent is not a number, an exponent overflow is infinity, a division by zero is a signed infinity), so that an out-of-domain point returns a flagged result in both languages instead

Table 5: Measured resources of each stage on the development workstation: peak memory of the process tree, disk written and wall time, from the product's measured-resources record of the release that introduced the present engine and models.

Stage	Peak memory	Disk	Time	Where it runs
Bake (`run.py all`, 27 cases and the HP500 documents, before CrusherBayes)	67 MB	286 KB	7.3 s	workstation
Training (`run.py models`: sweeps, two networks, export, evaluation)	453 MB	58 KB	48 s	workstation, CPU
Python tests (the whole suite)	268 MB	temporary	962 s	workstation
Web install (`npm ci`)	644 MB	328 MB	25 s	workstation, CI
Web build (`npm run build`)	567 MB	16.8 MB	12 s	workstation, CI
Web unit tests (309 tests)	1951 MB	none	10 s	workstation, CI

of raising. A shared reference file of 124 operating points holds the two engines equal: every result field and every feature vector agrees within 10^{-9} relative, and the largest relative difference measured over the file is 2.3×10^{-12}. A second test re-runs every baked case in TypeScript and compares it with its trace to the six significant digits the trace is written with.

The lane of a case is decided by measurement. A case may be re-run live in the browser only when its engine is pure Python with dependencies inside an allowed set (`numpy`), when one run completes within 1500 ms and when its trace is at most 256 KiB; otherwise the web would replay the committed artifact. The canonical engine evaluates a case in a median of 1.34 ms (the median over the 27 cases of the median of seven warmed runs; at most 2.03 ms), and the largest trace is 4,127 bytes, so every case is live-eligible. The measured decision and the two budgets are written into each manifest; the raw run time is not, because a manifest must not change between two bakes of the same inputs.

5.3 The web, the learned models in the browser, and the interface

The site is static and is deployed to GitHub Pages; it holds no secret and calls no server. A copy step places exactly the files the index declares into the site, and the web's types are generated from the contract schemas. The App is a workbench on the shared CAOS application shell: a case selector, the six controls, and groups of views for the product gradation, the performance (the capacity hump, the setting trade-off and the drift projection), the chamber (a two-dimensional section and a three-dimensional kinematic view of the chamber geometry, its motion and the engine's gradation, which is not a particle simulation), the learned models and the validation. Each view draws the live engine and, dashed, the replay of the committed trace. The two ONNX graphs run in the browser through onnxruntime-web 1.27.0, the same release as the Python onnxruntime that evaluates them, as single-threaded WebAssembly served by the site itself, because a static host does not send the headers that threaded WebAssembly needs. Calls to one model are serialised, because the runtime forbids concurrent runs on one inference instance. The web encodes the inputs and decodes the outputs entirely from the model contracts, so a retrained model ships by replacing its contract and its graph.

Every view shows the readiness of the contracts it reads, and every learned answer is labelled with the model that answered: the surrogate or, when the bundle is invalid or the point is outside the training ranges or on the calibrated lane, the engine. The dormant interface of `app/` is a FastAPI application with six read-only routes over the committed documents; it is run by its contract tests and is not deployed.

5.4 Resources

Table 5 gives what each stage needs on the development workstation (Windows 11, Python 3.13, Node 24), measured with a process-tree monitor that polls the peak memory every 20 ms, on an archive copy of the release branch so that the working tree is not touched. No stage needs a graphics processor or a network after the installation. The site is 16.8 MB, of which 13.5 MB is the ONNX Runtime WebAssembly binary; the committed artifacts are 286 KB. The CrusherBayes sampler is part of the HP500 step and was added after these measurements; its configuration fixes the work at $20 \times 7000 = 140{,}000$ log-probability evaluations per posterior, each of which evaluates the engine on the nine or ten training surveys of its fold, for eleven posteriors, and its wall time is not recorded in the committed documents.

6 Methods

This section gives one subsection per method of the product, in the order in which the engine composes them and then in the order of the evaluation: the feed and the sieve grid (Section 6.1), the breakage function (6.2), the classification (6.3), the Whiten solve (6.4), the capacity (6.5), the regimes and the nip angle (6.6), the Bond power and the current-based power form (6.7), Morrell's size-specific energy (6.8), the calibration of the HP500 lane (6.9), the setting-only line (6.10), the leave-one-survey-out protocol CrusherCal (6.11), the envelope check (6.12), the learned surrogate (6.13), the denoising autoencoder (6.14), the drift projection (6.15) and CrusherBayes (6.16). Every subsection states the theory and its source, the equations as the code evaluates them, the assumptions, the parameters with their values and how they were set, the cost, and the implementation. All modules named are in `data-pipeline/pipeline/` of the product at release 0.12.001, whose artifacts every number of the report is read from; the TypeScript counterparts of the engine modules live in `frontend/src/engine/` under the same names.

6.1 Rosin-Rammler feed on a square-root-of-two sieve grid

Theory. Every gradation the engine handles, the feed, the product and the classification, lives on one fixed sieve grid, and the feed is the discretisation of a Rosin-Rammler (Weibull) distribution on it [24]. The cumulative fraction passing a size x is

$$R(x) = 1 - \exp\left[-\left(\frac{x}{x_{63}}\right)^{n}\right], \tag{4}$$

where x_{63} is the size at which $1 - 1/e \approx 63.2\%$ passes and n the uniformity exponent: the larger n, the narrower the feed. The mass of class k, between its upper edge e_k and its lower edge e_{k+1}, is

$$f_k = \frac{\max(0,\ R(e_k) - R(e_{k+1}))}{\sum_{j=0}^{K-1} \max(0,\ R(e_j) - R(e_{j+1}))}, \qquad e_k = \frac{256}{(\sqrt{2})^k}\ \text{mm}, \quad k = 0, \ldots, K, \tag{5}$$

so the vector is renormalised to sum to one and the mass above the 256 mm top of the grid is folded into the classes below it. Inverting Equation (4) at $R = 0.8$ gives $F_{80} = x_{63}(\ln 5)^{1/n}$, which the product uses in reverse to rebuild the feed of a survey from its measured f_{80}: $x_{63} = f_{80}/(\ln 5)^{1/n}$ with $n = 0.9$.

Reading a gradation. Two interpolations read a curve on the grid. The size at a passing fraction q (for P_{80}, P_{50}, P_{20} and F_{80}) is linear in the logarithm of the size between the two edges whose cumulative passing brackets q,

$$x_q = \exp\left[\ln e_{k+1} + \frac{q - R_{k+1}}{R_k - R_{k+1}}\,(\ln e_k - \ln e_{k+1})\right], \qquad R_{k+1} \le q \le R_k, \tag{6}$$

with R_k the cumulative passing at edge k; the passing at a given size (the readouts at 1, 4, 8, 16 and 32 mm) is the converse interpolation, one above the top edge and zero below the finest. The classes run from coarse to fine, so that a matrix that moves mass only to finer classes is strictly lower triangular, which the Whiten solve needs (Section 6.4).

Parameters and how they were set. The grid has its top edge at 256 mm and $K = 28$ classes of ratio $\sqrt{2}$, the standard laboratory series, so the finest edge is 0.015625 mm; the top size bounds every product size the engine can return and is the upper end of the unit check of every size (Section 9). The ingestion contract accepts x_{63} above 0 and up to 2000 mm and n from 0.3 to 4. The exponent of a rebuilt survey feed, $n = 0.9$, is an assumption of the product (the survey table gives one size of the feed, not its spread). A feed coarser than the grid is truncated: a declared F_{80} of 180 mm reads about 136 mm on the grid, which is why the envelope check of the calibrated lane tests the declared F_{80} and not the grid value (Section 6.12). The circuit cases set x_{63} and n in their table; nothing in the product fits them to data.

Cost and numerical scheme. Building a feed and reading four sizes are $O(K)$ operations. The interpolation in the logarithm of the size is exact for a power-law segment of the passing curve and keeps a read size inside its bracket; at a plateau of the curve ($R_k = R_{k+1}$) the upper edge is returned.

Worked example. The reference case S01 has $x_{63} = 90$ mm and $n = 1.2$. Equation (4) gives a declared F_{80} of $90\,(\ln 5)^{1/1.2} = 133.8$ mm, while the untruncated law passes 97.0% at the 256 mm top of the grid; after the remaining 3% is folded into the classes by the renormalisation of Equation (5), the engine reads $F_{80} = 126.03$ mm on its grid. The cumulative passing of the gridded feed at the first edges (256, 181, 128, 90.5 and 64 mm) is 1.000, 0.929, 0.807, 0.654 and 0.500: half of the mass lies above 64 mm, so most of the feed is coarser than the 32 mm setting and enters the breakage zone. The effect of the truncation grows

with the feed: for the primary gyratory case G01, with $x_{63} = 600$ mm and $n = 0.8$, the law passes only 39.7% at 256 mm, the declared F_{80} is 1088 mm and the grid reads 180.3 mm. This is why the declared and the grid values are kept apart, and why the product's absolute numbers for the primary machines carry the grid as a limit.

Assumptions and limits. The Rosin-Rammler law is the standard parametric description of crushed rock, not a claim that a real feed follows it; a feed measured on a belt would be better served by its own table, which the engine does not take. The module also holds the Gaudin-Schuhmann power law as an alternative fit, which the engine does not call.

Implementation and tests. `model/crusher/feed.py` (`make_feed`, `rosin_rammler_passing`, `rosin_rammler_size_at`), `model/crusher/sieve.py` (`make_grid`, `size_at_passing`, `passing_at_size`) and `model/crusher/hp500.py` (`x63_from_f80`). Its acceptance criterion is that the feed masses sum to one and the coarsest edge passes 100% (`test_feed_psd_normalizes_and_f80_is_sensible`), and an oracle independent of the engine checks that the passing at the characteristic size equals $1 - 1/e$. The grid edges and mid-sizes are written into every trace, so the web draws every curve on the grid the engine used.

6.2 Breakage function: JKMRC t_{10} and the Austin appearance function

Theory. The breakage matrix B says where the mass of a broken particle goes. It is built from two relations. The JKMRC drop-weight relation ties the fraction of the progeny that passes one tenth of the parent size, t_{10}, to the specific comminution energy E_{cs} [21]:

$$t_{10} = A\left(1 - e^{-b\,E_{cs}}\right), \qquad E_{cs} = 0.033\,T_{ref}\,\frac{N}{350}, \tag{7}$$

with A and b the ore competence parameters, whose product $A \times b$ is the single hardness number the product takes (lower is harder). The engine fixes $A = 60\%$, sets $b = (A \times b)/A$, and caps t_{10} at 0.44. The specific energy per nip grows with the speed N and uses the design throw T_{ref} of the machine class, not the throw control: a wider throw coarsens the product through the classification window (Section 6.3) and does not add energy in this model. At the reference case S01 the energy is $0.033 \times 30 \times 360/350 = 1.018$ kWh/t. The progeny spread follows the cumulative appearance function of Austin, Klimpel and Luckie [23] in the relative size $u = x/y$ of a progeny size x to its parent size y,

$$B(u) = \varphi\,u^{\gamma} + (1 - \varphi)\,u^{\beta}, \qquad B(1) = 1, \qquad \varphi = \frac{t_{10} - 0.1^{\beta}}{0.1^{\gamma} - 0.1^{\beta}}, \tag{8}$$

where γ sets the slope of the fines and β the slope near the parent size, and φ is solved so that the curve passes through $(u = 0.1,\ B = t_{10})$, then clamped to $[0.02, 0.98]$. Figure 3(b) shows the function for three values of t_{10}. At $u = 0.1$ the largest value the function can reach (with $\varphi = 1$) is 0.1^{γ}, which for $\gamma = 0.35$ is about 0.447; this is why the cap of t_{10} is 0.44.

The matrix. For a parent class j of mid-size $\bar{d}_j$, the entry B_{ij} for a finer class $i > j$ is the difference of $B(u)$ at the class edges, with the upper edge capped at the parent size,

$$B_{ij} = \frac{\max\!\left(0,\ B\!\left(\min(e_i, \bar{d}_j)/\bar{d}_j\right) - B\!\left(e_{i+1}/\bar{d}_j\right)\right)}{\sum_{l>j} \max\!\left(0,\ B\!\left(\min(e_l, \bar{d}_j)/\bar{d}_j\right) - B\!\left(e_{l+1}/\bar{d}_j\right)\right)}, \qquad i > j, \tag{9}$$

and $B_{ij} = 0$ for $i \le j$. Every column is renormalised to exactly one, so broken mass is conserved, and the matrix is strictly lower triangular, so broken mass only reports to finer classes. On the calibrated HP500 lane the energy route is replaced by the fitted relation $t_{10}[\%] = 64 - 0.12\,s - 0.23\,f_{80}$ of Section 6.9, with the same cap.

Worked example. At S01 the ore has $A \times b = 55$, so $b = 55/60 = 0.917$ t/kWh, and with $E_{cs} = 1.018$ kWh/t Equation (7) gives $t_{10} = 60\,(1 - e^{-0.933}) = 36.4\%$, the value the trace records (0.364). The weight that makes the appearance function pass through it is $\varphi = 0.815$: four fifths of the progeny follow the fines branch $u^{0.35}$ and one fifth the coarse branch $u^{4.2}$. A harder ore lowers t_{10} at the same energy: the case S03, with $A \times b = 38$ and a speed of 470 rpm ($E_{cs} = 1.329$ kWh/t), reaches $t_{10} = 60\,(1 - e^{-0.633 \times 1.329}) = 34.1\%$, so a faster machine partly offsets a harder ore. Because the energy grows linearly with speed, the speed sweep of a case changes both the capacity (Section 6.5) and the fineness of the progeny, which is why the Bond power of S01 peaks at a higher speed (288.6 kW at 396 rpm in the sweep) than its throughput (643 t/h at 360 rpm).

Parameters and how they were set. $\gamma = 0.35$ (chosen so that the function can represent the full t_{10} range it is asked to pass through), $\beta = 4.2$ (the code describes it as illustrative, within 3 to 6), $A = 60\%$, the

energy coefficient 0.033 kWh/t per millimetre of design throw at 350 rpm, and the design throws of Table 20 (30 mm for the gyratory and the secondary cone, 38 mm for the jaw, 16 mm for the tertiary and short-head cones). These are typical values of the method, not a characterisation of an ore; no drop-weight test is run, and one progeny shape serves every ore.

Cost and numerical scheme. Filling the matrix evaluates the appearance function twice per entry below the diagonal: $K(K-1)/2 = 378$ entries, each two powers, so $O(K^2)$ operations; the column renormalisation is $O(K^2)$. The clamp of φ keeps the appearance function monotone in u and every entry non-negative.

Assumptions and limits. The relation is a size-independent breakage law: the probability that a particle breaks is the classification, not this function, and the energy per nip does not vary across the chamber. The t_{10} family is also the bridge to discrete-element crusher models that size progeny with the same drop-weight t_{10} [15], which makes trends comparable, not absolute progeny distributions.

Implementation and tests. `model/crusher/breakage.py` (`specific_energy`, `t10_of`, `t10_calibrated`, `phi_from_t10`, `appearance_cum`, `breakage_matrix`) and `model/crusher/machines.py`. The acceptance criterion is that the matrix is strictly lower triangular and every column conserves mass (`test_breakage_matrix_is_strictly_lower_triangular_and_columns_conserve_mass`); a second test checks that the throw does not enter the energy. The trace carries the t_{10} of each case.

6.3 Andersen and Napier-Munn classification

Theory. The classification function $C(d)$ is the probability that a particle of size d in the chamber is nipped and broken instead of passing through. With three parameters, in the form of Andersen and Napier-Munn [22] used in the Whiten crusher model [20, 21],

$$C(d) = \begin{cases} 0, & d \le K_1, \\ 1 - \left(\dfrac{K_2 - d}{K_2 - K_1} \right)^{K_3}, & K_1 < d < K_2, \\ 1, & d \ge K_2, \end{cases} \tag{10}$$

so every particle below K_1 escapes, every particle above K_2 is broken, and K_3 shapes the rise between them. The function is continuous and monotone non-decreasing for $K_3 > 0$. On the circuit machines K_1 is linear in the closed-side setting and K_2 is linear in the setting and in the throw,

$$K_1 = a_1 + b_1\, s, \qquad K_2 = \max\!\big(a_2 + b_2\, s + \tfrac{1}{2}\, b_2\, (T - T_{ref}),\ 1.05\, K_1\big)\,, \tag{11}$$

so K_2 tracks the open-side setting $s + T$: a wider throw lets larger particles escape during the open phase of the cycle and coarsens the product at a fixed setting, in the direction of the flow model of Evertsson [25], and at the design throw the term vanishes. The floor at $1.05\, K_1$ keeps the window from inverting. Figure 3(a) shows $C(d)$ of case S01 at three settings: at the case's 32 mm, $K_1 = 29.0$ mm and $K_2 = 68.4$ mm.

The shape of the window. Differentiating Equation (10) inside the window gives

$$\frac{dC}{dd} = \frac{K_3}{K_2 - K_1} \left(\frac{K_2 - d}{K_2 - K_1} \right)^{K_3 - 1}, \qquad \left.\frac{dC}{dd}\right|_{d=K_1} = \frac{K_3}{K_2 - K_1}, \qquad \left.\frac{dC}{dd}\right|_{d=K_2} = 0 \quad (K_3 > 1), \tag{12}$$

so for $K_3 > 1$ the probability of breakage rises steeply from K_1 and flattens as it approaches one at K_2, and a larger K_3 concentrates the rise near K_1. At S01 the slope at K_1 is $2.3/(68.4 - 29.0) = 0.058$ per millimetre. On the grid this gives $C = 1$ for the four coarsest classes (mid-sizes of 76 mm and above), 0.898 at 53.8 mm, 0.452 at 38.1 mm and zero from 26.9 mm down, as the trace of the case records: everything finer than the setting passes, and the fraction of the class around the setting that is broken is what the setting controls.

The throw term. By Equation (11), one millimetre of throw above the design value raises K_2 by $b_2/2$, that is by 0.975 mm on the secondary cone, and leaves K_1 unchanged, so the window widens upward and a class near its top is broken with a lower probability. The capacity rises at the same time, through the gap $s + T/2$ of Equation (16): a wider throw gives a coarser product at a higher throughput, which is the trade-off the throw control of the studio shows. On the calibrated lane the window follows the setting and the engine's grid F_{80}; for survey 1 (54 mm, grid $F_{80} = 84.86$ mm) the fitted relations give $K_1 = 37.9$ mm and $K_2 = 75.6$ mm.

Parameters and how they were set. The per-machine constants are in Table 20: for the secondary cone $a_1 = 0.2$ mm, $b_1 = 0.90$, $a_2 = 6.0$ mm, $b_2 = 1.95$ and $K_3 = 2.3$; the tertiary, short-head, gyratory

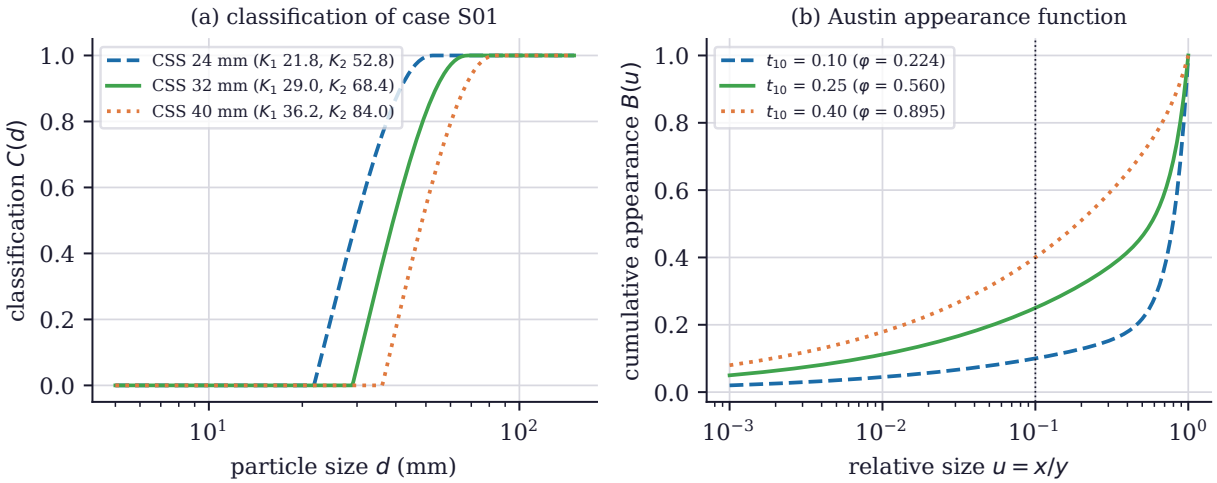


Figure 3: The two functions of the chamber, computed with the product's engine code: (a) the classification $C(d)$ of the secondary case S01 at settings of 24, 32 and 40 mm with the resulting K_1 and K_2; (b) the Austin appearance function at t_{10} of 0.10, 0.25 and 0.40, each passing through $B = t_{10}$ at $u = 0.1$ (dotted).

and jaw classes have their own. The code labels them provisional values typical of the literature; they reproduce the trends (a tighter setting gives a finer product, a wider throw a coarser one) and not a vendor's absolute sizes. On the calibrated lane they are replaced by the fitted relations $K_1 = 0.23\,s + 0.30\,f_{80}$, $K_2 = 12.0 + 0.55\,s + 0.40\,f_{80}$ and $K_3 = 2.3$, independent of the throw (Section 6.9); there CrusherBayes multiplies K_1 and K_2 by two scalars and samples K_3.

Cost and numerical scheme. The vector $\mathbf{c}$ is C at the 28 mid-sizes, $O(K)$ powers. The ratio in Equation (10) lies in $(0, 1)$ inside the window, so the power is well defined.

Assumptions and limits. The function describes the chamber as one classifying zone; it is not a measurement of a real chamber's classification, and it does not carry the size-by-size breakage energy, which is the breakage function's. In a real cone the classification varies along the chamber, from the feed opening to the parallel zone, and the Whiten form collapses it into one window; the published industrial calibrations of the form fit K_1, K_2 and K_3 to survey data, and Duarte et al. [10] go further and fit every entry of the breakage matrix as well. On the circuit cases of the product nothing is fitted, so the window is a typical shape whose position follows the setting and the throw, and its absolute sizes inherit the uncertainty of the constants.

Implementation and tests. `model/crusher/classification.py` (`KMODEL`, `classification_params`, `hp500_params`, `classify`). The acceptance criteria are that $C(d)$ is monotone, zero below K_1 and one above K_2 (`test_classification_is_monotone_non_decreasing_c_small_zero_c_large_one`), that a wider throw widens the open-side setting and gives a coarser product with a higher capacity, and that the throw is neutral at the design throw. The trace carries $\mathbf{c}$ for the product view.

6.4 The Whiten population balance

Theory. The chamber is one breakage zone [20, 21]. A particle of class j entering the zone is selected for breakage with probability c_j, and the progeny of a broken particle report to finer classes through B. Let $\mathbf{x}$ be the mass flow through the zone per unit of feed: it is the feed plus the progeny of everything selected, $\mathbf{x} = \mathbf{f} + \mathrm{BC}\mathbf{x}$, and the product is the part of the flow that was not selected, $\mathbf{p} = (\mathrm{I} - \mathrm{C})\mathbf{x}$. Hence

$$\mathbf{p} = (\mathrm{I} - \mathrm{C})\,(\mathrm{I} - \mathrm{BC})^{-1}\,\mathbf{f}. \tag{13}$$

Because B is strictly lower triangular and C is diagonal, BC is strictly lower triangular, so $\mathrm{M} = \mathrm{I} - \mathrm{BC}$ is unit lower triangular with determinant one and is never singular. Mass is conserved in exact arithmetic: summing the balance and using $\mathbf{1}^{\top}\mathrm{B} = \mathbf{1}^{\top}$ on the columns that have finer classes gives $\mathbf{1}^{\top}\mathbf{p} = \mathbf{1}^{\top}\mathbf{f}$, except for mass selected in the finest class, which has no finer class to go to; the engine reports the closure $|\mathbf{1}^{\top}\mathbf{p} - \mathbf{1}^{\top}\mathbf{f}|$ before it renormalises the product. The same balance gives the expected number of breakage events per unit of feed,

$$\nu = \mathbf{1}^{\top}\mathrm{C}\,\mathbf{x} = \sum_{k} c_k x_k, \tag{14}$$

Algorithm 1 The Whiten solve of one operating point, as `whiten_solve` runs it.

Require: classification $\mathbf{c} \in [0,1]^K$, breakage matrix B (strictly lower triangular, columns summing to one), feed $\mathbf{f}$ with $\mathbf{1}^\top \mathbf{f} = 1$
Ensure: product $\mathbf{p}$ with $\mathbf{1}^\top \mathbf{p} = 1$, mass closure ϵ, conditioning proxy κ
for $i = 1$ **to** K, $j = 1$ **to** K **do**
  $M_{ij} \leftarrow [i = j] - B_{ij}\, c_j$ ▷ $\mathrm{M} = \mathrm{I} - \mathrm{BC}$
end for
$\kappa \leftarrow \max_i \sum_j |M_{ij}| \,/\, \min_i \sum_j |M_{ij}|$
factor $\mathrm{PM} = \mathrm{LU}$ by Gaussian elimination with partial pivoting ▷ a zero pivot returns $\mathbf{f}$ and $\kappa = \infty$
solve $\mathrm{L}\mathbf{z} = \mathrm{P}\mathbf{f}$ forward, then $\mathrm{U}\mathbf{x} = \mathbf{z}$ backward
for $i = 1$ **to** K **do**
  $p_i \leftarrow \max(0,\ (1 - c_i)\, x_i)$
end for
$\epsilon \leftarrow |\sum_i p_i - \sum_i f_i|$ ▷ reported before renormalising
$\mathbf{p} \leftarrow \mathbf{p} / \sum_i p_i$
return $\mathbf{p}$, ϵ, κ

the quantity that Section 9 uses to check the solve against a particle-replacement walk.

Numerical scheme. The inverse is never formed. Algorithm 1 builds M, solves $\mathrm{M}\mathbf{x} = \mathbf{f}$ by LU decomposition with partial pivoting (ported from the TypeScript engine as written, so both languages run the same operations), applies $(\mathrm{I} - \mathrm{C})$ with a floor at zero, records the closure, and renormalises. A conditioning proxy, the ratio of the largest to the smallest absolute row sum of M,

$$\kappa = \frac{\max_i \sum_j |M_{ij}|}{\min_i \sum_j |M_{ij}|}, \tag{15}$$

is kept for the regime near choke, where many entries of $\mathbf{c}$ approach one; a point with κ above 10^6 or not finite is flagged ill-conditioned and marked invalid. At case S01 the proxy is 1.517 and the closure 3.3×10^{-16}. Pivoting is redundant for a unit triangular matrix but is kept so that the two engines stay identical for any input.

Reading the solution. Because M is unit lower triangular, the system $\mathrm{M}\mathbf{x} = \mathbf{f}$ is also solved by forward substitution from the coarsest class down, $x_i = f_i + \sum_{j<i} B_{ij}\, c_j\, x_j$: the flow through class i is its own feed plus the progeny of everything broken in the coarser classes. This makes the physical reading of the balance explicit, and it is the recursion that the particle-replacement check of Section 9.3 samples. At S01 the expected number of breakage events per unit of feed, Equation (14), is 0.886: a tonne of feed undergoes on average less than one nip in this model, because the half of the feed finer than about 64 mm is broken only in part and everything below 29 mm passes unbroken. The product of S01 has $P_{80} = 27.7$ mm, $P_{50} = 12.1$ mm and $P_{20} = 1.83$ mm, a reduction ratio of 4.55, and 15.0% of its mass passes 1 mm.

Cost. Building M and the proxy are $O(K^2)$; the factorisation is $O(K^3)$, about $2K^3/3 \approx 1.5 \times 10^4$ floating-point operations for $K = 28$; the two triangular solves are $O(K^2)$. One full evaluation of the engine, with the feed, both functions, the solve, the readouts, the capacity, the power and the checks, takes a median of 1.34 ms in Python (Section 5.2).

Assumptions and limits. One breakage zone, steady state, no spatial resolution of the chamber, no wear and no segregation. The balance is the fast approximation of what a discrete-element simulation resolves particle by particle, and it inherits none of that method's mechanism: its classification and breakage functions are inputs, not results.

Implementation and tests. `model/crusher/whiten.py` (`whiten_solve`, `assemble_chamber`, `replacement_walk`), `model/crusher/linalg.py` (`solve_lu`, `cond_proxy`) and `model/crusher/engine.py` (`evaluate`), which runs the steps in order: the feed and its F_{80}; K_1, K_2, K_3 and $\mathbf{c}$; the energy, t_{10}, φ and B; the solve; P_{80}, P_{50}, P_{20} and the readouts; the throughput and the power; the nip angle, the regime, the validity flags and the envelope codes. The acceptance criteria are mass closure with a product finer than the feed and a finer product at a tighter setting (`test_whiten_solve_conserves_mass_and_product_normalizes`, `test_tighter_css_gives_a_finer_product`).

6.5 Capacity: a reduced Evertsson flow model

Theory. The capacity of a cone crusher rises with eccentric speed up to an optimum and falls past it: at low speed the chamber discharges slowly, and past the optimum the gyrating head obstructs the free fall of the material faster than gravity clears it. The flow model of Evertsson [25, 26] resolves this in the chamber; the engine uses a reduced parametric form that keeps the hump and the dependence on the discharge gap,

$$Q = \max\!\left(0,\; Q_{ref}\,\frac{s + T/2}{g_{ref}}\,\sigma_N\, e^{\,1-\sigma_N}\right), \qquad \sigma_N = \frac{N}{N_{opt}}. \tag{16}$$

The factor $\sigma_N e^{1-\sigma_N}$ has its maximum value one at $\sigma_N = 1$ and is unimodal, and the opening factor is proportional to the mean discharge gap $s + T/2$. The derivative of the hump factor, $(1 - \sigma_N)e^{1-\sigma_N}$, is positive below the optimum and negative above it, so a case run past the optimum speed loses capacity as the speed rises. Figure 4(a) shows the speed sweeps of the secondary cases S01 (at the optimum, 643 t/h), S02 (below it) and S03 (past it, where capacity falls with speed).

Sensitivities. The elasticities of the capacity with respect to its levers follow from Equation (16) inside the positive range,

$$\frac{N}{Q}\frac{\partial Q}{\partial N} = 1 - \sigma_N, \qquad \frac{s}{Q}\frac{\partial Q}{\partial s} = \frac{s}{s + T/2}, \qquad \frac{T}{Q}\frac{\partial Q}{\partial T} = \frac{T/2}{s + T/2}, \tag{17}$$

so one percent more speed gives $1 - \sigma_N$ percent more capacity: a gain below the optimum, nothing at it and a loss past it. At S01 ($\sigma_N = 1$) the speed elasticity is zero; at S02 (300 rpm, $\sigma_N = 0.833$) it is +0.17, and at S03 (470 rpm, $\sigma_N = 1.306$) it is −0.31, which is the falling side of the hump that the case is built to show. The setting and throw elasticities add to one: at S01 they are $32/47 = 0.68$ and $15/47 = 0.32$. The speed sweep of S01 checks the closed form: at half the case speed ($\sigma_N = 0.5$) the factor is $0.5\,e^{0.5} = 0.824$ and the trace gives 530 t/h, and at 1.5 times it the factor is $1.5\,e^{-0.5} = 0.910$ and the trace gives 585 t/h, both equal to 643.16 t/h times the factor. The power sweep of the same case has its maximum at a higher speed, because the specific energy of Equation (7) also grows with speed and makes the product finer (Section 6.2).

Parameters and how they were set. Table 20 gives Q_{ref}, N_{opt} and g_{ref} per class: for the secondary cone 520 t/h, 360 rpm and 38 mm, so S01 (setting 32 mm, throw 30 mm, 360 rpm) gives $520 \times 47/38 = 643.16$ t/h. The circuit values are illustrative of their class; only the shape (unimodal in speed, increasing in setting and throw) is claimed for them. On the calibrated lane Q_{ref} is the capacity base $Q_T = 647$ t/h carried from the calibration of Rocha et al. [11], which CrusherCal refits per fold and CrusherBayes samples.

Cost and numerical scheme. A closed form, $O(1)$; the floor at zero keeps the throughput non-negative for any input.

Assumptions and limits. No cavity-level material balance, no feed segregation, no choke-level dynamics: the reduced form is the hump and the gap dependence. It is not a vendor capacity table. The throughput of a survey is set by the plant together with the setting, so the capacity model is a prediction of the feed rate only under the assumption that the machine runs at its capacity. On the ten surveys that assumption holds to first order: the measured feed rate rises with the setting at about 32 t/h per millimetre (Figure 1), and the capacity factor of the calibrated engine, with no other information than the setting, reproduces the measured feed rates to 11.55% in sample (Section 6.9). A plant that runs a crusher below its capacity, for example when the feed is restricted upstream, breaks the assumption, and the capacity is then an upper bound of the throughput rather than a prediction of it.

Implementation and tests. `model/crusher/capacity.py` (`CAP`, `HP500_QREF`, `throughput`, `optimal_speed`). The acceptance criterion is a capacity unimodal in speed (`test_capacity_is_unimodal_in_speed`); a second test requires a positive and finite throughput and power at every valid point. Each trace carries a sweep of 31 speeds from 0.5 to 1.5 times the case speed.

6.6 Regimes and the nip angle

Theory. Two outputs qualify a result rather than compute a product: the regime of the operating point and the nip angle against its grip limit. A setting at or above the feed's 80% passing size grips little of the feed, and a setting at or above its top size grips none of it; in both cases the population balance would still return a product, and the engine must say that the number means little. The nip angle of a compression crusher is the angle between the two crushing surfaces at the point where a particle is gripped; a particle is nipped when the angle is at most twice the friction angle, the grip limit,

$$\alpha = \alpha_0 + 3\left(\frac{s}{g_{ref}} - 1\right), \qquad \alpha_{lim} = 2\arctan\mu_f, \tag{18}$$

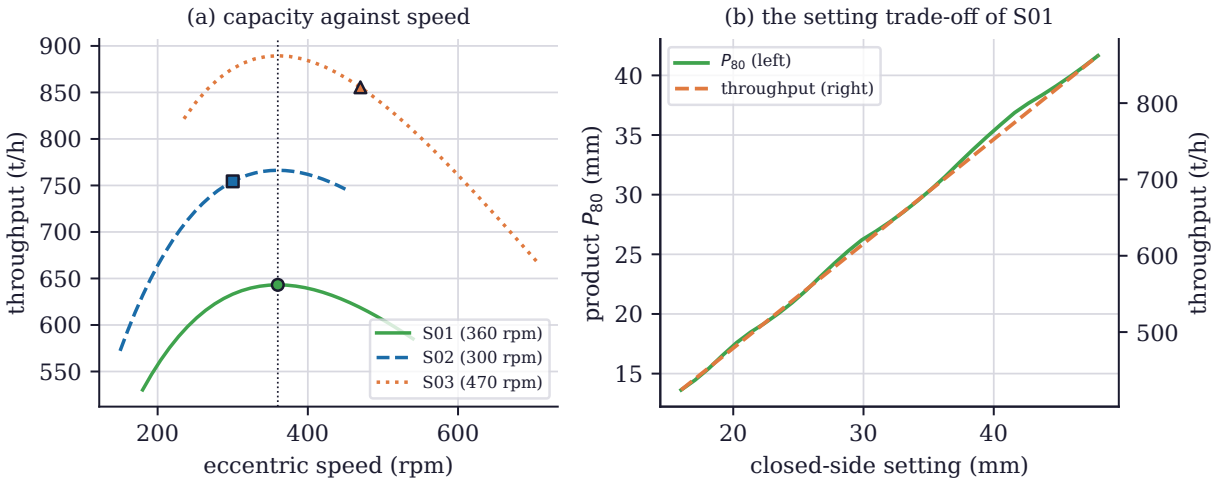


Figure 4: The engine's response to two levers, read from the committed traces: (a) throughput against eccentric speed for the secondary cases S01, S02 and S03, with each case's operating speed marked and the optimum of 360 rpm dotted; (b) product P_{80} and throughput of S01 against the closed-side setting.

with $\alpha_0 = 22°$ for the cones and the gyratory and 19° for the jaw, and $\mu_f = 0.35$, so $\alpha_{lim} = 38.58°$. The limit follows from a force balance on a particle squeezed between two surfaces that open at the angle α: each surface pushes with a normal force F_n inclined by $\alpha/2$ from the closing direction, and friction can hold at most $\mu_f F_n$ along each surface, so the particle is held when $2\mu_f F_n \cos(\alpha/2) \geq 2F_n \sin(\alpha/2)$, that is when $\tan(\alpha/2) \leq \mu_f$. At case S01 the nip angle is 21.53°, well inside the limit. Over the 27 committed traces the angle ranges from 18.18° (the jaw J02) to 41.11° (the invalid control CI1, whose setting of 280 mm is more than seven times the reference gap); CI1 is the only case above the limit, and the next largest is the pass-through control CP1 at 31.63°.

The rules, as the engine applies them. With F_{80} the feed size on the grid and $e_0 = 256$ mm the top edge, and $\rho = F_{80}/P_{80}$ the reduction ratio:

$$\text{regime} = \begin{cases} \text{invalid}, & s \geq e_0, \\ \text{pass-through}, & F_{80} \leq s < e_0 \quad \text{or} \quad \rho < 1.05, \\ \text{choke}, & \text{otherwise}, \end{cases} \tag{19}$$

and the point is marked invalid when $s \geq e_0$, when the conditioning proxy exceeds 10^6 or is not finite, or when the envelope check finds an implausible input (Section 6.12). Each rule writes a stable flag code into the manifest (`pass_through`, `css_ge_feed_top`, `negligible_reduction`, `ill_conditioned`), which the web maps to text in both languages and refuses when it does not know the code. Two sourced warnings, the setting below the vendor minimum of the machine class and a reduction ratio above the typical ceiling of one stage, never change the regime or the validity (their bounds are in Section 6.12). The regime type also lists *trickle*, which the engine never assigns. Among the 27 committed traces, 23 are choke, three pass-through (CP1, G02 and the survey HP500-03, whose measured f_{80} of 47 mm is below its setting of 48 mm) and one invalid (CI1).

Parameters and how they were set. α_0 and μ_f are typical values, not a vendor's; the factor 3 (half of 6° per unit of relative gap) makes the angle rise modestly with the setting. The thresholds 1.05 on the reduction ratio and 10^6 on the proxy are the engine's. The nip angle is reported and drawn in the chamber view; it does not set the regime.

Cost. $O(1)$ comparisons after the solve.

Assumptions and limits. *Choke* is the label when no rule fires; it is not a choke-feed model. The rules are not a safety check of a real machine. The pass-through rule compares the setting with the feed's F_{80} on the grid, so a coarse feed whose grid value is truncated (Section 6.1) can sit on either side of the rule depending on the grid; the survey HP500-03 shows the opposite case, a fine feed whose grid value of 47.15 mm lies just below its setting of 48 mm although its measured f_{80} of 47 mm would also. In both cases the engine still returns a product, flagged, and the manifest records the codes `pass_through` and `f80_lt_css`; the web shows the flags beside every number of such a case and never presents it as a confident answer.

Implementation and tests. `model/crusher/engine.py` and `model/crusher/capacity.py` (`nip_angle`,

`nip_limit`); the flag codes are in `core/manifest.py`. The acceptance criterion is that the pass-through and invalid flags fire at a setting at or above the feed's F_{80} and top size (`test_pass_through_and_invalid_guard_fires_when_css_ge_feed_top_size`); the controls CP1 and CI1 are baked to carry them.

6.7 Bond power and the current-based power form

Theory. Bond's third theory of comminution [27] gives the specific energy of a size reduction from a feed 80% passing size to a product 80% passing size, both in micrometres, through the work index of the ore:

$$W = 10\, W_i \left(\frac{1}{\sqrt{P_{80}}} - \frac{1}{\sqrt{F_{80}}} \right) \ \text{[kWh/t]}, \qquad P_d = \max(0,\ W)\, Q \ \text{[kW]}, \tag{20}$$

so the net draw P_d is the specific energy times the throughput of Section 6.5. On the circuit cases P_d is the power the engine reports; at S01 it is 287.4 kW. Bond's law was developed for grinding, and it is known to be a weak descriptor of coarse crushing duty; Morrell's size-specific energy (Section 6.8) is the documented alternative and is held out beside it. On the calibrated lane the engine applies the current-based form that Rocha et al. fitted to the HP500 [11], reading the paper's net power as the Bond draw,

$$P_c = \zeta\, P_d + P_n, \qquad \zeta = 1.30, \quad P_n = 110 \text{ kW}, \tag{21}$$

with P_n the no-load power. In the paper P_d is a size-specific net power of the paper's own model, not the Bond draw, so the two constants do not transfer; the backbone document states this in its note.

What the measured power says. Table 6 and Figure 5 hold the three power estimates of the calibrated engine against the ten measured values in sample. The bare Bond draw is nearly flat, from 196.1 to 237.5 kW, against a measured spread of 187 to 355 kW: its mean absolute percentage error is 26.88% and its R^2 is -1.29. The published form overshoots, at 47.58% and $R^2 = -3.64$, because its constants are for a different P_d. The best the form can do on these data, a least-squares refit of ζ and P_n on all ten surveys, reaches 19.28% with $\zeta = -2.41$ and $P_n = 828.6$ kW: a negative slope that collapses the prediction toward the survey mean, which is a fit and not a physical constant. The constant mean itself is at 22.03%. Out of sample the picture is the same (Section 8.2).

Why the draw is flat. At S01 the law gives $W = 140\,(27{,}698^{-1/2} - 126{,}030^{-1/2}) = 0.447$ kWh/t and, at 643.16 t/h, the 287.4 kW of the trace. On the survey lane the same decomposition explains the flat draw within the model: the specific energy of the backbone rises as the setting tightens, from 0.190 kWh/t at 54 mm (survey 1) to 0.229 kWh/t at 40 mm (survey 6), while its throughput falls from 1208.9 to 970.5 t/h, and the product of the two moves only from 229.5 to 222.5 kW. The measured power instead rises from 209 to 355 kW between the same two surveys. The engine's power therefore cannot follow the measured power, whatever its two constants, unless the specific energy rose much faster with a tighter setting than the Bond law and the engine's product sizes make it; whether the plant's current-based power measures something other than the comminution energy, a no-load share that grows at tight settings for example, cannot be decided from these data.

Parameters and how they were set. W_i is a case input on the circuit cases (2 to 40 kWh/t accepted) and the blend value 9.0 kWh/t on the HP500 lane, between the compact (7.2) and the supercompact (10.6) itabirite bands carried from the source; ζ and P_n are the published values, refit per fold by CrusherCal and sampled by CrusherBayes. The draw is floored at zero.

Cost. $O(1)$ after the solve. *Assumptions and limits.* The work index is a single number for the ore; the law gives energy from two sizes and says nothing about how the chamber spends it. The units of the power are checked against a range of 0 to 3000 kW, a unit-error ceiling and not an accuracy claim (Section 9).

Implementation and tests. `model/crusher/capacity.py` (`bond_power`, `calibrated_power`). The acceptance criteria are a positive and finite power at every valid point, and the in-sample and held-out power results reported as they are (`test_throughput_and_power_are_positive_and_finite_for_a_valid_point`, `test_loo_validates`).

6.8 Morrell's size-specific crusher energy

Theory. Morrell's method gives the specific energy of comminution circuits from the feed and product sizes and ore work indices measured by the SMC Test; for a conventional crusher the guideline of the Global Mining Standards and Guidelines Group prints, in its Annex B (equations B4 and B5) [29], following

Table 6: In-sample power of the calibrated engine against the ten measured powers: the constant mean, the bare Bond draw, the published current-based form applied to the Bond draw, and that form refit on the ten surveys. Generated from `loo.json`.

Power estimate (in sample)	MAPE (%)	RMSE (kW)	R^2	Max. dev. (%)
Constant mean of the ten surveys	22.03	61.65	0.000	54.01
Bare Bond draw P_d	26.88	93.22	−1.286	37.32
Published form $1.30\,P_d + 110$	47.58	132.85	−3.643	123.93
Refit form $\zeta P_d + P_n$	19.28	54.97	0.205	36.46

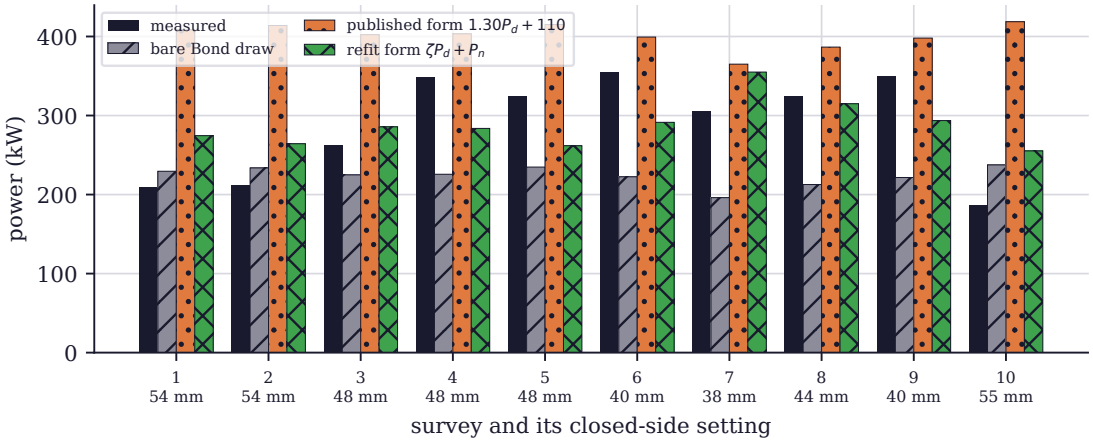


Figure 5: Power per HP500 survey in sample: the measured power, the bare Bond draw of the calibrated engine, the published form $1.30\,P_d + 110$ on that draw and the refit form, with the closed-side setting of each survey under its number.

Morrell [28],

$$W_c = S_c\,K_2\,M_{ic}\,4\left(x_2^{f(x_2)} - x_1^{f(x_1)}\right)\ \text{[kWh/t]}, \qquad f(x) = -\left(0.295 + \frac{x}{10^6}\right), \tag{22}$$

$$S_c = \min\!\left(1,\ K_s\,(x_1 x_2)^{-0.2}\right), \qquad K_s = 55, \tag{23}$$

with x_1 and x_2 the feed and product 80% passing sizes in micrometres, $K_2 = 1.0$ for a crusher in closed circuit with a classifying screen (1.19 in open circuit) and M_{ic} the crushing work index of the ore in kWh/t. The term $x^{f(x)}$ is a size in micrometres raised to a small negative power, a pure number, so W_c carries the unit of M_{ic}, and the power is $W_c\,Q$ in kW. The guideline defines the net power as the motor power less the no-load power, so a measured motor power is the net power plus a no-load term. The primary paper was not opened; the equations are taken from the guideline as printed.

The fit on the HP500 surveys. The M_{ic} of the Minas Rio itabirite is not published, so it is a fitted scalar, and so is the no-load power; both are refit on the nine training surveys of every fold and the held-out survey never enters. The design column of a survey is $g_i = W_c(M_{ic} = 1)\,Q_i$, with the measured f_{80} as x_1, the backbone's P_{80} as x_2 (a model output used as an input, never as a target) and the measured feed rate as Q_i. Two variants are held out, both closed circuit:

$$\hat{P}_i = M_{ic}\,g_i \quad \text{(through the origin)}, \qquad \hat{P}_i = M_{ic}\,g_i + P_{noload} \quad \text{(with a no-load term)}, \tag{24}$$

the first fitted by $M_{ic} = \sum g_i P_i / \sum g_i^2$ over the training surveys and the second by ordinary least squares. Their results are in Section 8.5: in every fold the variant with a no-load term fits a negative M_{ic}.

The design column on the surveys. Computed with the product's functions, the specific energy per unit work index, $W_c(M_{ic} = 1)$, lies between 0.058 (survey 1: $x_1 = 88{,}000$ μm, $x_2 = 32{,}560$ μm, $S_c = 0.706$, size term 0.0819) and 0.069 (survey 6: $S_c = 0.752$, size term 0.0923), so the energy law varies by a fifth across the surveys, while the feed rate varies by a factor of two. The design column g_i therefore follows the feed rate almost exactly (correlation 0.99, values from 56.4 to 94.8 kW per kWh/t), and because the measured power falls as the feed rate rises, g_i correlates with the measured power at −0.81. A positive M_{ic} through the origin can only scale a column that runs against the target, which is why its error is large; with a free intercept, least squares turns the slope negative. The diagnosis is the same as for the Bond draw

(Section 6.7): both energy laws multiply a specific energy that changes little by a throughput that changes much, and the measured power does not follow the throughput on these surveys.

Parameters and how they were set. $K_s = 55$ and $K_2 = 1.0$ from the guideline; the closed-circuit value is the product's choice, and an open-circuit factor would only rescale the fitted M_{ic}; the two fitted scalars per fold. The exponent $f(x)$ is not constant over crusher sizes: it is -0.315 at 20,000 μm and -0.435 at 140,000 μm, so the term $x^{f(x)}$ falls steeply with size and the size reduction term is dominated by the product size x_2. With the sizes of the surveys (x_1 from 47,000 to 140,000 μm and x_2 from 21,230 to 36,600 μm, the backbone's P_{80}), $K_s(x_1 x_2)^{-0.2}$ is below one, so S_c applies.

Cost. $O(1)$ per survey and an $O(n)$ fit per fold. *Assumptions and limits.* Only the conventional-crusher branch of the method is used: no tumbling-mill terms, no open-circuit K_2 in the fit (it would be absorbed by M_{ic}), and no size-distribution correction. The method is not part of the live engine and is not shown as a predictor in the App.

Implementation and tests. `model/crusher/morrell.py` (`f_exponent`, `size_term`, `hardness_parameter`, `crusher_specific_energy_kwh_t`, `crusher_power_kw`) and `hp500/morrell_fit.py` (`design_column`, `fit_through_origin`, `fit_with_no_load`). The tests check equation B4 against a hand computation, the unit of the result (linear in M_{ic}, kWh/t, times t/h gives kW), the rule on S_c, the refusal of non-positive sizes, the recovery of a planted M_{ic} and no-load power, and that the held-out numbers are reported as they come.

6.9 The Andersen-Whiten fit calibrated to the HP500 surveys

Theory and source. The one calibrated lane of the product is the secondary cone run with the calibration that Rocha, Campos, Silva and Tavares published for a Metso HP500 crushing itabirite iron ore [11], at the ten surveys of their Table A1. On this lane the surveys vary only the setting and the feed f_{80}; the throw (34 mm) and the speed (360 rpm) are machine constants, the feed is rebuilt from the measured f_{80} as a Rosin-Rammler feed with $n = 0.9$, the ore competence is the compact itabirite $A \times b = 677.3$ (metadata only, because the calibrated t_{10} does not use it) and the work index is the blend value 9.0 kWh/t. The engine then replaces three synthetic relations by the fitted ones, with sizes in millimetres,

$$K_1 = 0.23\,s + 0.30\,f_{80}, \qquad K_2 = \max(12.0 + 0.55\,s + 0.40\,f_{80},\ 1.05\,K_1)\,, \qquad K_3 = 2.3, \tag{25}$$

independent of the throw, and

$$t_{10}\,[\%] = 64 - 0.12\,s - 0.23\,f_{80}, \qquad Q = Q_T\,\frac{s + T/2}{g_{ref}}\,\sigma_N e^{1-\sigma_N}, \quad Q_T = 647 \text{ t/h}, \tag{26}$$

with the power of Equation (21). The constants are carried from the product's code and its earlier documentation; the publisher page of the paper refused automated access to every documenting attempt, so their correspondence to the numbered tables of the paper (Tables 1, 4 and 5 in the code's attributions) is not re-verified here.

The backbone. The calibrated engine evaluated at each survey is the backbone (Table 2). Its throughput depends only on the setting at the fixed throw and speed: from 936.45 t/h at 38 mm to 1225.89 t/h at 55 mm, linear in the setting. Its product P_{80} ranges from 21.23 mm (survey 3) to 36.60 mm (survey 7), between 0.35 and 1.1 times the setting, the physical relation it is graded by since no size distribution is measured. In sample, the capacity factor of the backbone with the published Q_T reproduces the measured feed rate with a mean absolute percentage error of 11.55% ($R^2 = 0.458$). Survey 3 is flagged pass-through and invalid, because its measured f_{80} (47 mm) reads as 47.15 mm on the grid, below its setting of 48 mm; it is kept in every evaluation, as a limit of the calibrated lane. The largest in-sample deviation of the throughput is 26.24% (survey 1, whose measured 1639 t/h is the largest feed rate of the set).

The fitted breakage on the surveys. The fitted t_{10} of Equation (26), evaluated with the grid F_{80} of each survey, ranges from 32.1% (survey 7, the coarsest feed, grid $F_{80} = 118.9$ mm at a 38 mm setting) to 47.4% (survey 3, the finest feed), so the cap of 44% of Section 6.2 binds on two surveys, survey 3 and survey 5 (44.06%, just above it), and on no other. The coarser the feed, the lower the fitted fineness of the progeny, which is the direction of the fit: the f_{80} coefficient of -0.23 points per millimetre dominates the setting coefficient of -0.12 over the range of the surveys. Because the product size is not measured, none of these values is checked against data; they enter the two measured targets only through the Bond term of the power.

Parameters and how they were set. Every constant of Equations (25) and (26) is the published fit; the nominal throw and speed are the product's assumptions (a representative mid-point of the HP500 throw

range and the secondary cone's capacity optimum); the feed exponent is the product's assumption. The calibrated support is the range of the surveys, a setting of 38 to 55 mm and a feed f_{80} of 47 to 140 mm; outside it the envelope check flags extrapolation.

Cost. One engine evaluation per survey. *Assumptions and limits.* The fit is the paper's: the product holds it to the ten surveys out of sample and does not perform the calibration. One machine and one ore, so the lane does not transfer.

Implementation and tests. `model/crusher/hp500.py` (`HP500_NOMINAL`, `Hp500Scalars`, `operating_from_survey`), `classification.py` (`hp500_params`), `breakage.py` (`t10_calibrated`), `capacity.py` (`HP500_QREF`) and `hp500/backbone.py`, which writes the backbone document. The acceptance criteria are that the calibrated engine at each survey equals the backbone document and that the ten surveys are physically plausible to the envelope check (`test_backbone_is_the_engine`, `test_envelope_gate_the_ten_real_hp500_surveys_are_physically_plausible`, which uses a proxy feed and therefore does not see the grid effect of survey 3).

6.10 The setting-only line

Theory. The baseline that every other predictor of the surveys is held to is an ordinary least-squares line of one target on the closed-side setting alone, refit on the training surveys of each fold:

$$\hat{y}_{t,i} = a_{t,-i} + b_{t,-i}\, s_i, \qquad (a_{t,-i},\, b_{t,-i}) = \arg\min_{a,b} \sum_{j \in \mathcal{T}_{-i}} \left(y_{t,j} - a - b\, s_j\right)^2, \tag{27}$$

for throughput and for power, with the closed form $b = \sum_j (s_j - \bar{s})(y_j - \bar{y}) / \sum_j (s_j - \bar{s})^2$ and $a = \bar{y} - b\bar{s}$ over the nine training surveys. It carries no physics and no engine. Its value is that the claim of any other method is the margin over it: the measured power correlates with the setting at -0.82, so a line on the setting already captures most of what the ten surveys say about power. Fitted on all ten surveys, the lines are $P = 686 - 8.49\, s$ and $Q = -351 + 31.92\, s$ (Figure 1), so each millimetre of setting is worth about 8.5 kW and 32 t/h on this machine and ore.

Held-out residuals in closed form. For an ordinary least-squares fit the leave-one-out residual has a closed form, which makes the held-out error of the line transparent. With the leverage of survey i on the setting,

$$h_{ii} = \frac{1}{n} + \frac{(s_i - \bar{s})^2}{\sum_{j=1}^{n} (s_j - \bar{s})^2}, \qquad y_{t,i} - \hat{y}_{t,i} = \frac{e_{t,i}}{1 - h_{ii}}, \tag{28}$$

where $e_{t,i}$ is the residual of the line fitted on all ten surveys, the strict held-out residual is the in-sample residual inflated by $1/(1 - h_{ii})$. On the survey table the leverages range from 0.103 (the three surveys at 48 mm, closest to the mean setting of 46.9 mm) to 0.324 (survey 7 at 38 mm, the tightest setting); surveys at the ends of the setting range are therefore the ones whose held-out errors grow most. Evaluating Equation (28) on the table reproduces the document's numbers exactly: an in-sample MAPE of 10.03% and a held-out MAPE of 12.78% on power, and 10.60% and 13.67% on throughput, which are the leaky and strict values of Table 11. The optimism of the leaky evaluation of the line is thus a pure leverage effect of ten points on one feature, with no fitting freedom to absorb a survey; the gap of the more flexible predictors in that table is larger because they also absorb part of the held-out survey's noise. These numbers are computed by `make_tables.py` from the survey table, as a check on the protocol, and agree with the committed document.

Why it is the baseline. A constant mean ignores the one lever that the surveys vary most, and a free-form fit on the setting and f_{80} spends a second coefficient on a variable that, on these data, adds no held-out skill on power. An exploratory screen of predictor sets on the same folds, run before the line was added to the product, placed the setting alone ahead of the setting with f_{80}, of $1/s$, of the setting with the measured throughput, and of the measured throughput times the Bond size term; the line was then added as a named predictor with its own acceptance criterion. The screen is the reason for the choice and not a result of the report, because several forms were tried on ten points.

Parameters. Two coefficients per target and fold, both fitted. *Cost.* $O(n)$ per fold. *Assumptions and limits.* Linear in the setting over 38 to 55 mm; it has no meaning outside the surveyed range, and it predicts no product size. Its power coefficient is not a mechanism: on these surveys the feed rate is set by the plant together with the setting (correlation +0.81), so the line on the setting also carries the effect of the feed rate on power, and a plant that changed the feed rate at a fixed setting would break it. It is a description of how this machine was operated over the surveyed period, and it is exactly the kind of relation a calibrated mechanistic model must reproduce before it can claim to add anything.

Implementation and tests. `hp500/blin.py` (`fit_line`, `predict_line`), called by `hp500/loo.py`. The acceptance criterion is that it reproduces the held-out errors of 12.78% (power) and 13.67% (throughput) of the exploratory screen on the same folds, and that a survey planted with an absurd target moves every other fold and not its own (`test_b_lin_reproduces_the_dossier_numbers`, `test_adversarial_a_fold_never_sees_its_held_out_survey`).

6.11 CrusherCal: leave-one-survey-out evaluation with controls and a predictive band

What it is. CrusherCal is the evaluation protocol of the HP500 lane: the predictors of the two measured targets are fitted and scored on the same ten folds, with negative controls, a leaky variant to expose optimism, and an 80% predictive band whose coverage is computed out of sample. It is a protocol and not a predictor of its own; every number it produces is computed from the committed backbone and the survey table.

The predictors. For each target, in each fold, on the nine training surveys:

- *Calibrated backbone.* The engine with its global scalars refit. For throughput, $\hat{Q}_i = Q_{T,-i}\,\phi_i$ with ϕ_i the engine's capacity factor (the backbone throughput over 647 t/h) and $Q_{T,-i} = \sum_j \phi_j Q_j / \sum_j \phi_j^2$; for power, the form of Equation (21) with (ζ, P_n) refit by ordinary least squares on the Bond draws of the training surveys.
- *Backbone with a bounded residual.* A ridge regression (penalty $\lambda = 1$, intercept not penalised) of the backbone residual on the standardised setting and f_{80}, evaluated at the held-out survey and clipped to $\pm 20\%$ of the backbone value, added to the backbone.
- *Free-form fit.* Ordinary least squares of the raw target on the setting and f_{80}, with no physics.
- The setting-only line (Section 6.10) and, for power, the two Morrell variants (Section 6.8).
- *Controls.* The constant mean of the training surveys; and a label-shuffle control, the bounded residual trained on the measured values permuted with a seeded generator, which must not beat the backbone.

The ridge fit of the bounded residual and its clipped correction are

$$\hat{\boldsymbol{\beta}}_{-i} = \left(\mathrm{X}_{-i}^{\top}\mathrm{X}_{-i} + \lambda \mathrm{D}\right)^{-1} \mathrm{X}_{-i}^{\top}\mathbf{r}_{-i}, \qquad \hat{y}_i^{\mathrm{res}} = \hat{y}_i^{\mathrm{bb}} + \mathrm{clip}\Big(\mathbf{x}_i^{\top}\hat{\boldsymbol{\beta}}_{-i},\ -0.2\,|\hat{y}_i^{\mathrm{bb}}|,\ 0.2\,|\hat{y}_i^{\mathrm{bb}}|\Big), \tag{29}$$

with X_{-i} the standardised setting and f_{80} of the training surveys with a column of ones, $\mathbf{r}_{-i}$ their backbone residuals, D the identity with a zero in the intercept's position, and $\mathbf{x}_i$ the held-out survey's row standardised with the training mean and standard deviation.

The band. The 80% interval of the backbone is its prediction plus or minus 1.2816 standard deviations of its held-out residuals,

$$\hat{y}_{t,i} \ \pm\ 1.2816\,\sigma_{-i}, \qquad \sigma_{-i}^2 = \frac{1}{n-2}\sum_{j\neq i}\left(r_j - \bar{r}_{-i}\right)^2, \quad r_j = y_{t,j} - \hat{y}_{t,j}, \tag{30}$$

where r_j are the strict held-out residuals and σ_{-i} their sample standard deviation over the other nine surveys, so the survey whose coverage is checked never contributes to its own band (nested coverage). The document also reports the in-sample coverage, in which σ comes from the same ten residuals it is checked against, only to show the difference. The factor 1.2816 is the two-sided normal quantile of a central 80% interval. The same nested band, applied to the residuals of the setting-only line and of the free-form fit, gives those predictors the bands that CrusherBayes is compared with.

Strict and leaky folds. Every fit is run twice: strict, on the nine training surveys, and leaky, on all ten including the held-out one. The difference between the two errors is the optimism of an evaluation that lets a survey into its own fit.

Parameters and how they were set. The clip fraction 0.20 (a physical bound on a correction to a physics prediction), the ridge penalty 1.0 (to keep a fit on nine points humble), the seed 42; all three were set before the first run and are recorded in the document. The expectation stated before the run was that the bounded residual would give a marginal or null improvement over the backbone; Section 8 reports what came out. *Cost.* Per target and mode, n least-squares fits of at most three coefficients for each predictor; the engine

Algorithm 2 CrusherCal: the strict and leaky leave-one-survey-out evaluation of one target, as `hp500/loo.py` runs it.

Require: surveys $(s_j, f_{80,j}, y_j)_{j=1}^{n}$; backbone capacity factors ϕ_j or Bond draws $P_{d,j}$; seed 42
Ensure: held-out predictions of every predictor, their scores, the band and its nested coverage

$\tilde{\mathbf{y}} \leftarrow$ a seeded permutation of $\mathbf{y}$ ▷ label-shuffle control
for mode $\in$ {strict, leaky} **do**
  for $i = 1$ **to** n **do**
    $\mathcal{T} \leftarrow \{1..n\} \setminus \{i\}$ if strict, else $\{1..n\}$
    fit the backbone scalars on $\mathcal{T}$; $\hat{y}_i^{\mathrm{bb}} \leftarrow$ backbone at survey i
    fit the ridge residual on $\mathcal{T}$; $\hat{y}_i^{\mathrm{res}} \leftarrow \hat{y}_i^{\mathrm{bb}} + \mathrm{clip}(\hat{r}_i, \pm 0.2\,|\hat{y}_i^{\mathrm{bb}}|)$
    fit the free-form fit, the setting-only line and, for power, both Morrell variants on $\mathcal{T}$; predict survey i
    $\hat{y}_i^{\mathrm{mean}} \leftarrow$ mean of y over $\mathcal{T}$; refit the residual on $\tilde{\mathbf{y}}$ for the control
  end for
  score every predictor: MAPE, RMSE, R^2, largest deviation
end for
$r_j \leftarrow y_j - \hat{y}_j^{\mathrm{bb}}$ (strict); for each i: $\sigma_{-i} \leftarrow \mathrm{sd}(r_{j \neq i})$, $\mathrm{covered}_i \leftarrow |r_i| \leq 1.2816\,\sigma_{-i}$
return predictions, scores, σ, half-width, in-sample and nested coverage

is evaluated once per survey, by the backbone, and not inside the folds. *Limits.* With $n = 10$, one survey moves a coverage by 0.1, and a difference below about three points of MAPE is within noise; the protocol is not a significance test.

Implementation and tests. `hp500/loo.py` (`fold_indices`, `fit_capacity_base`, `fit_power_form`, `fit_residual`, `fit_freeform`, `predictive_band`, `nested_band_coverage`, `band_factor`). The acceptance criteria are that the label-shuffle control does not beat the backbone, that the backbone beats the constant mean on throughput, that the leaky-strict gap is exposed and that the band's coverage is reported (`test_adversarial_the_label_shuffle_control_does_not_beat_the_backbone`, `test_tph_backbone_has_real_skill`, `test_adversarial_the_leaky_folds_expose_the_optimism_gap`).

6.12 The envelope check

Principle. An empirical model must flag implausible and out-of-calibration inputs and never answer them with a confident number. The envelope check applies five rules to an operating point and the feed size the engine reads; it is also the baseline of the anomaly slot, the answer given when no valid autoencoder is released.

Rules. Writing $\mathbb{1}[\cdot]$ for an indicator, the point is implausible, and therefore invalid, when

$$\iota(\mathbf{u}) = \mathbb{1}[s \leq 0] \ \vee \ \mathbb{1}[x_{63} \leq 0] \ \vee \ \mathbb{1}[F_{80} > 0 \ \wedge \ s \geq F_{80}], \tag{31}$$

with the codes `css_nonpositive`, `feed_nonpositive` and `f80_lt_css`; and a point on the calibrated lane is extrapolating, without being invalid, when

$$\xi(\mathbf{u}) = \mathbb{1}[s \notin [38, 55]\ \mathrm{mm}] \ \vee \ \mathbb{1}[F_{80}^{\mathrm{decl}} \notin [47, 140]\ \mathrm{mm}], \tag{32}$$

with the codes `css_outside_calibrated` and `f80_outside_calibrated`, where $F_{80}^{\mathrm{decl}} = x_{63}(\ln 5)^{1/n}$ is the declared feed size rounded to a micrometre (the untruncated value, so that a feed coarser than the grid is judged by its own size and a survey on the 140 mm edge is not pushed over it by round-off). The calibrated support is the range of the ten surveys of Rocha et al. [11].

Sourced warnings. Two further codes run after the check in both engines on the operating point and the computed reduction ratio. A setting below the vendor minimum of the machine class writes `css_below_vendor_minimum`: 16 mm for the secondary cone (Metso HP500 standard liners), 6 mm for the tertiary and short-head cones (HP500 short head) [40], 40 mm for the jaw (the smallest minimum of the Nordberg C Series) [41], and none for the gyratory, for which no source was found, so the check is skipped rather than guessed. A reduction ratio F_{80}/P_{80} above the typical ceiling of one stage writes `reduction_ratio_above_typical`: 5 for the HP standard chamber, 4 for the short-head chamber [40], and 4 for the jaw and the gyratory [42]. Both bounds are strict and both are warnings: they never change the validity, because the vendor calls the minimum setting soft (the setting at which the crusher runs without ring bounce, which can change with the rock) and the ratios are typical ranges, not maxima. Of the seventeen circuit cases, CK1,

S05, S06, T01, T02, H01 and H02 carry the ratio warning and S06, at 12 mm, also the setting warning; the ten surveys carry neither.

Committed behaviour. The controls CI1 and CP1, the gyratory case G02 and the survey HP500-03 carry `f80_lt_css` and are invalid. On the synthetic lane the check flags only implausible inputs, so a plausible point outside the training range of the learned models passes it; the autoencoder (Section 6.14) scores distance from the training manifold, the complementary check.

Why the declared size. The extrapolation rule of Equation (32) tests the declared feed size and not the grid value because the two differ for coarse feeds: a feed with a declared F_{80} of 180 mm and the exponent 0.9 of the survey feeds reads 135.8 mm on the 256 mm grid, inside the calibrated support, and testing the grid value would let a feed coarser than any survey pass as supported. The rounding to a micrometre is there because the feed of a survey is rebuilt from its f_{80} by the inverse of Equation (4), and the floating-point round trip can return a value a few units in the last place above 140 mm for the survey on the edge of the support. The implausibility rule, in contrast, uses the grid value, because it asks whether the engine itself can grip the feed it works with.

What the check does not see. On the synthetic lane the check knows three implausibility rules and nothing about the ranges over which the learned models were trained, so a secondary cone at a 90 mm setting with a plausible feed passes it while lying at the edge of the surrogate's training range; the applicability rule of the model slots (Section 6.13) and the autoencoder score handle that case.

Cost. $O(1)$. *Limits.* Five rules and two warnings, not the distribution of the data; a point that passes is not thereby accurate.

Implementation and tests. `model/crusher/envelope.py` (`check_envelope`, `HP500_ENVELOPE_CSS_MM`, `HP500_ENVELOPE_F80_MM`) and `model/crusher/bounds.py` (`check_bounds`). The acceptance criteria are that an implausible point is flagged and invalid and that a calibrated point outside the support is flagged as extrapolating (`test_envelope_gate_an_implausible_point_is_flagged_and_marked_invalid`, `test_envelope_gate_calibrated_lane_flags_out_of_support_as_extrapolating`); further tests cover the strict bounds, the gyratory skip and the unchanged validity of the warnings.

6.13 The learned surrogate of the engine

Theory and source. A learned surrogate of a phenomenological comminution model gives the same answers in one forward pass and makes exploration of the operating space instant; Koh, Amini, McLachlan and Beaton trained a deep network to approximate the phenomenological models of a comminution circuit for faster simulation [30]. The product follows that line for one crusher. The surrogate emulates the engine, which is itself a fast model; its purpose in the studio is a learned model that a reader can compare with the engine, labelled as such, and the evidence of how well such a model reproduces the engine on an independent draw. It does not emulate a plant.

Architecture. A multilayer perceptron with GELU activations,

$$\hat{\mathbf{y}}_z = \mathrm{W}_4\,\mathrm{g}(\mathrm{W}_3\,\mathrm{g}(\mathrm{W}_2\,\mathrm{g}(\mathrm{W}_1\mathbf{v}+\mathbf{b}_1)+\mathbf{b}_2)+\mathbf{b}_3)+\mathbf{b}_4, \qquad \mathrm{g}(a)=a\,\Phi(a), \tag{33}$$

with layer widths 11-64-64-32-10 (7,338 weights and biases) and Φ the standard normal distribution function. The input $\mathbf{v}$ is a one-hot block of the five machine classes followed by the six continuous inputs (setting, throw, speed, x_{63}, n and $A\times b$), each standardised by the mean and standard deviation of the training draw; the output is the standardised vector of P_{80}, P_{50}, P_{20}, the passing at 1, 4, 8, 16 and 32 mm, the throughput and the power, the three sizes fitted in $\log_{10}$ and read back as ten to the power. The Bond work index of a sweep point follows the ore, $W_i = \min(20, \max(8,\ 8 + 0.09\,(120 - A\times b)))$ kWh/t, so the model keeps six continuous inputs.

Training. The loss is the mean squared error in the standardised output space over the training draw $\mathcal{S}$,

$$\mathcal{L}(\mathrm{W}) = \frac{1}{|\mathcal{S}|\,10}\sum_{(\mathbf{v},\mathbf{y})\in\mathcal{S}} \|\hat{\mathbf{y}}_z(\mathbf{v}) - \mathbf{y}_z\|_2^2\,, \tag{34}$$

minimised by Adam at a learning rate of 2×10^{-3} with a weight decay of 10^{-5}, for 500 full-batch epochs, with torch seeded (12345) and single-threaded so that the run is deterministic on one machine (Algorithm 3). The training points are a Latin-hypercube sweep of the canonical engine, 1400 per machine class over that class's own ranges (Table 20; a gyratory never runs a 6 mm setting), generated by the same deterministic generator (mulberry32) as the earlier Node script so that a seed yields the same points; after dropping

Algorithm 3 Training, export and held-out evaluation of the two learned models (`stages/train.py`, `stages/evaluate.py`).

Require: engine $\mathcal{F}$; per-class ranges; seeds 12345 (training) and 67890 (held out)
draw 1400 Latin-hypercube points per class with seed 12345; label each with $\mathcal{F}$; drop degenerate points (6430 remain)
draw 480 points per class with seed 67890; label; drop degenerate points (2196 remain)
standardise inputs and outputs with the training draw's statistics (float32); fit $\log_{10}$ of the three sizes
for epoch $= 1$ **to** 500 **do** ▷ surrogate, full batch
    Adam step on $\mathcal{L}$ of Equation (34) (rate 2×10^{-3}, weight decay 10^{-5})
end for
for epoch $= 1$ **to** 400 **do** ▷ autoencoder, full batch
    $\tilde{\mathbf{h}} \leftarrow \mathbf{h} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim N(0, 0.05^2\mathrm{I})$; Adam step on $\|\mathrm{AE}(\tilde{\mathbf{h}}) - \mathbf{h}\|^2$
end for
threshold $\leftarrow$ 99th percentile of the training reconstruction scores
export both to ONNX (opset 17); quantise the surrogate to int8; record the export parity and the SHA-256 of each graph
evaluate the shipped graphs through onnxruntime on the held-out draw: R^2 and MAPE per output; false-alarm and detection rates

degenerate draws 6430 remain. The network is exported to ONNX (opset 17, a dynamic batch axis) and quantised to an int8 graph by dynamic quantisation of the weights.

Held-out evaluation. The held-out set is an independent second Latin-hypercube draw (seed 67890, 480 points per class, 2196 after dropping degenerate draws), not a split of the training draw. Every number is computed through onnxruntime on the shipped float32 graph against the canonical engine, and the MAPE also on the int8 graph, so the metrics describe what the browser runs. Monotonicity is checked on fifteen curves (16 settings across each class's range, at the 25, 50 and 75% points of its other inputs): P_{80} must not fall by more than 0.001 mm as the setting opens. The export parity is recorded twice: the trained torch module against the float32 graph on the held-out draw, and the float32 graph against the int8 graph. The results are in Section 8.7.

Cost. A forward pass is about 7.3×10^3 multiply-adds; training both models, with the two sweeps and the evaluation, took 48 s on the workstation's processor. *Applicability.* The slot answers only an operating point that is not calibrated and lies inside the training ranges; otherwise the engine answers, labelled *calibrated case* or *outside the training range*. The surrogate returns one value per output and no uncertainty.

Implementation and tests. `model/surrogate.py`, `model/sweep.py` (`RANGES`, `draw`), `stages/train.py`, `stages/evaluate.py`, `model/bundles.py` (the release rule: the slot is released only when its contract validates and every graph exists with its SHA-256 and size), and `model/crusher/encoding.py`, which reproduces the browser's encoding byte for byte. The acceptance criteria are an R^2 for every output computed from the shipped graph, P_{80} monotone in the setting on every class, and the engine answering, labelled, when the bundle is invalid (`test_metrics_validate`, `test_adversarial_swap_engine_baseline_and_surrogate`).

6.14 The denoising autoencoder as an extrapolation score

Theory. An autoencoder trained to reconstruct the signature of engine results reconstructs badly a result that lies far from the manifold it was trained on; its reconstruction error is then an anomaly score that says the learned surrogate is extrapolating. Training it on inputs corrupted by noise and asking for the clean input (a denoising autoencoder) makes the reconstruction a projection onto the training manifold rather than an identity map. A product of this kind is motivated by the use of the power-draw signal of a cone crusher to diagnose feed segregation, a real anomaly of the machine [37]; the product's score detects distance from the engine's sampled envelope, not a mechanical fault.

Architecture and score. The encoder maps the fourteen standardised features $\mathbf{h}$ (the $\log_{10}$ of P_{80}, P_{50} and P_{20}, the passing at five sizes, throughput, power, reduction ratio, $\log_{10} F_{80}$, specific energy and the feed exponent) through widths 14-16-6 with GELU, and the decoder maps back through 6-16-14 (692 weights and biases). The score and the alarm are

$$a(\mathbf{h}) = \frac{1}{14}\,\|\mathrm{AE}(\mathbf{h}) - \mathbf{h}\|_2^2\,, \qquad \text{alarm} \iff a(\mathbf{h}) > a_{99} \ \vee \ a(\mathbf{h}) \notin \mathbb{R}, \tag{35}$$

with $a_{99} = 0.24454$ the 99th percentile of the scores of the training draw (whose mean score is 0.05181). Training uses the same 6430 points as the surrogate, Adam at 2×10^{-3} with weight decay 10^{-5} and 400 full-batch epochs (Algorithm 3), on the loss

$$\mathcal{L}_{\mathrm{AE}} = \frac{1}{14\,|\mathcal{S}|} \sum_{\mathbf{h} \in \mathcal{S}} \|\mathrm{AE}(\mathbf{h} + \boldsymbol{\epsilon}) - \mathbf{h}\|_2^2\,, \qquad \boldsymbol{\epsilon} \sim N\big(\mathbf{0},\ 0.05^2\,\mathrm{I}_{14}\big)\,, \tag{36}$$

with a fresh noise draw at every epoch, so that the network learns to map a perturbed signature back to the training manifold. The noise of 0.05 is small against the unit standard deviation of each standardised feature, so the reconstruction of a training point stays close to the point (the mean training score of 0.052 is about a twentieth of the variance of a feature), while a point that no training signature resembles is mapped to the nearest one and scores high.

Why the features are engine outputs. The fourteen features are results of the engine, not its inputs: the score asks whether the product sizes, the readouts, the throughput, the power, the reduction ratio and the specific energy of a point are jointly typical of the training draw. A point whose inputs are each inside their training ranges but whose combination gives an unusual product, for example a fine feed at a wide setting, can therefore raise an alarm, while a point whose inputs lie outside the ranges but whose results resemble the training results does not. This is the complement of the envelope check, which tests inputs one at a time against rules.

Evaluation protocol. The false-alarm rate is the share of the 2196 held-out points that raise an alarm. The detection rate is measured on 2196 planted points, one per held-out point, whose operating inputs are moved outside that class's training range and then run through the engine: by the held-out index modulo 18, one of the six continuous inputs is moved above its upper bound by half the range width (patterns 0 to 5), or below its lower bound to half the bound (6 to 11), or two neighbouring inputs are moved, the first above and the second below (12 to 17); a planted point whose engine result has a non-finite feature is left out. The envelope check is scored on the same points; on the synthetic lane it flags only implausible inputs, so it is the baseline the score must complement.

Parameters and how they were set. The bottleneck width 6 and the noise 0.05 are the product's choices; the threshold is set by the training scores, so the expected false-alarm rate on new data from the training distribution is about one percent. *Cost.* About 6.6×10^2 multiply-adds per score. *Limits.* The training data are engine outputs; the one percent is measured on one held-out draw and is not a guaranteed rate on other data.

Implementation and tests. `model/psd_ae.py`, `stages/train.py`, `stages/evaluate.py` and `model/bundles.py` (`anomaly_score`, `reconstruction_score`, `applicability`). The acceptance criteria are the false-alarm and detection rates on the held-out and planted points reported beside those of the envelope check, and the envelope check answering, labelled, when the bundle is invalid (`test_metrics_report_the_anomaly_comparison_and_the_planted_points`, `test_adversarial_swap_envelope_baseline_and_autoencoder`).

6.15 The drift projection: hours until the product leaves its size target

Theory and source. As the liners of a cone wear, the closed-side setting opens and the product coarsens; plants compensate by re-setting the machine. One sourced cone-liner wear model is the shear-augmented Archard form of Lindqvist and Evertsson [38], extended with a time term by Ma, Fan and He [39], in which the wear of a crushing zone after t hours is the wear near the setting point scaled by the zone's relative stress and growing with the cumulative tonnage; in the plant of that study the setting was re-set every 45 h. The parameters of that model, a liner wear rate in grams per tonne, a liner life in hours or a drift of the setting in millimetres per hour were not found in any source the product could read, so the product models no wear and asserts no drift rate. What it computes is the opening of the setting that a product size target allows, from the engine's own P_{80}, and the hours that opening lasts at a drift rate the user supplies:

$$s^* = \min\left\{ s' \geq s_0 : P_{80}(\mathbf{u}; s') \geq P_{80}^{\mathrm{target}} \right\}, \qquad \Delta s = s^* - s_0, \tag{37}$$

$$t_{out} = \frac{\max(\Delta s,\ 0)}{r}, \qquad r > 0, \tag{38}$$

with s_0 the current setting, $P_{80}(\mathbf{u}; s')$ the engine's product size at the operating point with the setting replaced by s', and r the drift rate in mm/h. The view plots t_{out} against r on logarithmic axes from 0.01 to 1 mm/h, with the target as a control that opens at the case's P_{80} plus 10% (a control value, not a claim) and the 45 h interval of that plant as the one reference line (Figure 12 in Section 8.8).

Algorithm 4 The setting at which the engine's P_{80} reaches a target (`css_at_p80_target`).

Require: operating point $\mathbf{u}$ with setting s_0; target $P^* > 0$; ceiling $s_{\max} > s_0$
Ensure: s^*, or no answer

$p_0 \leftarrow P_{80}(\mathbf{u}; s_0)$; **if** $\mathbf{u}$ is invalid **then return** no answer
if $p_0 \geq P^*$ **then return** s_0 ▷ target already reached: zero hours
end if
$(s_{\text{last}}, p_{\text{last}}) \leftarrow (s_0, p_0)$
for $i = 1$ **to** 63 **do**
    $s \leftarrow s_0 + (s_{\max} - s_0)\, i/63$; **if** the engine marks s invalid **then break**
    if $P_{80}(\mathbf{u}; s) < p_{\text{last}} - 4\,\text{ulp}(p_{\text{last}})$ **then return** no answer ▷ not monotone
    $(s_{\text{last}}, p_{\text{last}}) \leftarrow (s, P_{80}(\mathbf{u}; s))$
end for
if $p_{\text{last}} < P^*$ **then return** no answer ▷ never reached at a valid setting
$(l, h) \leftarrow (s_0, s_{\text{last}})$
for $k = 1$ **to** 60 **do**
    $m \leftarrow (l + h)/2$; **if** m is invalid or $P_{80}(\mathbf{u}; m) \geq P^*$ **then** $h \leftarrow m$ **else** $l \leftarrow m$
end for
return $(l + h)/2$

Numerical scheme. Algorithm 4 finds s^* by a bracket and a bisection. It samples the engine on 64 settings from s_0 to a ceiling $s_{\max}$, stops at the first setting the engine marks invalid (for example a pass-through feed), requires that P_{80} never falls over the valid samples by more than four units in the last place, and bisects 60 times between s_0 and the last valid sample. Sixty halvings of a bracket of at most 240 mm leave an interval below 10^{-15} mm, so the result is limited by floating-point resolution and both engines run the same iterations. The ceiling is the upper bound of the class's training range: 90 mm for the secondary cone, 22 tertiary, 16 short-head, 160 jaw and 240 gyratory.

The shared table. The 45 rows of the reference table hold, for each of nine operating points, three targets above the current P_{80} (+3, +10 and +30%), one target already exceeded and one that is never reached. For S01 they give the settings 33.043, 35.212 and 40.673 mm for the first three, the current 32 mm for the exceeded target (zero hours), and no answer for a target of 10^6 mm, which no valid setting below the 90 mm ceiling reaches. Both engines must reproduce every row to 10^{-6} mm, and a further test, which does not use the table, evaluates the engine at each returned setting and requires its P_{80} to equal the target. A patched engine whose P_{80} falls over the bracket must return no answer, so the monotonicity check is itself tested.

Parameters. None with a default for the drift: no function and no control of the view has a default rate. The 64 samples, the 60 halvings and the tolerance of four units in the last place are the engine's. *Cost.* At most $64 + 60$ engine evaluations, about 0.17 s at the median engine time. *Limits.* Not a wear model and not a set point; the projection is the engine's (or the calibrated engine's) P_{80} at the opened setting, with the limits of the engine, and it ignores the profile distortion of worn liners and their work hardening.

Implementation and tests. `css_at_p80_target` and `hours_to_target` in `model/crusher/`, and their TypeScript port. The acceptance criteria are that both engines return the same setting on a shared table of 45 rows (nine operating points, including the calibrated lane, at five targets each) to 10^{-6} mm, that the engine's P_{80} at the returned setting is the target, and that a target already exceeded, never reached, an invalid point and a non-monotone P_{80} give zero hours or no answer, never a guess (`test_the_shared_table_is_reproduced_to_1e6_mm`, `test_a_target_never_reached_below_the_ceiling_is_none`).

6.16 CrusherBayes: Bayesian calibration with a structural discrepancy

Idea. The calibration of Rocha et al. is a point fit [11]. CrusherBayes, the method the product proposes, places a prior on the engine's fitted scalars, adds an explicit model-form discrepancy per target in the form of Kennedy and O'Hagan [31], and returns a posterior predictive distribution for a survey that was not used in the fit. The prediction stated before the computation was that this distribution would be better calibrated and sharper than the residual band of CrusherCal, and would show which scalars the ten surveys identify and which stay at their prior. The method is derived and analysed in its own manuscript [36]; this subsection gives what a reader of the product needs to reproduce it. The searches that preceded it found no Bayesian calibration with discrepancy of a cone-crusher population-balance model evaluated held out; they were not exhaustive (Section 11).

Algorithm 5 One CrusherBayes posterior and its held-out prediction (`hp500/crusherbayes.py`, `_fit`).

Require: training surveys $\mathcal{T}_{-h}$ (or all ten), held-out survey h (or none), seed; priors of Table 7
standardise the setting and f_{80} on $\mathcal{T}_{-h}$; build Z
$\log \pi(\boldsymbol{\theta}) \leftarrow \log p(\boldsymbol{\theta}) + \sum_{t\in\{Q,P\}} \log N\big(\mathbf{y}_t;\ f_t(\boldsymbol{\vartheta}),\ \sigma_t^2 \mathrm{I} + \tau_t^2 \mathrm{Z}\mathrm{Z}^\top\big)$ ▷ engine on every training survey
initialise 20 walkers at the prior centre plus 0.05 prior standard deviations of Gaussian noise
run 7000 steps of the ensemble sampler with DE (weight 0.8) and DE-snooker (weight 0.2) moves
$\hat{\tau}_{\max} \leftarrow$ largest integrated autocorrelation time after 1000 steps; converged $\leftarrow 6000 \geq 50\,\hat{\tau}_{\max}$
retain the chain after 1000 steps, thinned by 10 (12,000 samples)
if h is given **then**
    for each retained sample **do**
        compute the exact conditional of $\mathbf{d}_t$ (Equation (42)); predictive mean and variance at survey h
        draw 20 predictive values
    end for
    return mean of the predictive means; 10th and 90th percentiles of the draws; log predictive density
else
    draw $\mathbf{d}_t$ once per sample from Equation (42); **return** the samples for the contraction table
end if

Model. For survey i and target t (throughput in t/h, power in kW; P_{80} is not measured and is never a target),

$$y_{t,i} = f_t(\boldsymbol{\vartheta};\ s_i, f_{80,i}) + \delta_t(\mathbf{z}_i) + \varepsilon_{t,i}, \qquad \delta_t(\mathbf{z}) = d_{t0} + d_{t1}\, z_{\mathrm{CSS}} + d_{t2}\, z_{f_{80}}, \tag{39}$$

$$d_{tj} \sim N(0, \tau_t^2), \qquad \varepsilon_{t,i} \sim N(0, \sigma_t^2), \qquad \tau_Q = 140 \text{ t/h}, \quad \tau_P = 36 \text{ kW}, \tag{40}$$

where f_t is the engine at the eight scalars $\boldsymbol{\vartheta} = (Q_T, W_i, \zeta, P_n, s_1, s_2, K_3, \Delta t_{10})$ of Table 7, with s_1 and s_2 multipliers of K_1 and K_2 and Δt_{10} a shift of the constant 64 of the t_{10} relation, and $\mathbf{z}$ the setting and f_{80} standardised on the training surveys of the fold. The discrepancy scales are about 12% of the typical level of each target. Given $\boldsymbol{\vartheta}$ the discrepancy and the noise are Gaussian, so the discrepancy is integrated out exactly: with Z the 9×3 design matrix $[\mathbf{1}, \mathbf{z}_{\mathrm{CSS}}, \mathbf{z}_{f_{80}}]$ of the training surveys,

$$\mathbf{y}_t \mid \boldsymbol{\vartheta}, \sigma_t \ \sim\ N(f_t(\boldsymbol{\vartheta}),\ \boldsymbol{\Sigma}_t)\,, \qquad \boldsymbol{\Sigma}_t = \sigma_t^2\, \mathrm{I} + \tau_t^2\, \mathrm{Z}\mathrm{Z}^\top, \tag{41}$$

and the sampler explores ten dimensions, the eight scalars and the two log noise scales. The log posterior is the sum of the two Gaussian log densities of Equation (41), computed by a Cholesky factorisation of $\boldsymbol{\Sigma}_t$, and the truncated-normal log prior. The discrepancy coefficients are drawn afterwards from their exact Gaussian conditional,

$$\mathbf{d}_t \mid \boldsymbol{\vartheta}, \sigma_t, \mathbf{y}_t \ \sim\ N\big(\tau_t^2 \mathrm{Z}^\top \boldsymbol{\Sigma}_t^{-1} \mathbf{r}_t,\ \ \tau_t^2 \mathrm{I} - \tau_t^4\, \mathrm{Z}^\top \boldsymbol{\Sigma}_t^{-1} \mathrm{Z}\big)\,, \qquad \mathbf{r}_t = \mathbf{y}_t - f_t(\boldsymbol{\vartheta}), \tag{42}$$

and the predictive distribution of a held-out survey with standardised row $\mathbf{z}_h$ at one posterior sample is Gaussian with mean $f_t(\boldsymbol{\vartheta}; s_h, f_{80,h}) + \mathbf{z}_h^\top \mathbb{E}[\mathbf{d}_t]$ and variance $\sigma_t^2 + \mathbf{z}_h^\top \mathrm{Cov}[\mathbf{d}_t]\, \mathbf{z}_h$. The engine is its own simulator: at about 1.3 ms per evaluation no emulator is needed.

Priors. Each engine scalar has a truncated normal prior centred on the engine's own value, on the log scale where marked (Table 7): the nominal values are those of the source calibration and of the itabirite blend, and the widths are weakly informative choices of the author that bracket the quoted spans, because no published range of these fitted scalars for this ore was found. The noise has $\ln \sigma_Q \sim N(\ln 60, 1)$ and $\ln \sigma_P \sim N(\ln 15, 1)$, about 5% of the typical level, truncated to $[\ln 1, \ln 1000]$.

Sampler. The ensemble sampler of `emcee` 3.1.6 [35] with the differential-evolution moves of ter Braak [32] (weight 0.8) and the snooker update of ter Braak and Vrugt [33] (weight 0.2): gradient-free, as the engine requires. The affine-invariant stretch move of Goodman and Weare [34] was tried first and did not converge: its autocorrelation time stayed above 100 steps and grew with the chain, along the parameters the surveys do not identify. Algorithm 5 gives one posterior. Twenty walkers start at the prior centre perturbed by 0.05 prior standard deviations, run 7000 steps, discard 1000, and are thinned by 10, which leaves 12,000 samples. A posterior is recorded as converged when the post-burn chain is at least 50 integrated autocorrelation times long, the largest over the parameters; a posterior that fails is recorded as not converged and is never kept silently. The 80% interval of a held-out survey is the 10th to 90th percentile of 20 predictive draws per retained sample.

Evaluation and adoption rule. Strict leave-one-survey-out as in CrusherCal: one posterior per fold from nine surveys, the held-out survey only in the prediction, and one more posterior from all ten for the

Table 7: Priors of the eight CrusherBayes engine scalars: truncated normal around the engine's own value, on the scale shown, with the truncation on that scale. Nominal values are those of the source calibration and the itabirite blend; widths are the author's. Generated from `crusherbayes.json`.

Scalar	Unit	Nominal	Scale	Prior sd	Truncation (on the scale)
Q_T, capacity base	t/h	647.00	log	0.25	[5.704, 7.244]
W_i, blend work index	kWh/t	9.00	log	0.17	[1.609, 2.639]
ζ, power slope	–	1.30	log	0.23	[−0.693, 0.916]
P_n, no-load power	kW	110.00	linear	50.00	[0.000, 300.000]
s_1, multiplier of K_1	–	1.00	log	0.15	[−0.693, 0.693]
s_2, multiplier of K_2	–	1.00	log	0.15	[−0.693, 0.693]
K_3, classification exponent	–	2.30	linear	0.50	[1.000, 4.000]
Δt_{10}, shift of the constant 64	points	0.00	linear	4.00	[−15.000, 15.000]

contraction table, the ratio of posterior to prior standard deviation of every parameter. Held out, per target: the MAPE of the predictive mean, the RMSE, the mean log predictive density, the coverage of the 80% interval and its mean width, beside the setting-only line, the calibrated backbone and the free-form fit scored with their nested bands. The adoption rule, written before the run and applied by code, drops the method as a predictor if (a) its power MAPE is not below the line's by at least three points and its throughput MAPE does not beat the backbone's by at least three points, and (b) on either target its coverage lies outside 0.7 to 0.9 or its mean band is wider than the backbone's band. The contraction table is reported in either case. The results are in Section 8.6.

Cost. Each log-posterior evaluation runs the engine on the nine (or ten) training surveys, so one posterior costs $140{,}000 \times 9$ engine evaluations; the bake spreads the walkers of the eleven posteriors over the processor's cores. *Limits.* With ten surveys and two targets the posterior of most scalars is the prior; the discrepancy can absorb what the engine cannot express, which is the known confounding of calibration and discrepancy; and the power form of the engine is the weak part, so most of any gain on power may come from the discrepancy, which the setting-only line already approximates.

Implementation and tests. `hp500/crusherbayes.py` (`PRIORS`, `log_prior`, `_Problem.log_prob`, `_Problem.conditional`, `_Problem.predict`, `_fit`, `kill_criterion`); the engine takes the scalars through one optional argument whose defaults are the shipped values, so the TypeScript engine and the shared reference are untouched. The tests check that the default scalars are the shipped engine, that each prior is centred on the engine's value with a source, the prior bounds, that the folds never see their held-out survey, the seed, that the document is byte-identical and independent of the number of worker processes, that a short chain is recorded as not converged, the discrepancy marginal against a Monte Carlo of the model, and the adoption rule in every combination of its terms.

7 Experimental protocol

7.1 Cases and what each set of cases tests

Three sets of operating points serve three different questions, and no result of one set is read as evidence about another. The seventeen circuit cases test the behaviour of the coupled engine: whether it reproduces the size-reduction ladder of a crushing circuit and the trends of each lever, and whether its checks flag the controls. They are baked at their operating points and judged against the numeric expected ranges each case declares (for example, for S01, a P_{80} between 22 and 35 mm, a throughput between 520 and 780 t/h and a reduction ratio between 3.5 and 5.5), ranges written from engineering relations and not from the engine's own value. The ten HP500 surveys test prediction against measurement, for the two measured targets only, under the leave-one-survey-out protocol of Section 6.11. The two Latin-hypercube draws test the learned models against the engine. Each case also has two variants computed live in the browser, a tight setting (15% lower) and an open setting (15% higher), which the studio shows and which are not scored.

7.2 Metrics

For a target with measured values y_i and held-out predictions $\hat{y}_i$, $i = 1, \ldots, n$, the point scores are

$$\mathrm{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{\hat{y}_i - y_i}{y_i} \right|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2}, \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{43}$$

and the largest deviation $\max_i |\hat{y}_i - y_i|/y_i$ in percent. A held-out R^2 can be negative: a predictor worse than the mean of the measured values scores below zero. For an interval $[\ell_i, u_i]$ the coverage and the mean width are

$$\text{cov} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[\ell_i \le y_i \le u_i], \qquad \bar{w} = \frac{1}{n}\sum_{i=1}^{n}(u_i - \ell_i), \tag{44}$$

and the mean log predictive density of a predictive distribution with density p_i is $\overline{\text{lpd}} = \frac{1}{n}\sum_i \log p_i(y_i)$. For CrusherBayes p_i is the posterior predictive density, estimated as the log of the mean over the retained samples of the Gaussian predictive density at each sample; for a residual-band predictor it is the normal density with standard deviation σ_{-i} at its residual. The contraction of a parameter is the ratio of its posterior to its prior standard deviation on its natural scale; below 0.5 the surveys identify the parameter, and near one it stays at its prior. For the learned models, R^2 and MAPE are computed per output in the original units over the 2196 held-out points; the false-alarm rate is the share of held-out points that raise an alarm and the detection rate the share of planted points that do.

7.3 Baselines and controls

Every comparison has a baseline that a method must beat before it claims skill. On the surveys: the constant mean of the training surveys (no information), the label-shuffle control (a fit that learns noise, which must not beat the backbone), and the setting-only line (one feature, no physics), which is the bar for any claim on power. For the surrogate the baseline is the engine itself, which is what the slot answers with when its bundle is invalid; for the autoencoder the baseline is the envelope check. The leaky variant of every survey fit is reported beside the strict one, so that the optimism of letting a survey into its own fit is measured and not assumed. The published calibration is a baseline of a different kind: its own error on the surveys is not available to the product (Section 9.6), so the comparison with it is made by running its constants in the engine and refitting its scalars per fold, never by quoting a number. A predictor that does not beat the setting-only line on a target by more than the noise of ten surveys has no claim of skill on that target.

7.4 The protocol shared by every survey predictor

All predictors of the surveys use the same ten folds, the same targets in the same units, the same score functions and the same nested band construction (Section 6.11); CrusherBayes uses its posterior predictive interval in place of the band. In every fold every fitted quantity, the capacity base, the power-form constants, the ridge and free-form coefficients, the line, M_{ic} and the no-load power, and the whole CrusherBayes posterior, is refit on the nine training surveys, and the held-out survey enters only the prediction. A test plants an absurd target in one survey and checks that every other fold's prediction moves and the planted survey's own does not. With $n = 10$ the protocol has a resolution of 0.1 in coverage, and a difference below about three points of MAPE is treated as noise; the adoption rule of CrusherBayes uses that margin. No predictor is tuned on the held-out scores: every constant of every fit was set before the first run and is recorded in its document.

7.5 Seeds, repetitions, hardware and run times

The survey evaluation is deterministic given the seed 42, which fixes the label-shuffle permutation; CrusherBayes spawns one seed per posterior from a seed sequence started at 42, uses that seed plus one for the predictive draws of the posterior, and its document is byte-identical whatever the number of worker processes. The learned models use the training seed 12345 and the held-out seed 67890, with torch single-threaded. The particle-replacement check of Section 9 uses five seeds per parcel count. Each posterior is one chain of twenty walkers; its repetition is the eleven posteriors of the folds, whose convergence diagnostics are reported. The measured run times, on a Windows 11 workstation with Python 3.13 and Node 24, are in Table 5: 7.3 s for the bake before CrusherBayes, 48 s to train and evaluate both learned models on the processor, and a median of 1.34 ms per engine evaluation. No stage uses a graphics processor. The wall time of the CrusherBayes sampler is not recorded in the committed documents; its size, 140,000 log-probability evaluations per posterior, is (Section 5.4).

Table 8: The acceptance criterion of each method of the product, as the product's test suite checks it. A failing criterion fails the build; the accuracy of a method against the surveys is reported as a result and is not a criterion.

Method	Acceptance criterion
Rosin-Rammler feed and grid	the feed masses sum to one and the coarsest edge passes 100%
Breakage function	the matrix is strictly lower triangular and every column conserves mass
Classification	$C(d)$ is monotone, 0 below K_1 and 1 above K_2; a wider throw coarsens the product; the throw is neutral at the design throw
Whiten solve	mass closure and a product finer than the feed; a tighter setting gives a finer product
Capacity	the throughput is unimodal in speed
Regimes and nip angle	pass-through and invalid flags at a setting at or above the feed's F_{80} and top size
Bond power and current-based form	positive and finite at every valid point; the held-out power result reported as computed
Morrell's energy	equation B4 as printed, the units kWh/t and kW, the rule on S_c; the held-out results reported as computed, the negative M_{ic} flagged
HP500 calibration	the calibrated engine at each survey equals the backbone document; the surveys are physically plausible
Setting-only line	reproduces 12.78% (power) and 13.67% (throughput); a planted survey moves every fold but its own
CrusherCal	the label-shuffle control does not beat the backbone; the backbone beats the constant mean on throughput; the optimism gap is exposed; coverage reported
Envelope check	an implausible point is invalid; a calibrated point outside the support is flagged as extrapolating
Surrogate	R^2 of every output from the shipped graph; P_{80} monotone in the setting; the engine answers, labelled, with an invalid bundle
Autoencoder	false-alarm and detection rates beside those of the envelope check; the envelope check answers, labelled, with an invalid bundle
Drift projection	both engines return the same setting on 45 rows to 10^{-6} mm; P_{80} at the setting is the target; no answer is never a guess
CrusherBayes	the adoption rule applied as written; folds refit without the held-out survey; a short chain recorded as not converged; the document byte-identical

7.6 Acceptance criteria

Table 8 states, for each method, the criterion that a test checks and that would fail the build. They are criteria of correctness and of reporting, not of accuracy against the plant: a method that predicts the surveys poorly passes when its result is computed as stated and reported as it is, and its poor accuracy is a result (Section 8). The distinction matters for the power results: the criterion of the Bond power and of Morrell's energy is that their held-out numbers are computed and reported as they come, so the build passes with a held-out power error of 25.94% for the backbone and 38.08% for Morrell through the origin. Each criterion names its test in the product's design document, and a requirement without a test that exists fails a check of the repository; the criteria below are therefore the ones the committed build satisfies, not intentions.

8 Results

8.1 The engine across the circuit cases

Figure 6 and Table 3 give the coupled engine over the circuit. The engine reproduces the size-reduction ladder of a crushing circuit: the gyratory case G01 gives a coarse product (P_{80} = 153.6 mm) at a high throughput (4629 t/h) with a reduction ratio of 1.17; the jaw J01 an intermediate product (126.4 mm at 923 t/h); the secondary cone S01 27.7 mm at 643 t/h and a reduction ratio of 4.55; the tertiary cone T01 7.1 mm at 240 t/h; and the short-head cone H01 4.6 mm at 216 t/h, the finest product of the set. Within the secondary class the setting and the speed move the product as expected: S03, run past the speed optimum at 470 rpm, gives 856 t/h on the falling side of its hump (Figure 4(a)), and S06, at a setting of 12 mm, the finest secondary product (9.1 mm) with a reduction ratio of 9.80, above the typical ceiling of one stage and flagged as such. The setting sweep of S01 (Figure 4(b)) shows the trade-off a process engineer reads: between 16 and 48 mm, P_{80} rises from about 14 to 42 mm and the throughput from about 420 to 860 t/h, both close to linear, because the capacity is linear in the setting at a fixed throw and speed and the classification window moves with the setting. The three controls behave as stated: CP1 is pass-through with a reduction ratio of 1.12, CI1 is invalid with a reduction ratio of one and zero power, and CK1 is a valid choke point. Every case lies inside its declared expected ranges, otherwise the bake would have stopped. What this shows is that the coupled models behave as the cited models are known to behave; it is not a validation against a plant, which only the HP500 lane can give.

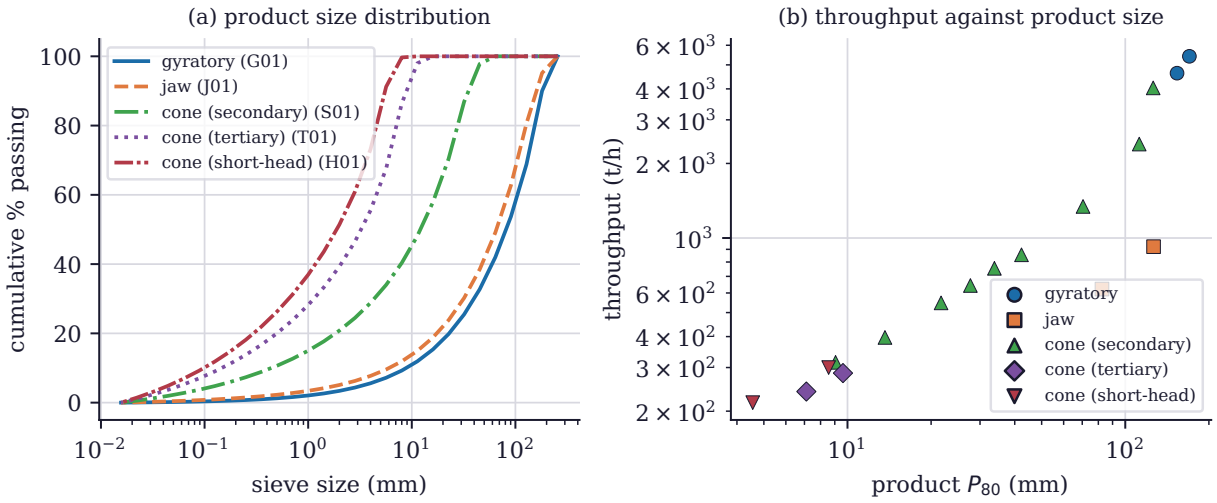


Figure 6: The coupled engine over the circuit cases, from the committed traces: (a) product size distributions of one gyratory (G01), jaw (J01), secondary cone (S01), tertiary cone (T01) and short-head cone (H01) case; (b) product P_{80} against throughput for the seventeen circuit cases, by machine type.

Table 9: Strict leave-one-survey-out scores of every predictor on the same ten HP500 surveys: MAPE in percent, RMSE in the unit of the target, held-out R^2, coverage of the 80% interval and its mean width in the unit of the target. Residual-band predictors carry the nested band of their own held-out residuals; CrusherBayes carries its posterior predictive interval. A dash marks a score that is not defined. Generated from `loo.json` and `crusherbayes.json`.

	Throughput (t/h)					Power (kW)				
Method	MAPE	RMSE	R^2	Cov.	Width	MAPE	RMSE	R^2	Cov.	Width
Constant mean	18.29	258.5	−0.235	–	–	24.48	68.5	−0.235	–	–
Label-shuffle control	25.04	322.8	−0.925	–	–	40.19	110.5	−2.214	–	–
Calibrated backbone	12.09	176.7	0.423	0.80	473.8	25.94	78.5	−0.622	0.90	209.8
Backbone with bounded residual	13.45	189.5	0.337	–	–	25.37	92.1	−1.231	–	–
Free-form fit on CSS and f_{80}	12.18	174.4	0.438	0.80	468.5	14.03	48.7	0.377	0.80	129.8
Setting-only line	13.67	177.8	0.416	0.80	478.8	12.78	44.6	0.478	0.80	119.6
Morrell, through the origin	–	–	–	–	–	38.08	111.7	−2.281	–	–
Morrell with a no-load term	–	–	–	–	–	15.21	46.5	0.430	–	–
CrusherBayes (posterior predictive)	12.00	168.6	–	0.80	439.1	14.31	44.7	–	0.90	117.7

8.2 Benchmark on the ten HP500 surveys

Table 9 gives every predictor on the same ten surveys under strict leave-one-survey-out, and Figure 7 the held-out predictions against the measured values. On throughput the predictors that use the setting and the capacity physics sit between 12.00% and 13.67%, within the noise of ten surveys of each other, and all beat the constant mean (18.29%); the label-shuffle control (25.04%) is worse than the backbone, as it must be. The calibrated backbone, with one fitted scalar and the engine's shape, matches the two-coefficient free-form fit (12.09% against 12.18%), and its in-sample error is 11.55%, so its held-out number is not an artifact of the fit. The bounded residual does not improve on the backbone held out (13.45%), the null result stated as the expectation before the run.

On power the picture is different. The calibrated backbone, whose power is the Bond draw refit in the published form, is worse than the constant mean (25.94% against 24.48%, held-out $R^2 = -0.62$), because the Bond draw is nearly flat while the measured power is not (Section 6.7). The setting-only line is the best power predictor (12.78%, $R^2 = 0.48$), followed by the free-form fit (14.03%), CrusherBayes (14.31%) and Morrell with a no-load term (15.21%). CrusherBayes is 1.53 points behind the line, within the noise of $n = 10$ and not a gain; Morrell through the origin, at 38.08%, is the worst size-aware model. The mean log predictive densities (Table 10) rank the four predictive distributions the same way: on power the line (−5.341) and CrusherBayes (−5.346) are tied and the backbone band is the worst (−5.847); on throughput the free-form fit (−6.647) and CrusherBayes (−6.678) are ahead of the line (−6.709) and the backbone (−6.896), by differences that one survey can reverse.

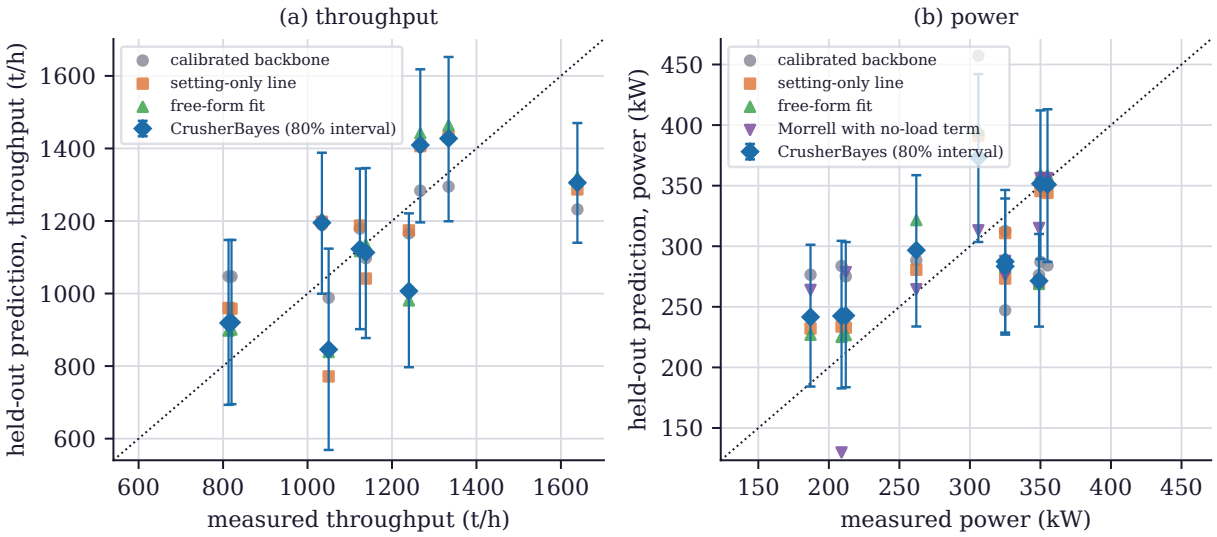


Figure 7: Held-out predictions against the measured values on the ten HP500 surveys, for (a) throughput and (b) power: the calibrated backbone, the free-form fit, the setting-only line and, for power, Morrell with a no-load term, and the CrusherBayes predictive mean with its 80% interval. The dotted line is parity.

Table 10: Mean log predictive density of the held-out surveys for the four predictors that give a distribution: three residual-band predictors with their nested Gaussian band and CrusherBayes with its posterior predictive. Higher is better. Generated from `crusherbayes.json`.

Method	Throughput	Power
Setting-only line	−6.709	−5.341
Calibrated backbone	−6.896	−5.847
Free-form fit	−6.647	−5.399
CrusherBayes	−6.678	−5.346

Table 11: MAPE in percent of every predictor with the held-out survey outside its fit (strict) and inside it (leaky), and their difference in points, on the same ten surveys and folds. Generated from `loo.json`.

	Throughput MAPE (%)			Power MAPE (%)		
Method	Strict	Leaky	Gap	Strict	Leaky	Gap
Constant mean	18.29	16.46	1.83	24.48	22.03	2.45
Label-shuffle control	25.04	21.18	3.86	40.19	15.50	24.69
Calibrated backbone	12.09	10.95	1.14	25.94	19.28	6.66
Backbone with bounded residual	13.45	8.51	4.94	25.37	12.00	13.37
Free-form fit on CSS and f_{80}	12.18	8.24	3.94	14.03	9.46	4.57
Setting-only line	13.67	10.60	3.07	12.78	10.03	2.75
Morrell, through the origin	–	–	–	38.08	33.81	4.27
Morrell with a no-load term	–	–	–	15.21	11.50	3.71

8.3 Optimism of an evaluation that leaks

Table 11 gives the same predictors when the held-out survey is allowed into its own fit. The fits that adapt most to the data gain most: the bounded residual falls from 13.45% to 8.51% on throughput and from 25.37% to 12.00% on power, a gap of 13.37 points, and the free-form fit from 14.03% to 9.46% on power. The calibrated backbone, with one scalar for throughput, gains 1.14 points. The label-shuffle control on power falls from 40.19% to 15.50%, a gap of 24.69 points: a fit of permuted labels looks almost as good as the line when the survey is in the fit, which is how much of the apparent skill of a leaky evaluation is the fit absorbing the survey. The setting-only line has one of the smallest gaps (2.75 points on power), as a two-coefficient fit on one feature should. An evaluation of a calibrated crusher model on the surveys it was fitted to would therefore have ranked the bounded residual first on both targets; held out it ranks behind the backbone on throughput and ties the backbone on power.

Table 12: The nested 80% band of the calibrated backbone per held-out survey: the held-out residual (measured minus prediction), the standard deviation σ_{-i} of the other nine residuals, the half-width $1.2816\,\sigma_{-i}$ and whether the survey lies inside. Generated from `loo.json`.

	Throughput (t/h)				Power (kW)			
Survey	Residual	σ_{-i}	Half-width	Inside	Residual	σ_{-i}	Half-width	Inside
1	407.1	123.2	157.9	no	−74.7	83.6	107.2	yes
2	−16.8	197.3	252.9	yes	−63.1	84.7	108.6	yes
3	74.2	195.1	250.0	yes	−26.3	86.8	111.2	yes
4	−155.3	189.4	242.8	yes	72.7	81.2	104.1	yes
5	−55.1	196.6	251.9	yes	77.9	80.4	103.1	yes
6	−234.5	178.2	228.4	no	70.8	81.5	104.4	yes
7	61.6	195.7	250.9	yes	−151.3	69.4	89.0	no
8	40.5	196.6	252.0	yes	12.5	86.5	110.9	yes
9	−226.9	179.5	230.0	yes	62.9	82.5	105.7	yes
10	39.2	196.6	252.0	yes	−89.5	81.9	104.9	yes

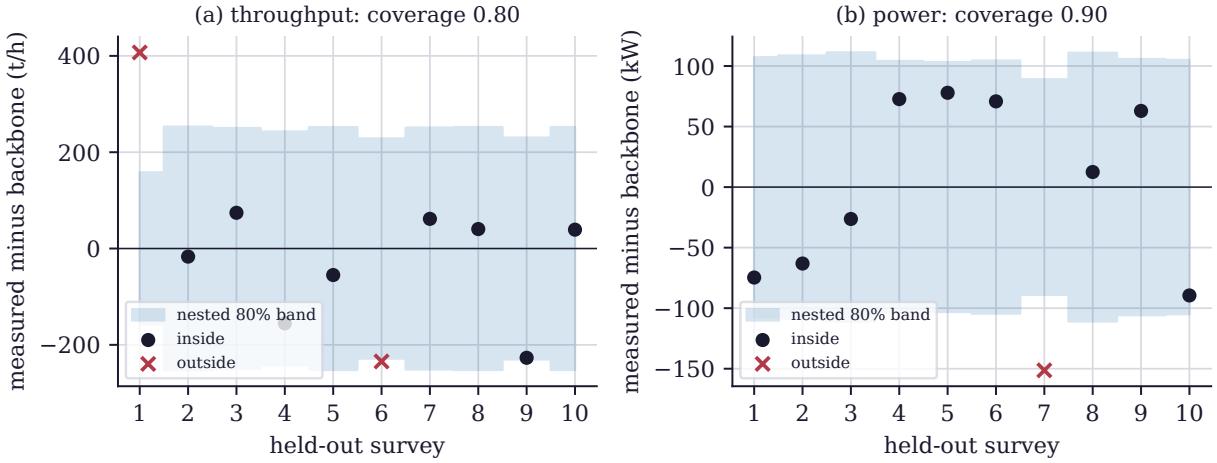


Figure 8: The held-out residual of the calibrated backbone per survey against its nested 80% band, for (a) throughput and (b) power, with the surveys outside the band marked by crosses; the nested coverage of each target is in the panel title.

8.4 The predictive band of the calibrated backbone

Table 12 and Figure 8 give the band survey by survey. With σ from all ten held-out residuals, the band's half-width is 238.5 t/h (20.8% of the mean throughput) and 105.1 kW (36.5% of the mean power), and its in-sample coverage is 0.90 on both targets. Nested, with σ_{-i} from the other nine residuals, the coverage is 0.80 on throughput and 0.90 on power. Two surveys fall outside the throughput band: survey 1, whose measured feed rate of 1639 t/h is the largest of the set and 407 t/h above the backbone, and survey 6, 235 t/h below it; survey 9 lies inside by 3 t/h. On power one survey falls outside, survey 7 (151 kW below the backbone), whose feed f_{80} of 140 mm is the coarsest of the set. The in-sample coverage of 0.90 on throughput flatters the band; the nested 0.80 is the figure a reader can rely on, and with ten surveys either moves by 0.1 per survey. The relative half-width on throughput, 20.81%, is the lower bound the studio shows as a band on the synthetic secondary-cone cases, read at run time from the document: an uncalibrated case cannot be more accurate than the same model family once calibrated.

8.5 Morrell's size-specific energy

Table 13 and Figure 9 give the two Morrell variants fold by fold. Without a no-load term the fit is one positive scalar, M_{ic} between 3.70 and 4.21 kWh/t across the folds, and the held-out error is 38.08% ($R^2 = -2.28$). With a no-load term the error falls to 15.21%, but the fit is not Morrell's physics: in every fold the fitted M_{ic} is negative, from −6.29 to −4.16 kWh/t, with a no-load power of 591 to 727 kW, about twice the largest measured power of 355 kW. The measured power falls as the throughput rises (correlation −0.85), and the design column g_i rises with the throughput, so the least-squares line through it has a negative slope and a large intercept. The variant is a descriptive line on that column and not an estimate of M_{ic} or of a

Table 13: The Morrell fits per fold, each on the nine training surveys: M_{ic} of the variant through the origin, M_{ic} and no-load power of the variant with a no-load term, and whether that variant's M_{ic} is positive. Generated from `loo.json`.

Held-out survey	M_{ic}, origin (kWh/t)	M_{ic}, no-load (kWh/t)	P_{noload} (kW)	$M_{ic} > 0$ (no-load)
1	4.213	−6.295	726.7	no
2	3.986	−4.458	611.0	no
3	3.915	−4.653	618.4	no
4	3.726	−4.465	601.4	no
5	3.792	−4.727	618.5	no
6	3.703	−4.680	620.2	no
7	3.790	−4.697	622.0	no
8	3.781	−4.639	613.5	no
9	3.709	−4.743	625.2	no
10	4.065	−4.158	590.7	no

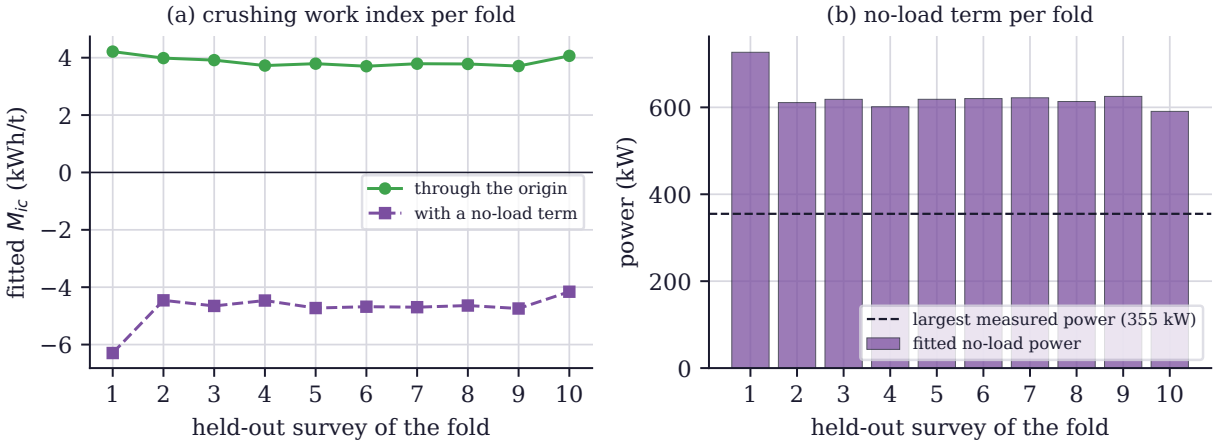


Figure 9: The two Morrell variants fold by fold: (a) the fitted crushing work index M_{ic} through the origin and with a no-load term; (b) the fitted no-load power against the largest measured motor power of the ten surveys.

no-load power. Both variants are worse than the setting-only line; the difference of 2.4 points for the variant with a no-load term is within the noise, and the 25 points of the variant without it are not. On these ten surveys the size-specific energy adds no predictive skill. An SMC Test of the ore and a no-load reading of the machine would replace the two fitted scalars by measurements and turn the method into a prediction.

8.6 CrusherBayes: held-out scores, adoption and what the surveys identify

On throughput CrusherBayes has the lowest held-out MAPE (12.00%, RMSE 168.6 t/h) and the narrowest interval (439.1 t/h against 473.8 t/h for the backbone band, 7% narrower) at coverage 0.80; on power its MAPE is 14.31%, its interval 117.7 kW wide, 44% narrower than the backbone band (209.8 kW) at the same coverage of 0.90, and about the width of the line's band (119.6 kW at coverage 0.80). Table 14 evaluates the adoption rule term by term. Condition (a) holds: the power MAPE is 1.53 points above the line's, not three below, and the throughput MAPE only 0.09 points below the backbone's. Condition (b) does not hold: both coverages, 0.80 and 0.90, lie inside 0.7 to 0.9, and both intervals are narrower than the backbone's band. The rule drops the method only when both conditions hold, so it keeps CrusherBayes. That is the outcome of the rule as written and not a claim of skill over the line: the method gives a calibrated and narrower interval, and does not predict the power better than one feature.

Figure 10 and Table 15 give the contraction of the posterior fitted to all ten surveys. The capacity base is identified (contraction 0.454, posterior mean 646.3 t/h against the nominal 647), and so are the two noise scales (0.301 and 0.332) and the four discrepancy slopes on the setting and on f_{80} (0.340 to 0.381). The posterior noise is 145.2 t/h and 38.8 kW, much larger than the prior medians of 60 t/h and 15 kW. The discrepancy slope on the setting is large on both targets, +100.8 t/h and −57.1 kW per standard deviation of the setting: the engine's dependence on the setting differs from the measured one, on power by about 57 kW per standard deviation, of the same order as the slope of the setting-only line (−8.49 kW/mm) times the standard deviation of the setting over the ten surveys (5.94 mm), about −50 kW. The work index, ζ

Table 14: The adoption rule of CrusherBayes evaluated term by term on the held-out scores: the value of CrusherBayes, the reference predictor's value, the difference, the condition stated before the run and whether CrusherBayes meets it. Generated from `crusherbayes.json`.

Term of the adoption rule	CrusherBayes	Reference	Difference	Condition	Met
Power MAPE against the setting-only line (points)	14.31	12.78	−1.53	gain ≥ 3	no
Throughput MAPE against the backbone (points)	12.00	12.09	0.09	gain ≥ 3	no
Throughput coverage of the 80% interval	0.80	–	–	0.7 to 0.9	yes
Power coverage of the 80% interval	0.90	–	–	0.7 to 0.9	yes
Throughput mean width (t/h) against the backbone band	439.1	473.8	34.6	not wider	yes
Power mean width (kW) against the backbone band	117.7	209.8	92.0	not wider	yes

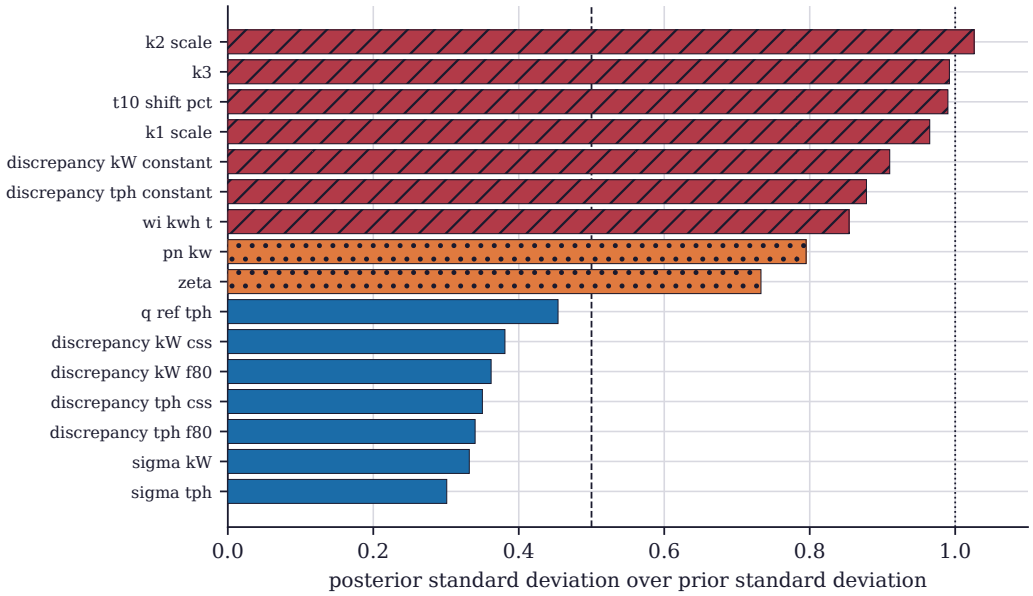


Figure 10: Posterior over prior standard deviation of each CrusherBayes parameter for the posterior fitted to all ten surveys. Below 0.5 (dashed) the surveys identify the parameter; near one (dotted) it stays at its prior. Plain bars are identified, dotted bars weakly identified, hatched bars not identified.

and P_n contract weakly (0.854, 0.733 and 0.795), and ζ and W_i enter the power only as a product. The two classification multipliers, K_3, the t_{10} shift and the two discrepancy constants stay at their prior (0.878 to 1.026). The scalars that set the product size are not identified because the two measured targets carry the product size only through the Bond term of the power, and the power follows the setting, not the product size. Ten surveys without a measured product size cannot calibrate the population balance.

Table 16 gives the convergence of the eleven posteriors. All are converged under the criterion of fifty autocorrelation times: the largest autocorrelation time is 48.0 to 84.8 steps and the 6000 post-burn steps are 70.7 to 125.0 times it, with acceptance fractions between 0.171 and 0.204. The posterior of the fold that holds out survey 4 mixes slowest (84.8 steps); it still passes with a margin of 1.4 over the criterion.

8.7 The learned models

The surrogate reproduces every output of the engine on the independent held-out draw of 2196 points (Figure 11, Table 17): R^2 from 0.9792 (power) to 0.9984 (throughput) and MAPE from 2.73% (passing 32 mm) to 9.21% (P_{20}). The error grows toward the fine tail of the product (P_{80} 4.06%, P_{50} 5.73%, P_{20} 9.21%), where the passing fractions are small and a relative error is large for a small absolute error. P_{80} is monotone in the setting on all fifteen test curves. The export is faithful: the largest absolute difference of the standardised outputs between the trained torch module and the float32 graph is 1.7×10^{-6}. The int8 graph's largest difference from the float32 one is 0.235 in standardised units, yet its MAPE stays within 0.51 points of the float32 MAPE on every output, and on power it is 0.035 points lower.

The autoencoder raises an alarm on 1.00% of the held-out points, as its threshold at the 99th percentile of the training scores implies, and on 27.96% of the planted off-manifold points; the envelope check raises none on the held-out points and flags 13.89% of the planted ones (Table 18). The score therefore doubles

Table 15: Prior and posterior of every CrusherBayes parameter for the fit to all ten surveys, on the natural scale: prior centre and standard deviation, posterior mean and standard deviation, and their ratio (the contraction). Discrepancy rows are the exact conditional given the engine scalars. Generated from `crusherbayes.json`.

Parameter	Unit	Prior centre	Prior sd	Posterior mean	Posterior sd	Contraction
Q_T, capacity base	t/h	647.00	167.01	646.35	75.74	0.454
W_i, blend work index	kWh/t	9.00	1.52	8.39	1.29	0.854
ζ, power slope	–	1.30	0.30	1.14	0.22	0.733
P_n, no-load power	kW	110.00	47.85	86.35	38.04	0.795
s_1, multiplier of K_1	–	1.00	0.15	1.07	0.15	0.965
s_2, multiplier of K_2	–	1.00	0.15	1.05	0.16	1.026
K_3, classification exponent	–	2.30	0.49	2.23	0.49	0.992
Δt_{10}, shift of the constant 64	points	0.00	3.99	−0.11	3.95	0.990
σ_Q, throughput noise	t/h	60.00	129.67	145.23	39.04	0.301
σ_P, power noise	kW	15.00	32.42	38.81	10.75	0.332
d_{Q0}, throughput constant	t/h	0.00	140.00	53.36	122.88	0.878
d_{Q1}, throughput slope on CSS	t/h	0.00	140.00	100.84	49.00	0.350
d_{Q2}, throughput slope on f_{80}	t/h	0.00	140.00	56.57	47.63	0.340
d_{P0}, power constant	kW	0.00	36.00	−12.57	32.77	0.910
d_{P1}, power slope on CSS	kW	0.00	36.00	−57.07	13.70	0.381
d_{P2}, power slope on f_{80}	kW	0.00	36.00	−5.16	13.03	0.362

Table 16: Convergence of the eleven CrusherBayes posteriors (ten folds and the fit to all ten surveys): largest integrated autocorrelation time, post-burn steps, their ratio, acceptance fraction, retained samples and whether the criterion of fifty autocorrelation times holds. Generated from `crusherbayes.json`.

Posterior (held-out survey)	$\hat{\tau}_{\max}$ (steps)	Post-burn steps	Steps/$\hat{\tau}_{\max}$	Acceptance	Retained samples	Converged
1	52.6	6000	114.1	0.204	12000	yes
2	52.7	6000	113.8	0.201	12000	yes
3	54.7	6000	109.7	0.203	12000	yes
4	84.8	6000	70.7	0.171	12000	yes
5	52.7	6000	113.9	0.203	12000	yes
6	48.0	6000	125.0	0.204	12000	yes
7	55.1	6000	108.9	0.200	12000	yes
8	48.9	6000	122.7	0.204	12000	yes
9	52.6	6000	114.1	0.203	12000	yes
10	50.7	6000	118.3	0.202	12000	yes
all ten	48.6	6000	123.5	0.204	12000	yes



Figure 11: The surrogate against the engine per output on the independent held-out draw of 2196 points: R^2 (bars, left axis) and MAPE in percent (squares, right axis), computed through onnxruntime on the shipped float32 graph. Read from `model-metrics.json`.

the detection of the rule-based check, at a false-alarm rate of one percent, and still misses most planted points: it is a complement to the check, not a replacement. Neither learned model is held to the HP500 surveys; both are compared with the engine.

Table 17: Surrogate accuracy per output against the engine on the held-out draw: R^2 and MAPE of the float32 graph, MAPE of the int8 graph and the difference in points. Generated from `model-metrics.json`.

Output	R^2 (float32)	MAPE float32 (%)	MAPE int8 (%)	Difference (points)
P_{80}	0.9970	4.063	4.568	0.505
P_{50}	0.9947	5.732	6.125	0.393
P_{20}	0.9886	9.213	9.480	0.267
passing 1 mm	0.9868	8.115	8.329	0.214
passing 4 mm	0.9884	7.700	7.909	0.209
passing 8 mm	0.9930	5.296	5.644	0.348
passing 16 mm	0.9936	5.293	5.538	0.245
passing 32 mm	0.9971	2.730	2.869	0.139
throughput	0.9984	5.017	5.434	0.417
power	0.9792	6.466	6.431	−0.035

Table 18: The autoencoder's extrapolation score against the envelope check on the same 2196 held-out points and 2196 planted off-manifold points: false-alarm and detection rates, and the threshold and mean training score of the score. Generated from `model-metrics.json` and `psd-ae.json`.

Quantity	Autoencoder	Envelope check
False-alarm rate on the held-out draw	0.0100	0.0000
Detection rate on planted off-manifold points	0.2796	0.1389
Points scored (held-out, planted)	2196, 2196	2196, 2196
Alarm threshold (99th percentile of training scores)	0.24454	–
Mean training reconstruction score	0.05181	–

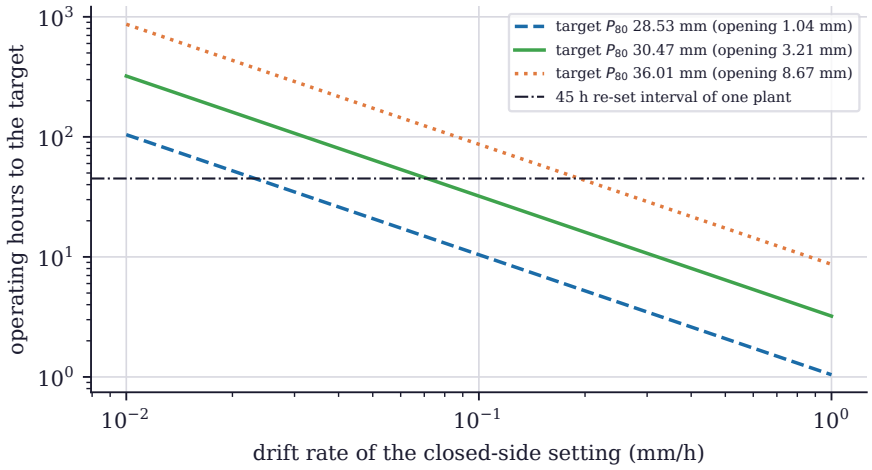


Figure 12: Operating hours until the product P_{80} of case S01 reaches a target, against the drift rate of the closed-side setting, for targets 3, 10 and 30% above the current P_{80} from the shared reference table, with the 45 h re-set interval of one plant as reference.

8.8 The drift projection

Figure 12 gives the drift projection of case S01 for the three targets of the shared reference table. A target 3% above the current P_{80} (28.53 mm) allows the setting to open by 1.04 mm, a target 10% above (30.47 mm) by 3.21 mm, and a target 30% above (36.01 mm) by 8.67 mm. The hours to the target are inversely proportional to the drift rate: at 0.1 mm/h the three openings last 10.4, 32.1 and 86.7 h. The 45 h re-set interval of one plant is reached by the 10% target at a drift of 0.071 mm/h; at a faster drift that target is left before the re-set. The figure is a what-if on the engine: the product asserts no drift rate, and the reader supplies the one measured at the plant, for example from the movement of the adjustment ring between two liner changes.

9 Validation

This section states how the results of Section 8 are known to be computed correctly: by closed forms independent of the engine's numerics, by two implementations held equal, by a stochastic re-derivation of the balance, by a units check on every prediction, by tests that plant defects, and by reconciliation with the published calibration. None of these is a model judging a model.

9.1 Closed forms and conservation

Three oracles in `pipeline/oracles.py` compute quantities by closed form, independently of the engine's numerics, and the tests compare them with the engine. The Rosin-Rammler passing at the characteristic size must equal $1 - 1/e$ (Equation (4) at $x = x_{63}$); the Bond energy of a reduction from F_{80} to P_{80} must equal its closed form (Equation (20)) evaluated directly; and the mass of every baked product must sum to the feed's within the solver's closure. The closure itself is reported in every trace before the product is renormalised: at S01 it is 3.3×10^{-16}, a few units of round-off for a sum of 28 terms, and the conditioning proxy is 1.517, so the system is far from the threshold of 10^6 at which a point is flagged. The breakage matrix of every case is strictly lower triangular with columns summing to one, which, by the derivation of Section 6.4, is what makes the balance conserve mass. The trends that the engine must reproduce are tested as inequalities: a tighter setting gives a finer product, a wider throw a coarser product with a higher capacity, the capacity is unimodal in speed, and the classification is monotone.

9.2 Two engines held equal

The Python and TypeScript engines are held to a shared reference file of 124 operating points, which covers the five machine classes, 27 points with the calibrated switch on, and 107 choke, 13 pass-through and 4 invalid points. For every point every result field and every feature vector of the two learned models must agree within 10^{-9} relative; the largest relative difference measured over the file is 2.3×10^{-12}, about 10^4 units of round-off, which is what two implementations of the same operations in the same order produce. A second test re-runs every baked case in TypeScript and compares it with the committed trace to the six significant digits the trace is written with, so a change to either engine that is not replicated in the other fails both a unit test and a build. The drift projection has its own shared table of 45 rows, reproduced on both sides to 10^{-6} mm, and the model encodings of the browser are checked against the same reference, float32 rounding included.

9.3 The balance re-derived by a particle-replacement walk

The Whiten product is the expectation of a simple stochastic process: parcels of feed enter the chamber with the feed proportions; in each pass a class-j parcel is nipped with probability c_j; the parcels not nipped leave as product, and the nipped ones of class j are redistributed over the finer classes by the column $\mathrm{B}_{\cdot j}$ and enter the next pass, until none is inside. This is the particle-replacement rule of discrete-element crusher models reduced to counts of parcels per class. If $\mathbf{n}^{(m)}$ is the expected number of parcels inside at pass m per unit of feed, then $\mathbf{n}^{(0)} = \mathbf{f}$ and $\mathbf{n}^{(m+1)} = \mathrm{BC}\,\mathbf{n}^{(m)}$, so the expected product and the expected number of nips per parcel are

$$\mathbb{E}[\mathbf{p}_{\mathrm{walk}}] = (\mathrm{I} - \mathrm{C}) \sum_{m \geq 0} (\mathrm{BC})^m \, \mathbf{f} = (\mathrm{I} - \mathrm{C})(\mathrm{I} - \mathrm{BC})^{-1}\mathbf{f}, \qquad \mathbb{E}[\nu_{\mathrm{walk}}] = \mathbf{1}^{\top}\mathrm{C}\,(\mathrm{I} - \mathrm{BC})^{-1}\mathbf{f}, \tag{45}$$

the Neumann series converging because BC is nilpotent (strictly lower triangular). A walk of N parcels therefore estimates the Whiten product with a Monte Carlo error that should fall as $N^{-1/2}$. The product's function `replacement_walk` implements this walk; the figure script of the report runs it on the chamber of case S01 (assembled by `assemble_chamber`) for 100 to 100,000 parcels with five seeds each, and compares it with `evaluate`. These numbers are computed by the committed script `figures/make_figs.py` calling the product's code, and are not stored in a product artifact. Figure 13(a) shows the largest difference per class between the walk and the Whiten product: 0.060 at 100 parcels and 0.0020 at 100,000, along a slope of $-1/2$. Figure 13(b) shows the mean number of nips per parcel converging to the value $\sum_i c_i x_i = 0.8858$ of Equation (14). The walk and the solve are two derivations of the same model, so their agreement validates the implementation of the solve, not the physics of the model.

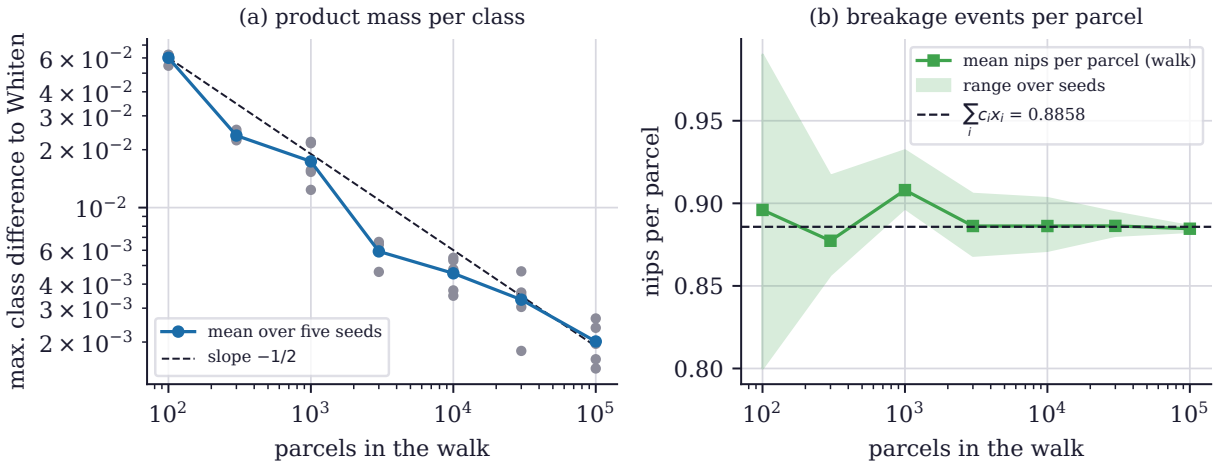


Figure 13: The particle-replacement walk on the chamber of case S01 against the Whiten solve, computed by the report's figure script with the product's engine code: (a) largest difference per size class between the walk's product and the Whiten product against the number of parcels (five seeds, grey; mean, blue; slope $-1/2$, dashed); (b) mean nips per parcel against the expectation $\sum_i c_i x_i$.

9.4 Units of every prediction

Every prediction of a case must live in the units of its observable. For an observable with range $[L, H]$, the prediction $\hat{y}$ must be finite and satisfy

$$L - (H - L) \;\le\; \hat{y} \;\le\; H + (H - L), \tag{46}$$

a range widened by one full range on each side. For the three measured observables of a real case (throughput, power and f_{80}) the range is that of the ten surveys; for every other observable, and for every observable of a circuit case, it is the physical domain the engine declares: sizes from 0 to the 256 mm top of the grid, a reduction ratio from one to the ratio of the top to the finest edge, a throughput from 0 to ten times the reference capacity of the machine class, a power from 0 to 3000 kW, and a specific energy from 0 to 20 kWh/t. The check runs inside the bake and again over the shipped artifacts in continuous integration, with the standard library only. It is a units check, not an accuracy check: a poor prediction inside the widened range passes, which is intended; and it cannot catch a swap between two observables whose ranges overlap numerically (a power and a throughput on the HP500 range), which the per-case expected ranges and a cross-case test catch instead. A separate script checks that the engine's constants carry their units.

9.5 Tests that plant defects

Several results are guarded by tests that plant the defect they guard against. An absurd target planted in one survey must move every other fold's prediction and not its own, for every fitted predictor including the line and CrusherBayes, which proves that no fold sees its held-out survey. The label-shuffle control must not beat the backbone. Swapping a model slot to its baseline must keep the contract, so that the web never shows a learned number when the bundle is invalid. A short chain must be recorded as not converged. The CrusherBayes document must be byte-identical whatever the number of worker processes, and the integrated-out discrepancy of Equation (41) is checked against a Monte Carlo of the model in Equation (39). Every committed document validates against its schema on both sides of its contract, the readiness document must be current with the catalog, a planted document that breaks a cross-field rule must be refused, and a guard of the test suite fails any test that would rewrite the committed artifacts.

9.6 Reconciliation with the published calibration

The calibrated lane is reconciled with the published calibration in two ways. The engine with the published constants reproduces the measured feed rates of the ten surveys with an in-sample MAPE of 11.55%, and its capacity base refit per fold stays close to the published value: the posterior mean of the capacity base fitted to all ten surveys is 646.3 t/h against the published 647 t/h. The published power constants do not reconcile with the Bond draw (47.58% in sample), which the source itself explains, because they belong to

a different net power. The paper's own error figures are printed as figures and the publisher page could not be opened, so no comparison of the held-out errors with an in-sample error band of the paper is made; the held-out error is set beside the in-sample error of the same backbone computed by the product. A re-transcription of the published survey table that disagreed with the committed file would remove the real lane from a build until the table is corrected.

10 Discussion

10.1 What the results mean for the setting decision

The question a process engineer asks of a calibrated crusher model is how well it predicts a survey it was not fitted to. On the ten HP500 surveys the answer has two parts. For throughput, the calibrated engine is a useful predictor: its held-out error of 12.09% is close to its in-sample error of 11.55%, it beats the constant mean by six points, and a nested 80% band of about ±239 t/h covers the measured feed rate at the stated rate. The capacity physics, with one refit scalar, does as well as a free-form regression with two coefficients, which is what a physical shape should buy on ten points. For power, the calibrated engine is not a useful predictor: the Bond draw it computes hardly varies across the surveys while the measured power spans almost a factor of two, and the published power constants belong to a different net power. A plant that wants to predict the power of this machine at a new setting is better served, on these data, by a line on the setting, which reaches 12.78% held out, than by any of the size-aware energy laws the product implements. The recommendation that follows is specific: use the calibrated engine for throughput and for the product size trends it was built for, and read power from the measured relation with the setting until the plant measures the quantities that the energy laws need (the crushing work index from an SMC Test, the no-load power, and the product size distribution of each survey).

10.2 Why each method behaves as it does

The setting-only line wins on power because the measured power on these surveys is, to first order, a function of the setting ($r = -0.82$), and the line spends its two coefficients on exactly that relation. The free-form fit spends a third coefficient on f_{80}, which adds noise on ten points and costs it 1.25 points held out. The Bond draw is flat because, as the setting tightens, the finer product raises the specific energy while the lower throughput lowers the tonnage, and the two nearly cancel in kilowatts; the measured power instead rises as the setting tightens, which a model of the machine's mechanics might explain, for example an energy computed as the product of the liner pressure and the displacement over the chamber [43], which the product does not implement. Morrell's energy inherits the same design column, the specific energy times the measured throughput, so its fit has the same problem as Bond's, and the variant with a no-load term recovers accuracy only by fitting a negative work index. The bounded residual fails held out because it is a two-feature correction fitted on nine points to residuals that are themselves noisy: its leaky error is the lowest of all predictors and its strict error is above the backbone's, the signature of overfitting. CrusherBayes calibrates its interval because its noise and discrepancy scales are learned from the residuals of the engine (the posterior noise is about two and a half times the prior median on both targets), and narrows it relative to the backbone band on power because its discrepancy slope on the setting absorbs the relation that the Bond draw misses, about 57 kW per standard deviation of the setting. That slope plays the role of the setting-only line inside the Bayesian model, which is consistent with CrusherBayes approaching but not passing the line on power.

10.3 Where each method fails

Each method has a failure that the results expose. The Bond draw and the published power form fail on the measured power (Table 6). Morrell through the origin fails by 25 points against the line, and with a no-load term it fits a physically impossible work index in every fold (Table 13). The bounded residual fails to beat the backbone held out, and the label-shuffle control shows how a leaky evaluation would have hidden that (Table 11). CrusherBayes does not beat the setting-only line on power, 14.31% against 12.78%, and does not beat the backbone on throughput beyond noise (0.09 points); its posterior leaves the classification and breakage scalars at their prior, so it cannot say anything about the product size distribution. The calibrated backbone misses two surveys of ten on throughput, survey 1 by 407 t/h, so a single survey can sit well outside a model whose average error is twelve percent. The learned surrogate is worst on the fine

tail of the product (9.21% on P_{20}), and the autoencoder misses 72% of the planted off-manifold points. The circuit cases have no measured data, so their absolute numbers carry no validation at all. The survey HP500-03 is flagged pass-through because its feed is barely coarser than its setting; a model reading that survey's product size is outside its regime.

10.4 What the product recommends and when

Three recommendations follow, each with the condition under which it holds. First, read the calibrated engine's throughput with its nested band (about ±21%) for a secondary HP500 on this ore inside the surveyed support of 38 to 55 mm and 47 to 140 mm; outside it the envelope check flags extrapolation and the band does not apply. Second, for power, prefer the measured relation with the setting, and use the CrusherBayes interval, which covers the measured power at the stated rate and is narrower than the backbone band, when an uncertainty is needed; do not read the engine's Bond power as a prediction of a survey. Third, to calibrate the population balance itself, which ten surveys without product sizes cannot do, measure the product size distribution of each survey: the contraction table shows that no other addition to the present data would identify the classification and breakage scalars. For the synthetic machine classes the product makes no recommendation beyond the trends; their constants are typical of their class and their band on the secondary cone is a lower bound.

11 Limitations

Sample size. The real data are ten surveys of one machine on one ore. A difference below about three MAPE points is within noise, and one survey moves a coverage by 0.1, so the coverages of 0.80 and 0.90 are coarse estimates, and the ranking of predictors within three points of each other is not stable. The surveys are treated as exchangeable; their order in time is not known to the product.

What is not measured. The product size distribution of the surveys is printed only as figures in the source, so the product size is never a target, and every statement about the engine's P_{80} on the survey lane is a model output. The throw and the speed of each survey are not in the table and are held at nominal values. The crushing work index and the no-load power of the machine are not published, so Morrell's method and the power form are fits.

The source calibration. The constants of the calibrated lane are carried from the product's code and earlier documentation; the publisher page of the source refused automated access in every documenting attempt, so their correspondence to the numbered tables of the paper, and the paper's own error figures, are not re-verified here. The priors of CrusherBayes have widths chosen by the author, because no published range of these fitted scalars for this ore was found.

Scope of the engine. The engine is a single-zone population balance with a reduced capacity model and an empirical power law. It has no chamber geometry, no liner wear, no feed segregation and no choke-level dynamics. The synthetic machines are not calibrated: their constants are illustrative of their class and reproduce trends. The three-dimensional chamber view is a kinematic view of the geometry and not a particle simulation. The learned models emulate the engine and not a plant, and the autoencoder detects distance from the engine's sampled envelope and not a mechanical fault. The drift projection models no wear and asserts no drift rate.

Discrete-element simulation. A discrete-element simulation with breakage, by particle replacement with the Tavares breakage model [47] in a separate package, is being built for validation on the laboratory jaw crusher of Barrios et al. [48]; it is not part of the methods or the results of this report. The open HP500 data could not validate it in any case, because the liner geometry of that machine is not public.

Prior work. The searches that preceded CrusherBayes found no Bayesian calibration with structural discrepancy of a cone-crusher population-balance model evaluated held out; they were not exhaustive (two citation indexes, Scopus and Web of Science, were not searched). Cone-crusher models that couple energy and breakage [44, 45] and machine-learning models of cone-crusher power [46] exist and were read only at the level of their metadata; the product implements neither.

Table 19: SHA-256 digests (first sixteen hexadecimal digits) of the committed artifacts every table and figure of the report is generated from, written by `make_tables.py` together with the full digests.

Artifact	SHA-256 (first 16 hex digits)
`data/examples/hp500-surveys.csv`	`ab9609a79ba64b4d...`
`data/examples/params.csv`	`d0419c5388d0b8ee...`
`data/derived/hp500/backbone.json`	`0d55b80a191ddf53...`
`data/derived/hp500/loo.json`	`672c4b21a0dc3703...`
`data/derived/hp500/crusherbayes.json`	`f284ec33a390dbaf...`
`data/derived/model-metrics.json`	`086ca0e63364e925...`
`data/derived/models/surrogate.onnx`	`233024c3fb93e24e...`
`data/derived/models/psd-ae.onnx`	`22f844b802f37432...`
`data/derived/manifests/index.json`	`5f75bb453a83360a...`

12 Reproducibility

Repository and release. The source code, the data, the committed artifacts and the sources of this report are in the public repository `https://github.com/fsantibanezleal/CAOS_ChancaDEM` (MIT licence for the code and the product's own data; CC BY 4.0 for the survey table and everything derived from it). Every number of the report is read from the artifacts of release 0.12.001; the studio runs at `https://chancadem.fasl-work.com`.

Commands. From the repository root, with the pipeline environment of `requirements-precompute.txt`:

- `python data-pipeline/run.py all` bakes the 27 cases, the three HP500 documents (the sampler of CrusherBayes runs in this step), the index and the readiness document; `python data-pipeline/run.py hp500` rebuilds only the three HP500 documents;
- `python data-pipeline/run.py models` draws the sweeps, trains and exports the two learned models and scores them held out (torch on the processor);
- `python manuscripts/comminution-studio/figures/make_tables.py` regenerates every table of the report from the committed artifacts, the catalog and the schemas, recomputes the engine parity figure, and writes the digests of `data/sha256.txt`;
- `python manuscripts/comminution-studio/figures/make_figs.py` regenerates the thirteen figures as vector PDF (matplotlib, from the notebook environment); two of them call the engine code (the chamber functions and the particle-replacement walk);
- `latexmk -pdf main.tex`, run in `manuscripts/comminution-studio/tex`, builds the report.

Versions. Python 3.13 on the workstation (3.12 in continuous integration), numpy 2.4.6, emcee 3.1.6, torch 2.12.1 on the processor, onnx 1.22.0, onnxruntime 1.27.0 and onnxruntime-web 1.27.0, matplotlib 3.11.2, Node 24. The committed artifacts record the engine release 0.12.001 and the seeds 42 (surveys), 12345 and 67890 (learned models).

Archived artifacts. Table 19 gives the SHA-256 digests of the artifacts the report reads; the full digests are in `manuscripts/comminution-studio/data/sha256.txt`, and the CrusherBayes document records the digest of its own inputs (`483d24c5...b0ce605a`).

Notebooks. `notebooks/en` and `notebooks/es` hold the same didactic notebooks, from getting started and the size distributions and the breakage and classification functions to the capacity and the power, the real surveys, the held-out validation, the learned models, Morrell's energy and CrusherBayes, on the product's own data and modules.

13 Conclusions

On ten industrial surveys of one cone crusher, a calibrated Whiten, Evertsson and Bond engine predicts the throughput of a survey it has not seen with a mean absolute percentage error of 12.09%, against 11.55% in sample and 18.29% for the constant mean, with a nested 80% band that covers eight surveys of ten. It does not predict the power: its Bond draw is flat across the surveys, and its held-out power error of 25.94% is worse than the constant mean. A line on the closed-side setting alone is the hardest baseline on power

(12.78%), and neither Morrell's size-specific energy with a fitted work index (38.08%, and 15.21% with a no-load term whose fitted index is negative in every fold) nor CrusherBayes (14.31%) passes it. CrusherBayes gives held-out intervals of the stated coverage that are 44% narrower than the backbone band on power and 7% on throughput, and its posterior shows that the ten surveys identify the capacity base, the noise and the dependence of the discrepancy on the setting, and cannot identify the classification and breakage scalars. A leaky evaluation would have ranked the bounded residual first; held out it is behind the backbone. The next steps are measurements, not models: the product size distribution of each survey, which the population balance needs to be calibrated; the crushing work index and the no-load power of the machine, which would turn Morrell's method into a prediction; and more surveys, which would resolve differences of a few points that ten cannot.

References

[1] Comisión Chilena del Cobre (Cochilco), *Consumo de energía en la minería del cobre al año 2024* (DEPP 17/2025). Santiago, 2025. https://www.cochilco.cl/web/download/991/2025/15151/consumo-de-energia-en-la-mineria-del-cobre-actualizacion-ano-2024.pdf

[2] Comisión Chilena del Cobre (Cochilco), *Proyección de consumo de la energía eléctrica de la minería del cobre chilena en el contexto de la transición energética 2025–2034* (DEPP 03/2026). Santiago, 2026. https://www.cochilco.cl/web/download/1014/2026/15677/proyeccion-de-consumo-de-energia-electrica-de-la-mineria-del-cobre-periodo-2025-2034.pdf

[3] G. Calvo, G. Mudd, A. Valero, and A. Valero, "Decreasing ore grades in global metallic mining: a theoretical issue or a global reality?," *Resources*, vol. 5, art. 36, 2016. https://doi.org/10.3390/resources5040036

[4] International Energy Agency, *Global Critical Minerals Outlook 2025*, chapter "Outlook for key minerals." Paris, 2025. https://www.iea.org/reports/global-critical-minerals-outlook-2025/outlook-for-key-minerals

[5] United States Department of Energy, Industrial Technologies Program, *Mining Industry Energy Bandwidth Study* (data year 2000). Washington, 2007. https://www.energy.gov/sites/prod/files/2013/11/f4/mining_bandwidth.pdf

[6] Jacob, Cooper, Gutowski, and Ramos-Grez, "The efficiency of copper ore comminution: a thermodynamic exergy analysis," *Minerals Engineering*, vol. 109, 2017. Record of the paper at the Pontificia Universidad Católica de Chile: https://ing.uc.cl/?p=49101

[7] Lacerda Gomes, Delboni Júnior, Chieregati, and Alves Jatobá, "Development of ball mill test for simulation of industrial circuit," *REM, Revista Escola de Minas*, vol. 69, no. 1, 2016. https://www.redalyc.org/pdf/564/56444608012.pdf

[8] Bailey, Lane, Morrell, and Staples, "What can go wrong in comminution circuit design?," in *Tenth Mill Operators' Conference*, AusIMM, 2009. https://www.ausimm.com/publications/conference-proceedings/tenth-mill-operators-conference-2009/what-can-go-wrong-in-comminution-circuit-design/

[9] A. Yamashita, A. Thivierge, and T. Euzébio, "A review of modeling and control strategies for cone crushers in the mineral processing and quarrying industries," *Minerals Engineering*, vol. 170, art. 107036, 2021. https://doi.org/10.1016/j.mineng.2021.107036

[10] R. Duarte, A. Yamashita, M. da Silva, L. Cota, and T. Euzébio, "Calibration and validation of a cone crusher model with industrial data," *Minerals*, vol. 11, art. 1256, 2021. https://doi.org/10.3390/min11111256

[11] B. K. N. Rocha, T. M. Campos, J. Silva, and L. M. Tavares, "Fit-for-purpose model of HP500 cone crusher in size reduction of itabirite iron ore," *Minerals*, vol. 14, no. 9, art. 919, 2024. CC BY 4.0. https://doi.org/10.3390/min14090919

[12] J. Quist and C. Evertsson, "Cone crusher modelling and simulation using DEM," *Minerals Engineering*, vol. 85, pp. 92–105, 2016. https://doi.org/10.1016/j.mineng.2015.11.004

[13] M. Johansson, J. Quist, M. Evertsson, and E. Hulthén, "Cone crusher performance evaluation using DEM simulations and laboratory experiments for model validation," *Minerals Engineering*, vol. 103–104, pp. 93–101, 2017. https://doi.org/10.1016/j.mineng.2016.09.015

[14] P. Cleary and M. Sinnott, "Simulation of particle flows and breakage in crushers using DEM: Part 1, compression crushers," *Minerals Engineering*, vol. 74, pp. 178–197, 2015. https://doi.org/10.1016/j.mineng.2014.10.021

[15] G. Delaney, R. Morrison, M. Sinnott, S. Cummins, and P. Cleary, "DEM modelling of non-spherical particle breakage and flow in an industrial scale cone crusher," *Minerals Engineering*, vol. 74, pp. 112–122, 2015. https://doi.org/10.1016/j.mineng.2015.01.013

[16] J. Quist, *Cone Crusher Modelling and Simulation*, thesis, Chalmers University of Technology, Göteborg, 2012. http://publications.lib.chalmers.se/records/fulltext/164816/164816.pdf

[17] Hulthén, *Real-Time Optimization of Cone Crushers*, doctoral thesis, Chalmers University of Technology, Göteborg, 2010. https://research.chalmers.se/en/publication/128844

[18] Evertsson, Bengtsson, Hulthén, and Quist, "Control systems for improvement of cone crusher production yield and fatigue life," in *Conference in Minerals Engineering*, 2015. https://research.chalmers.se/en/publication/212373

[19] Lindqvist and Evertsson, "Prediction of worn geometry in cone crushers," *Minerals Engineering*, vol. 16, no. 12, 2003. https://research.chalmers.se/en/publication/10218

[20] W. J. Whiten, "The simulation of crushing plants with models developed using multiple spline regression," *Journal of the South African Institute of Mining and Metallurgy*, vol. 72, pp. 257–264, 1972 (APCOM 1972). https://www.saimm.co.za/Conferences/Apcom72/317-Whiten.pdf

[21] T. J. Napier-Munn, S. Morrell, R. D. Morrison, and T. Kojovic, *Mineral Comminution Circuits: Their Operation and Optimisation*. Brisbane: Julius Kruttschnitt Mineral Research Centre, 1996. https://search.worldcat.org/title/37081193

[22] Andersen and Napier-Munn, "Power prediction for cone crushers," 1988. https://www.semanticscholar.org/paper/Power-prediction-for-cone-crushers-Andersen-Napier-Munn/2c60e81b909b65cba85b44583354dc8e310ec2f8

[23] Austin, Klimpel, and Luckie, *Process Engineering of Size Reduction: Ball Milling*, 1984. https://search.worldcat.org/title/11603131

[24] Rosin and Rammler, "The laws governing the fineness of powdered coal," 1933. https://cir.nii.ac.jp/crid/1572543024341127296

[25] C. M. Evertsson, "Modelling of flow in cone crushers," *Minerals Engineering*, vol. 12, no. 12, pp. 1479–1499, 1999. https://doi.org/10.1016/S0892-6875(99)00136-3

[26] C. M. Evertsson, *Cone Crusher Performance*, doctoral thesis, Chalmers University of Technology, Göteborg, 2000. https://research.chalmers.se/en/publication/759

[27] F. C. Bond, "The third theory of comminution," *Transactions of the American Institute of Mining, Metallurgical and Petroleum Engineers*, vol. 193, pp. 484–494, 1952. https://onemine.org/documents/the-third-theory-of-comminution

[28] S. Morrell, "Predicting the overall specific energy requirement of crushing, high pressure grinding roll and tumbling mill circuits," *Minerals Engineering*, vol. 22, no. 6, pp. 544–549, 2009. https://doi.org/10.1016/j.mineng.2009.01.005

[29] Global Mining Standards and Guidelines Group, *Morrell Method for Determining Comminution Circuit Specific Energy and Assessing Energy Utilization Efficiency of Existing Circuits*, Annex B, equations B4 and B5. https://www.smctesting.com/documents/Morrell_method_for_determining_comminution_circuit_specific_energy_and_assessing_energy_utilization_efficiency_of_existing_circuits.pdf

[30] E. J. Y. Koh, E. Amini, G. J. McLachlan, and N. Beaton, "Utilising a deep neural network as a surrogate model to approximate phenomenological models of a comminution circuit for faster simulations," *Minerals Engineering*, vol. 170, art. 107026, 2021. https://doi.org/10.1016/j.mineng.2021.107026

[31] M. C. Kennedy and A. O'Hagan, "Bayesian calibration of computer models," *Journal of the Royal Statistical Society: Series B*, vol. 63, no. 3, pp. 425–464, 2001. https://doi.org/10.1111/1467-9868.00294

[32] C. J. F. ter Braak, "A Markov chain Monte Carlo version of the genetic algorithm Differential Evolution: easy Bayesian computing for real parameter spaces," *Statistics and Computing*, vol. 16, pp. 239–249, 2006. https://doi.org/10.1007/s11222-006-8769-1

[33] C. J. F. ter Braak and J. A. Vrugt, "Differential Evolution Markov Chain with snooker updater and fewer chains," *Statistics and Computing*, vol. 18, pp. 435–446, 2008. https://doi.org/10.1007/s11222-008-9104-9

[34] J. Goodman and J. Weare, "Ensemble samplers with affine invariance," *Communications in Applied Mathematics and Computational Science*, vol. 5, no. 1, pp. 65–80, 2010. https://doi.org/10.2140/camcos.2010.5.65

[35] D. Foreman-Mackey, D. W. Hogg, D. Lang, and J. Goodman, "emcee: the MCMC hammer," *Publications of the Astronomical Society of the Pacific*, vol. 125, pp. 306–312, 2013. https://doi.org/10.1086/670067

[36] F. Santibáñez-Leal, "What ten industrial surveys identify in a cone-crusher population balance model: Bayesian calibration with structural discrepancy," preprint, Zenodo, 2026. https://doi.org/10.5281/zenodo.23843809

[37] A. Gröndahl, G. Asbjörnsson, E. Hulthén, and M. Evertsson, "Diagnostics of cone crusher feed segregation using power draw measurements," *Minerals Engineering*, vol. 127, pp. 15–21, 2018. https://doi.org/10.1016/j.mineng.2018.07.008

[38] M. Lindqvist and C. Evertsson, "Development of wear model for cone crushers," *Wear*, vol. 261, pp. 435–442, 2006. https://doi.org/10.1016/j.wear.2005.12.010

[39] Y. Ma, X. Fan, and Q. He, "Prediction of cone crusher performance considering liner wear," *Applied Sciences*, vol. 6, art. 404, 2016. https://doi.org/10.3390/app6120404

[40] Metso, *Nordberg HP Series Cone Crushers: Wear Parts Application Guide* (code 2602). https://www.metso.com/contentassets/a817531ccc6c422cb9b1df9a5e5b701d/application-guide-nordberg-hp-2602-en.pdf

[41] Metso, *Nordberg C Series Jaw Crushers* (brochure 4226-04-24), 2024. https://www.metso.com/globalassets/brochure-nordberg-c-series-4226-04-24-en-agg.pdf

[42] Metso, *Basics in Minerals Processing*, 8th ed., 2011. https://www.ausimm.com/globalassets/insights-and-resources/minerals-processing-toolbox/basics-in-minerals-processing_metso.pdf

[43] R. Liu, B. Shi, G. Li, and H. Yu, "Influence of operating conditions and crushing chamber on energy consumption of cone crusher," *Energies*, vol. 11, art. 1102, 2018. https://doi.org/10.3390/en11051102

[44] M. Lindqvist and H. Li, "A cone crusher model with a coupled energy- and breakage dependency," *Minerals Engineering*, vol. 172, art. 107147, 2021. https://doi.org/10.1016/j.mineng.2021.107147

[45] M. Bengtsson, "Modelling energy and size distribution in cone crushers," *Minerals Engineering*, vol. 139, art. 105869, 2019. https://doi.org/10.1016/j.mineng.2019.105869

[46] R. Salehzadeh Gharaei, A. Gholami, H. Khoshdast, and H. Biglari, "A multi-method machine learning framework for simulating power dynamics in an iron ore cone crusher plant," *Mineral Processing and Extractive Metallurgy Review*, vol. 46, pp. 783–798, 2024. https://doi.org/10.1080/08827508.2024.2395828

[47] L. M. Tavares, "Review and further validation of a practical single-particle breakage model," *KONA Powder and Particle Journal*, vol. 39, pp. 62–83, 2022. https://www.jstage.jst.go.jp/article/kona/39/0/39_2022012/_pdf

[48] G. Barrios, N. Jiménez-Herrera, S. Fuentes-Torres, and L. Tavares, "DEM simulation of laboratory-scale jaw crushing of a gold-bearing ore using a particle replacement model," *Minerals*, vol. 10, art. 717, 2020. https://doi.org/10.3390/min10080717

A Parameter tables

Table 20 gives the per-machine constants of the engine, generated from the engine's own modules: the classification constants of Equation (11), the design throw that enters the specific energy of Equation (7) and the throw term of K_2, the capacity constants of Equation (16), and the ranges of the setting and the speed over which the Latin-hypercube sweeps of the learned models draw each machine class (the sweeps also draw the throw, x_{63}, n and $A \times b$ over per-class ranges recorded in the surrogate contract). The circuit constants are typical values of their class; only the secondary cone is replaced, on the calibrated lane, by the fitted relations of Section 6.9. Table 21 collects the remaining constants of the engine, the evaluation and CrusherBayes, each with the module that holds it; every one of them is named in the methods where it acts. A reimplementation that changes any of them changes the numbers of the report, which is why they are listed with their exact values rather than rounded.

B The integrated discrepancy of CrusherBayes

The marginal likelihood of Equation (41) and the conditional of Equation (42) follow from the Gaussian structure of the model. For one target and one fold, write the residual of the engine on the m training

Table 20: Per-machine constants of the engine, generated from `classification.py`, `capacity.py`, `machines.py` and `sweep.py`: classification constants, design throw, reference capacity, optimum speed, reference gap, and the setting and speed ranges of the learned-model sweeps.

Machine	a_1 (mm)	b_1	a_2 (mm)	b_2	K_3	T_{ref} (mm)	Q_{ref} (t/h)	N_{opt} (rpm)	g_{ref} (mm)	CSS range (mm)	Speed range (rpm)
gyratory	0.6	0.95	10.0	2.10	2.0	30	4500	150	175	120–240	100–200
jaw	2.0	0.95	20.0	2.30	2.1	38	700	300	110	50–160	180–380
secondary cone	0.2	0.90	6.0	1.95	2.3	30	520	360	38	6–90	180–560
tertiary cone	0.1	0.85	3.0	1.75	2.5	16	240	400	16	4–22	220–600
short-head cone	0.1	0.82	2.4	1.65	2.7	16	200	480	12	4–16	380–600

Table 21: Further constants of the engine, of the leave-one-survey-out evaluation and of CrusherBayes, with their values as the code holds them and the module of `data-pipeline/pipeline/` that defines each.

Constant	Value	Module
Top of the sieve grid, number of classes, ratio	256 mm, 28, $\sqrt{2}$	`model/crusher/engine.py`, `sieve.py`
Readout sizes	1, 4, 8, 16, 32 mm	`model/crusher/engine.py`
Ill-conditioning threshold of κ	10^6	`model/crusher/engine.py`
Reduction ratio flagged as negligible	below 1.05	`model/crusher/engine.py`
Austin slopes γ, β	0.35, 4.2	`model/crusher/breakage.py`
A; energy coefficient; reference speed	60%; 0.033 kWh/t per mm; 350 rpm	`model/crusher/breakage.py`
Cap of t_{10}; clamp of φ	0.44; $[0.02, 0.98]$	`model/crusher/breakage.py`
Throw fraction of K_2; floor of K_2	0.5; 1.05 K_1	`model/crusher/classification.py`
Nip angle base (cones, gyratory; jaw); friction	22°; 19°; $\mu_f = 0.35$	`model/crusher/capacity.py`
HP500 throw and speed; feed exponent; blend W_i	34 mm, 360 rpm; 0.9; 9.0 kWh/t	`model/crusher/hp500.py`
HP500 support (setting, feed F_{80})	38 to 55 mm, 47 to 140 mm	`model/crusher/envelope.py`
Lane budgets (run time, trace size)	1500 ms, 256 KiB	`core/gate.py`
Units slack	one full range on each side	`core/unit_gate.py`
Residual clip; ridge penalty; band factor; seed	0.20; 1.0; 1.2816; 42	`hp500/loo.py`
Morrell K_s; K_2 (closed, open circuit)	55; 1.0, 1.19	`model/crusher/morrell.py`
Discrepancy prior sd τ_Q, τ_P	140 t/h, 36 kW	`hp500/crusherbayes.py`
Noise prior medians; log sd; log bounds	60 t/h, 15 kW; 1; $[\ln 1, \ln 1000]$	`hp500/crusherbayes.py`
Walkers; steps; burn-in; thinning; draws	20; 7000; 1000; 10; 20	`hp500/crusherbayes.py`
Moves (DE, DE-snooker); convergence factor; margin	0.8, 0.2; 50; 3 points	`hp500/crusherbayes.py`
Drift projection grid; bisection steps; slack	64; 60; 4 ulp	`model/crusher/` (drift functions)
Learned models: epochs; learning rate; weight decay	500 and 400; 2×10^{-3}; 10^{-5}	`stages/train.py`
Learned models: points per class; seeds; noise; opset	1400 and 480; 12345, 67890; 0.05; 17	`stages/train.py`

surveys as $\mathbf{r} = \mathbf{y} - f(\boldsymbol{\vartheta})$. By Equation (39), $\mathbf{r} = \mathrm{Z}\mathbf{d} + \boldsymbol{\varepsilon}$ with $\mathbf{d} \sim N(\mathbf{0}, \tau^2 \mathrm{I}_3)$ independent of $\boldsymbol{\varepsilon} \sim N(\mathbf{0}, \sigma^2 \mathrm{I}_m)$, so $\mathbf{r}$ is Gaussian with mean zero and covariance

$$\mathrm{Cov}[\mathbf{r}] = \mathrm{Z}\,\mathrm{Cov}[\mathbf{d}]\,\mathrm{Z}^\top + \mathrm{Cov}[\boldsymbol{\varepsilon}] = \tau^2 \mathrm{Z}\mathrm{Z}^\top + \sigma^2 \mathrm{I}_m = \boldsymbol{\Sigma}, \tag{47}$$

which is Equation (41). The pair $(\mathbf{d}, \mathbf{r})$ is jointly Gaussian with $\mathrm{Cov}[\mathbf{d}, \mathbf{r}] = \tau^2 \mathrm{Z}^\top$, so the conditional of $\mathbf{d}$ given $\mathbf{r}$ is Gaussian with

$$\mathbb{E}[\mathbf{d} \mid \mathbf{r}] = \tau^2 \mathrm{Z}^\top \boldsymbol{\Sigma}^{-1} \mathbf{r}, \qquad \mathrm{Cov}[\mathbf{d} \mid \mathbf{r}] = \tau^2 \mathrm{I}_3 - \tau^4\, \mathrm{Z}^\top \boldsymbol{\Sigma}^{-1} \mathrm{Z}, \tag{48}$$

which is Equation (42). The log density of Equation (41) is evaluated with the Cholesky factor L of $\boldsymbol{\Sigma}$ as $-\frac{1}{2}\|\mathrm{L}^{-1}\mathbf{r}\|^2 - \sum_k \ln L_{kk} - \frac{m}{2} \ln 2\pi$, and a covariance that is not positive definite returns minus infinity, so the sampler rejects that point. Because Z has three columns and $m = 9$ or 10 rows, $\boldsymbol{\Sigma}$ is a 9×9 or 10×10 matrix, and the integration replaces six sampled dimensions per fold by one factorisation per target and posterior evaluation, which keeps each evaluation at the cost of the engine runs. The code checks the marginal against a Monte Carlo of the model with the coefficients drawn explicitly.

C Contract schemas

The tables below are generated from the schema files of `docs/contracts/schemas/` and from the OpenAPI document by `make_tables.py`. Each row is one field: its dotted path (one level of nesting is expanded; `[]` marks the items of an array), its type, its range or allowed values as the schema states them, whether it is required at its level, and its description, shortened where it is long. A field described only by a reference to a shared definition carries the type of that definition. Rules that a schema cannot state, such as a count that must equal the length of a list or a path that must exist, are checked by the producer and consumer tests of each contract and are described in Section 4; the units of each field are in its description or in the subsection of that section. The catalog row of every contract, with its producer, its consumers, its swap test and its missing-data behaviour, is the authority when this appendix and the catalog differ.

params-record

Field	Type	Range or values	Req.	Meaning
case_id	string	pattern ^[A-Z][A-Z0-9-]{1,15}$	yes	The case identifier, for example S01 or HP500-01.
machine	enum	cone-sec, cone-tert, cone-short-head, gyratory, jaw	yes	The crusher kind; the engine's per-machine constants follow it.
css_mm	number	> 0; <= 600	yes	Closed-side setting, the narrowest gap between the mantle and the concave (or the jaws), mm.
throw_mm	number	>= 1; <= 100	yes	Eccentric throw, the open-side setting minus the closed-side setting, mm.
speed_rpm	number	>= 50; <= 1000	yes	Eccentric (gyration) speed, revolutions per minute.
feed_x63_mm	number	> 0; <= 2000	yes	Rosin-Rammler characteristic size of the feed (63.2 percent passing), mm.
feed_m	number	>= 0.3; <= 4	yes	Rosin-Rammler uniformity exponent of the feed, dimensionless.
ore_axb	number	>= 10; <= 1000	yes	JKMRC drop-weight ore competence A times b, dimensionless; lower is harder.
ore_wi	number	>= 2; <= 40	yes	Bond work index of the ore, kWh per tonne.

hp500-survey

Field	Type	Range or values	Req.	Meaning
survey	integer	>= 1; <= 10	yes	The survey number as published.
f80_mm	number	> 0; <= 400	yes	Measured feed 80 percent passing size, mm.
css_mm	number	> 0; <= 200	yes	Measured closed-side setting, mm.
feed_tph	number	> 0; <= 5000	yes	Measured solid feed rate, tonnes per hour.
power_kw	number	> 0; <= 1000	yes	Crusher power estimated from the measured current, kW.

case-index

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.index/v1	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	The product version that baked the index; older than VERSION means the bake is stale.
n_cases	integer	>= 1	yes	–
default_case	string	–	yes	–
cases	array of object	1 to n items	yes	–
cases[].case_id	string	–	yes	–
cases[].title	object	–	yes	–
cases[].category	object	–	yes	–
cases[].real_or_synthetic	enum	real, synthetic	yes	real: one of the measured HP500 surveys; synthetic: a circuit case of the engine.
cases[].manifest_path	string	pattern ^manifests/[^/\\]+\.json$	yes	–
files	array of string	items pattern ^(?!/)(?!.*\.\.)[^\\]+\.(json|onnx)$	yes	The further committed files the site serves (the model files, the HP500 validation documents, the model metrics), relative to data/derived; ...

case-manifest

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.manifest/v1	yes	–
case_id	string	–	yes	–
title	object	–	yes	–
title.en	string	–	yes	–
title.es	string	–	yes	–
category	object	–	yes	–
category.en	string	–	yes	–
category.es	string	–	yes	–
stage	enum	primary, secondary, tertiary, control	yes	The circuit stage the case belongs to.
control	enum	null, negative, invalid, calibration	yes	For a control case, what it controls; null otherwise.
real_or_synthetic	enum	real, synthetic	yes	–

Field	Type	Range or values	Req.	Meaning
expected_band	object	–	yes	What a crushing engineer should see, in words; expect states the same in numbers.
expected_band.en	string	–	yes	–
expected_band.es	string	–	yes	–
context	object	–	yes	The case write-up shown in the workbench Context: the question, the variables that matter and how to read the views.
context.problem	object	–	yes	–
context.variables	object	–	yes	–
context.reading	object	–	yes	–
engine	object	–	yes	–
engine.package	const	pipeline.model.crusher	yes	–
engine.version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
engine.model	string	–	yes	–
params	object	–	yes	The operating point the case was baked at (contract 1 accepted it); calibrated is true only on the HP500 surveys.
params.machine	enum	cone-sec, cone-tert, cone-short-head, gyratory, jaw	yes	–
params.css_mm	number	–	yes	–
params.throw_mm	number	–	yes	–
params.speed_rpm	number	–	yes	–
params.feed_x63_mm	number	–	yes	–
params.feed_m	number	–	yes	–
params.ore_axb	number	–	yes	–
params.ore_wi	number	–	yes	–
params.calibrated	boolean	–	yes	–
measured	object	–	no	Only on a real case: the survey row exactly as contract hp500-survey accepted it.
measured.survey	integer	>= 1	yes	–
measured.f80_mm	number	–	yes	–
measured.css_mm	number	–	yes	–
measured.feed_tph	number	–	yes	–
measured.power_kw	number	–	yes	–
seed	integer	–	yes	–
artifact	object	–	yes	–
artifact.path	string	pattern ^(?!/)(?!.*\.\.)[^\\]+\.json$	yes	Relative to data/derived; no leading slash, no '..', no backslash.
artifact.format	const	json	yes	–
artifact.trace_schema	const	chancadem.trace/v1	yes	–
artifact.bytes	integer	>= 1	yes	–
artifact.sha256	string	pattern ^[0-9a-f]{64}$	yes	SHA-256 of the trace file bytes as committed.
lane	enum	live, precompute	yes	–
gate	object	–	yes	The measured lane decision of data-pipeline/pipeline/core/gate.py classify_lane: the engine run is timed in the bake (the median of several warmed ...
gate.lane	enum	live, precompute	yes	–
gate.pure_python	boolean	–	yes	–
gate.wheels	array of string	–	yes	–
gate.trace_bytes	integer	>= 1	yes	–
gate.run_ms_budget	number	> 0	yes	–
gate.trace_bytes_budget	integer	>= 1	yes	–
gate.reasons	array of string	–	yes	–
flags	array of enum	items css_ge_feed_top, pass_through, negligible_reduction, ill_conditioned, css_nonpositive, feed_nonpositive, f80_lt_css, css_outside_calibrated, f80_outside_calibrated, css_below_vendor_minimum, reduction_ratio_above_typical	yes	Stable codes of the engine's flags and envelope codes for this case, in the engine's order: css_ge_feed_top (nothing is gripped), pass_through ...
metrics	object	–	yes	The case's result scalars (p80_mm, p50_mm, p20_mm, f80_mm, reduction_ratio, throughput_tph, power_kw, specific_energy_kwh_t) and, on a real case, ...
models	object	–	yes	Per model slot (catalog row id), what answers in this bake: the released ONNX model (real) or the slot's baseline.
expect	object	–	yes	The checked expectation of the case: [low, high] per result metric; the bake fails outside it.
source	any	–	yes	Where the case comes from: the authored circuit matrix, or the publication of a measured survey.

trace

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.trace/v1	yes	–
case_id	string	–	yes	–
sieve_mm	array of number	2 to 200 items	yes	Sieve aperture edges, coarse to fine, mm (the engine grid: 256 mm, square-root-of-two series, 29 edges).
mid_mm	array of number	2 to 200 items	yes	Geometric mid-size of each size class, mm.
feed_passing	array of number	2 to 200 items	yes	Feed cumulative fraction passing at each edge.
product_passing	array of number	2 to 200 items	yes	Product cumulative fraction passing at each edge.
feed_mass	array of number	2 to 200 items	yes	Feed mass fraction in each class (sums to 1).
product_mass	array of number	2 to 200 items	yes	Product mass fraction in each class (sums to 1 within the mass-closure tolerance).
classification	array of number	2 to 200 items	yes	Whiten classification function C(d) at each class mid-size: the probability a particle of that size is selected for breakage.
appearance_t10	number	>= 0; <= 100	yes	The t10 of the breakage appearance function used, percent passing one tenth of the parent size.
summary	object	–	yes	–
summary.f80_mm	number	>= 0	yes	–
summary.p80_mm	number	>= 0	yes	–
summary.p50_mm	number	>= 0	yes	–
summary.p20_mm	number	>= 0	yes	–
summary.reduction_ratio	number	>= 0	yes	F80 over P80.
summary.pct_passing	object	–	yes	Product fraction passing at 1, 4, 8, 16 and 32 mm.
summary.throughput_tph	number	>= 0	yes	–
summary.power_kw	number	>= 0	yes	–
summary.specific_energy_kwh_t	number	>= 0	yes	–
summary.oss_mm	number	>= 0	yes	Open-side setting, the closed-side setting plus the throw, mm.
summary.nip_angle_deg	number	>= 0	yes	–
summary.nip_limit_deg	number	>= 0	yes	–
summary.regime	enum	choke, trickle, pass-through, invalid	yes	–
summary.valid	boolean	–	yes	–
summary.out_of_envelope	boolean	–	yes	–
summary.implausible	boolean	–	yes	–
summary.mass_closure	number	>= 0	yes	–
summary.cond_estimate	number or string	–	yes	Conditioning proxy of (I - B C); the string Infinity when the solve is singular.
speed_sweep	object	–	yes	The capacity hump: the engine at the case's point with only the eccentric speed varied over the machine's range.
speed_sweep.speed_rpm	array of number	2 to 200 items	yes	–
speed_sweep.throughput_tph	array of number	2 to 200 items	yes	–
speed_sweep.power_kw	array of number	2 to 200 items	yes	–
css_sweep	object	–	yes	The setting trade-off: the engine at the case's point with only the closed-side setting varied.
css_sweep.css_mm	array of number	2 to 200 items	yes	–
css_sweep.p80_mm	array of number	2 to 200 items	yes	–
css_sweep.throughput_tph	array of number	2 to 200 items	yes	–
css_sweep.power_kw	array of number	2 to 200 items	yes	–

hp500-backbone

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.hp500.backbone/v1	yes	–
source	object	–	yes	–
source.paper	string	–	yes	–
source.doi	const	10.3390/min14090919	yes	–
source.license	const	CC BY 4.0	yes	–
source.table	string	–	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
note	string	–	yes	–
n	integer	>= 1	yes	–

Field	Type	Range or values	Req.	Meaning
surveys	array of object	1 to n items	yes	–
surveys[].id	string	pattern ^HP500-[0-9]{2}$	yes	–
surveys[].n	integer	>= 1	yes	–
surveys[].measured	object	–	yes	–
surveys[].backbone	object	–	yes	–

hp500-loo

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.hp500.loo/v1	yes	–
source	object	–	yes	–
source.paper	string	–	yes	–
source.doi	const	10.3390/min14090919	yes	–
source.license	const	CC BY 4.0	yes	–
source.table	string	–	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
n	integer	>= 3	yes	–
seed	integer	–	yes	–
config	object	–	yes	–
config.resid_clip_frac	number	>= 0	yes	–
config.ridge_lambda	number	>= 0	yes	–
config.features	array of string	–	yes	–
config.targets_primary	array of enum	items tph, kW	yes	–
config.target_secondary	array of const	items p80	yes	–
config.morrell	object	–	no	–
loo	object	–	yes	–
loo.tph	object	–	yes	–
loo.kW	object	–	yes	–
loo.p80	object	–	yes	–
power_parity	object	–	yes	–
power_parity.note	string	–	yes	–
power_parity.bond_pd_range_kw	array	2 to 2 items	yes	–
power_parity.measured_range_kw	array	2 to 2 items	yes	–
power_parity.bare_bond	object	–	yes	–
power_parity.engine_pc_1p30_110	object	–	yes	–
power_parity.refit_pc	object	–	yes	–
power_parity.constant_mean	object	–	yes	–
power_parity.per_survey	array of object	–	yes	–
tph_parity	object	–	yes	–
tph_parity.note	string	–	yes	–
tph_parity.backbone	object	–	yes	–
tph_parity.per_survey	array of object	–	yes	–

hp500-crusherbayes

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.hp500.crusherbayes/v1	yes	–
source	object	–	yes	–
source.paper	string	–	yes	–
source.doi	const	10.3390/min14090919	yes	–
source.license	const	CC BY 4.0	yes	–
source.table	string	–	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
n	integer	>= 3	yes	–
seed	integer	–	yes	–
inputs_sha256	string	pattern ^[0-9a-f]{64}$	yes	SHA-256 over the survey table, the sampler size, the seed and the priors the document was sampled from.
method	object	–	yes	–
method.sampler	const	emcee	yes	–
method.sampler_version	string	–	yes	–
method.move	string	–	yes	–
method.walkers	integer	>= 2	yes	–
method.steps	integer	>= 1	yes	–

Field	Type	Range or values	Req.	Meaning
method.burn_in_steps	integer	>= 0	yes	–
method.thin	integer	>= 1	yes	–
method.predictive_draws_per_sample	integer	>= 1	yes	–
method.likelihood	string	–	yes	–
method.discrepancy_prior_sd	object	–	yes	–
method.noise_prior	object	–	yes	–
method.p80	string	–	yes	–
method.interval_80	string	–	yes	–
priors	array of object	1 to n items	yes	–
priors[].name	string	–	yes	–
priors[].unit	string	–	yes	–
priors[].nominal	number	–	yes	–
priors[].log	boolean	–	yes	–
priors[].mu	number	–	yes	–
priors[].sd	number	> 0	yes	–
priors[].lo	number	–	yes	–
priors[].hi	number	–	yes	–
priors[].source	string	–	yes	–
convergence	object	–	yes	–
convergence.criterion	string	–	yes	–
convergence.tau_factor	integer	>= 1	yes	–
convergence.folds	array of object	2 to n items	yes	–
convergence.all_converged	boolean	–	yes	–
loo	object	–	yes	–
loo.tph	object	–	yes	–
loo.kW	object	–	yes	–
comparators	object	–	yes	–
comparators.tph	object	–	yes	–
comparators.kW	object	–	yes	–
comparators_note	string	–	yes	–
contraction	array of object	1 to n items	yes	–
contraction[].name	string	–	yes	–
contraction[].unit	string	–	yes	–
contraction[].nominal	number	–	yes	–
contraction[].prior_sd	number	> 0	yes	–
contraction[].posterior_mean	number	–	yes	–
contraction[].posterior_sd	number	>= 0	yes	–
contraction[].contraction	number	>= 0	yes	–
contraction_note	string	–	yes	–
kill_criterion	object	–	yes	–
kill_criterion.rule	string	–	yes	–
kill_criterion.margin_mape_pts	number	>= 0	yes	–
kill_criterion.coverage_band	array of number	2 to 2 items	yes	–
kill_criterion.power_mape_pct	number	>= 0	yes	–
kill_criterion.power_mape_blin_pct	number	>= 0	yes	–
kill_criterion.power_gain_over_blin_pts	number	–	yes	–
kill_criterion.a_power_beats_blin_by_margin	boolean	–	yes	–
kill_criterion.tph_mape_pct	number	>= 0	yes	–
kill_criterion.tph_mape_m0_pct	number	>= 0	yes	–
kill_criterion.tph_gain_over_m0_pts	number	–	yes	–
kill_criterion.a_tph_beats_m0_by_margin	boolean	–	yes	–
kill_criterion.a_holds	boolean	–	yes	–
kill_criterion.coverage	object	–	yes	–
kill_criterion.coverage_outside_band	object	–	yes	–
kill_criterion.mean_width	object	–	yes	–
kill_criterion.mean_width_crushercal	object	–	yes	–
kill_criterion.band_wider_than_crushercal	object	–	yes	–
kill_criterion.b_holds	boolean	–	yes	–
kill_criterion.verdict	enum	kept, dropped as a predictor	yes	–

surrogate-model

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.surrogate/v1	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	The product version whose engine produced the training labels.
machine_order	array	5 to 5 items	yes	–
input_order	array	6 to 6 items	yes	–
in_mean	array of number	6 to 6 items	yes	–
in_std	array of number	6 to 6 items	yes	–
output_order	array	10 to 10 items	yes	–
out_log	array of boolean	10 to 10 items	yes	–
out_mean	array of number	10 to 10 items	yes	–
out_std	array of number	10 to 10 items	yes	–
graphs	object	–	yes	–
graphs.float32	object	–	yes	–
graphs.int8	object	–	yes	–
architecture	string	–	yes	The layer sizes and activations, for example 11-64-64-32-10, SiLU.
training	object	–	yes	–
training.seed	integer	–	yes	–
training.n_train	integer	>= 1	yes	–
training.n_test	integer	>= 1	yes	–
training.sampler	string	–	yes	How the operating points were drawn, with the bounds per input; the test draw is independent of the training draw.
training.input_ranges	object	–	yes	–
training.torch_version	string	–	yes	–
training.export_parity_max_abs	number	>= 0	no	The maximum absolute difference between the trained torch module and the exported float32 graph (run through onnxruntime) on the held-out draw, in ...

anomaly-model

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.anomaly/v1	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
feature_order	array	14 to 14 items	yes	–
mean	array of number	14 to 14 items	yes	–
std	array of number	14 to 14 items	yes	–
threshold_p99	number	> 0	yes	–
train_recon_mean	number	>= 0	yes	–
graph	object	–	yes	–
graph.path	const	models/psd-ae.onnx	yes	–
graph.bytes	integer	>= 1; <= 5242880	yes	–
graph.sha256	string	pattern ^[0-9a-f]{64}$	yes	–
graph.opset	integer	>= 13	yes	–
graph.input	const	x	yes	–
graph.output	const	xr	yes	–
architecture	string	–	yes	The layer sizes, for example 14-16-6-16-14, and the input noise used in training.
training	object	–	yes	–
training.seed	integer	–	yes	–
training.n_train	integer	>= 1	yes	–
training.noise_std	number	>= 0	yes	–
training.input_ranges	object	–	yes	–
training.torch_version	string	–	yes	–
training.export_parity_max_abs	number	>= 0	no	The maximum absolute difference between the trained torch module's reconstruction and the exported graph's (run through onnxruntime) on the ...

model-metrics

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.model-metrics/v1	yes	–
engine_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
surrogate	object	–	yes	–
surrogate.n_train	integer	>= 1	yes	–

Field	Type	Range or values	Req.	Meaning
surrogate.n_test	integer	>= 1	yes	–
surrogate.test_seed	integer	–	yes	–
surrogate.per_output	object	–	yes	–
surrogate.p80_monotone_vs_css	boolean	–	yes	–
surrogate.parity	object	–	yes	–
surrogate.note	string	–	yes	–
anomaly	object	–	yes	–
anomaly.n_heldout	integer	>= 1	yes	–
anomaly.false_alarm_rate	number	>= 0; <= 1	yes	–
anomaly.n_planted	integer	>= 1	yes	–
anomaly.detection_rate	number	>= 0; <= 1	yes	–
anomaly.envelope_false_alarm_rate	number	>= 0; <= 1	yes	–
anomaly.envelope_detection_rate	number	>= 0; <= 1	yes	–
anomaly.planted	string	–	yes	How the off-manifold points were planted (which inputs moved outside the training bounds, and by how much).
anomaly.note	string	–	yes	–

readiness

Field	Type	Range or values	Req.	Meaning
schema	const	chancadem.readiness/v1	yes	–
product_version	string	pattern ^\d+\.\d{2}\.\d{3}$	yes	–
catalog_sha256	string	pattern ^[0-9a-f]{64}$	yes	SHA-256 of docs/contracts/catalog.md (its UTF-8 bytes with CRLF normalised to LF) the document was computed from.
contracts	array of object	1 to n items	yes	–
contracts[].id	string	pattern ^[a-z0-9][a-z0-9-]*$	yes	The catalog row id.
contracts[].version	string	pattern ^\d+\.\d+\.\d+$	yes	The catalog row's contract version (semver).
contracts[].status	enum	real, active	yes	The catalog's design status: real once the real producer serves the row, active while its provider does.
contracts[].provenance	enum	real, stub, simulator, fixture, baseline, null	yes	What actually served the row in this build; null when nothing did (state missing).
contracts[].state	enum	available, partial, stale, provider, training, missing	yes	–
contracts[].reason	any	–	yes	Why the state is not available; null only when it is.
contracts[].missing	array of string	–	yes	For partial: what is absent (file paths, rejected record ids). Empty otherwise.

api (OpenAPI routes)

Route	Responses	Meaning
GET /health	200	Liveness.
GET /healthz	200	Liveness (alias of /health).
GET /api/cases	200, 503	The case index (catalog row case-index).
GET /api/cases/{case_id}/manifest	200, 404, 503	One case manifest (catalog row case-manifest).
GET /api/artifacts/{rel_path}	200, 404, 503	An artifact a manifest names (catalog row trace), by its path under data/derived.
GET /api/readiness	200, 503	What is real in this build (catalog row readiness).